

Why we use Heatmap

A heatmap shows values using colors. It helps quickly understand patterns, correlations, and intensity of values.

When to use

Use heatmap when you want to visualize:

Correlation matrix between variables

Large matrix data

Relationship strength between features

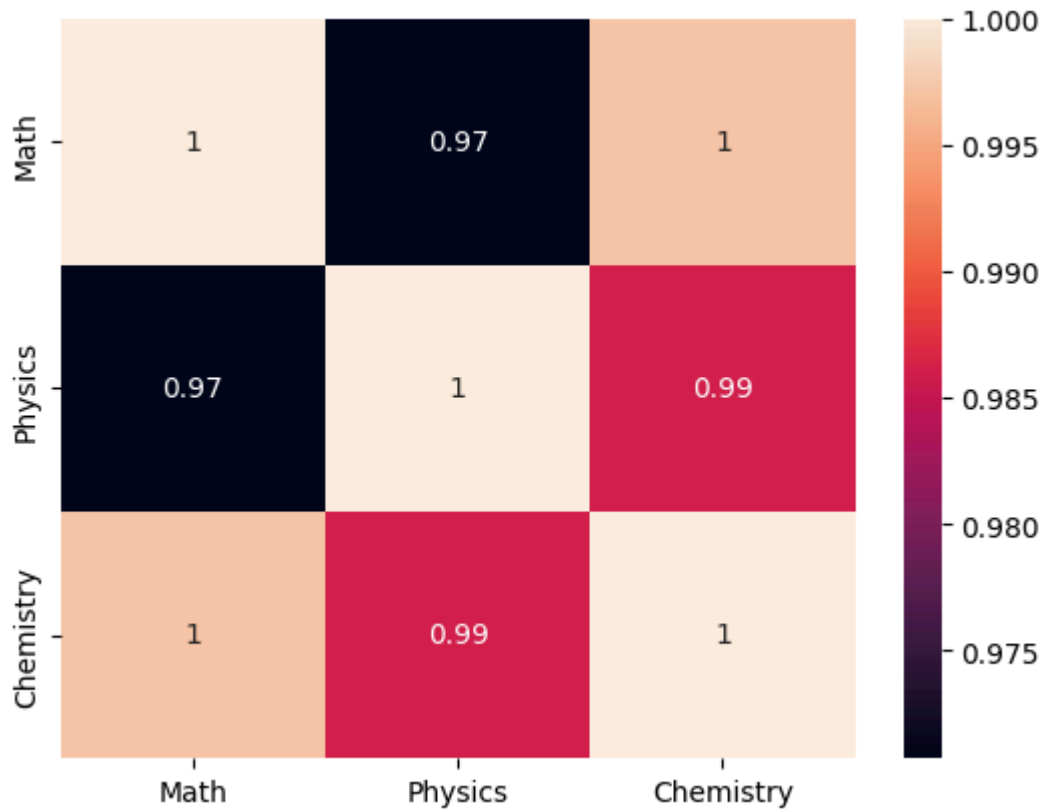
Missing values or density patterns

```
In [94]: import seaborn as sns
import pandas as pd

data = pd.DataFrame({
    "Math": [80, 90, 70],
    "Physics": [85, 95, 60],
    "Chemistry": [78, 88, 65]
})

sns.heatmap(data.corr(), annot=True)
```

```
Out[94]: <Axes: >
```



`seaborn.heatmap(data, vmin=None, vmax=None, cmap=None, linewidths=0, linecolor='white', annot=True, cbar=True)`

Parameter	Meaning
<code>data</code>	matrix or correlation table to visualize
<code>vmin</code>	minimum value for color scale
<code>vmax</code>	maximum value for color scale

Parameter	Meaning
cmap	color palette used in heatmap
linewidths	width of grid lines between cells
linecolor	color of grid lines
annot=True	display numbers inside each cell
cbar=True	show color scale bar

Heatmap showing correlation between engagement metrics.

Variables used: likes, comments, shares

Relationship	Meaning
likes – comments ≈ 0.30	weak positive relation
likes – shares ≈ 0.24	weak positive relation
comments – shares ≈ 0.25	weak positive relation

```
In [95]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

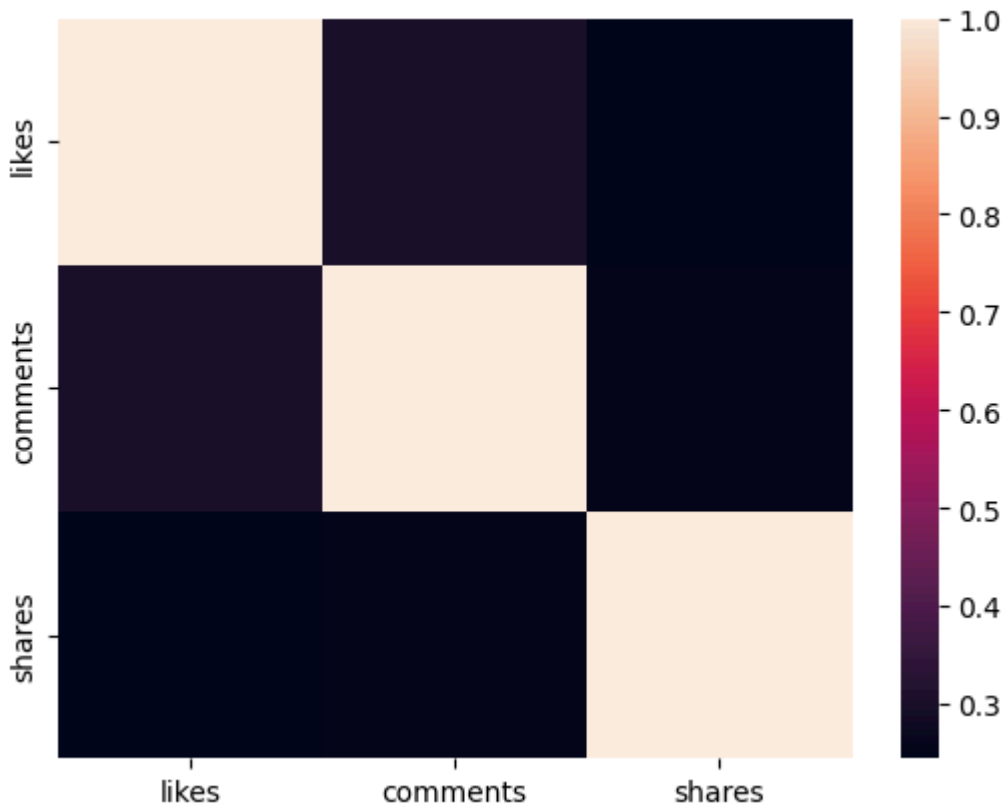
df = pd.read_csv("social_media_engagement.csv")

engagement = df[["likes","comments","shares"]]

sns.heatmap(engagement.corr(numeric_only=True))
```

Out[95]: <Axes: >

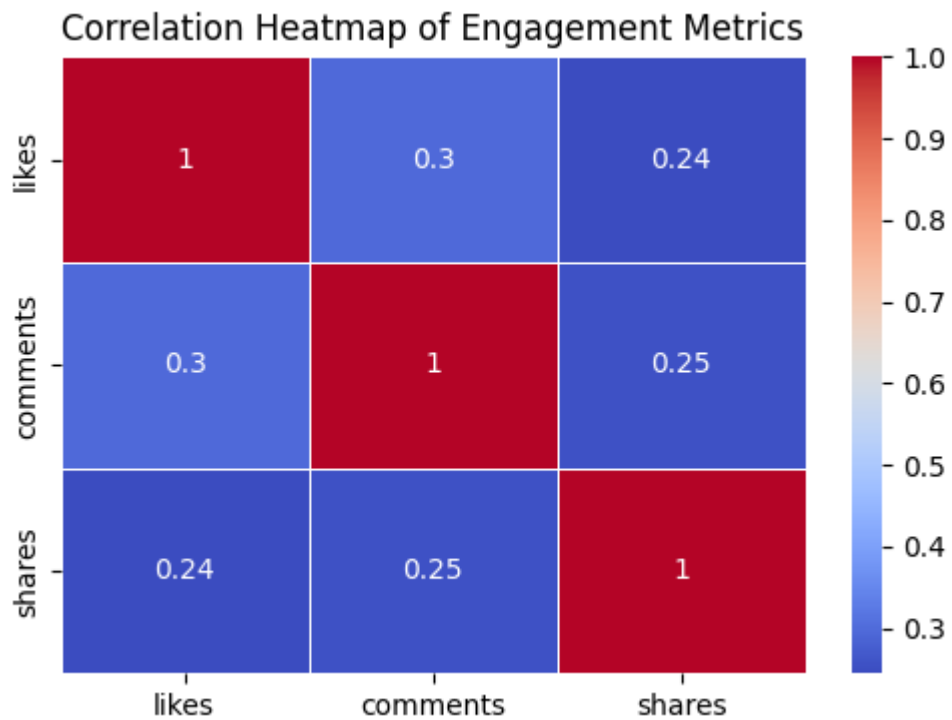
Vishal Acharya



```
In [96]: plt.figure(figsize=(6,4))

sns.heatmap(
    engagement.corr(numeric_only=True),    # data
    vmin=None,                            # minimum value of color scale
    vmax=None,                            # maximum value of color scale
    cmap="coolwarm",                      # color palette
    linewidths=0.5,                       # width of grid lines
    linecolor="white",                    # grid line color
    annot=True,                           # show values inside cells
    cbar=True                             # show color bar
)

plt.title("Correlation Heatmap of Engagement Metrics")
plt.show()
```



Heatmap of:

Platform vs Day → Average Likes

Platform	Insight
Instagram	highest average likes
Facebook	medium engagement
Twitter	lowest engagement

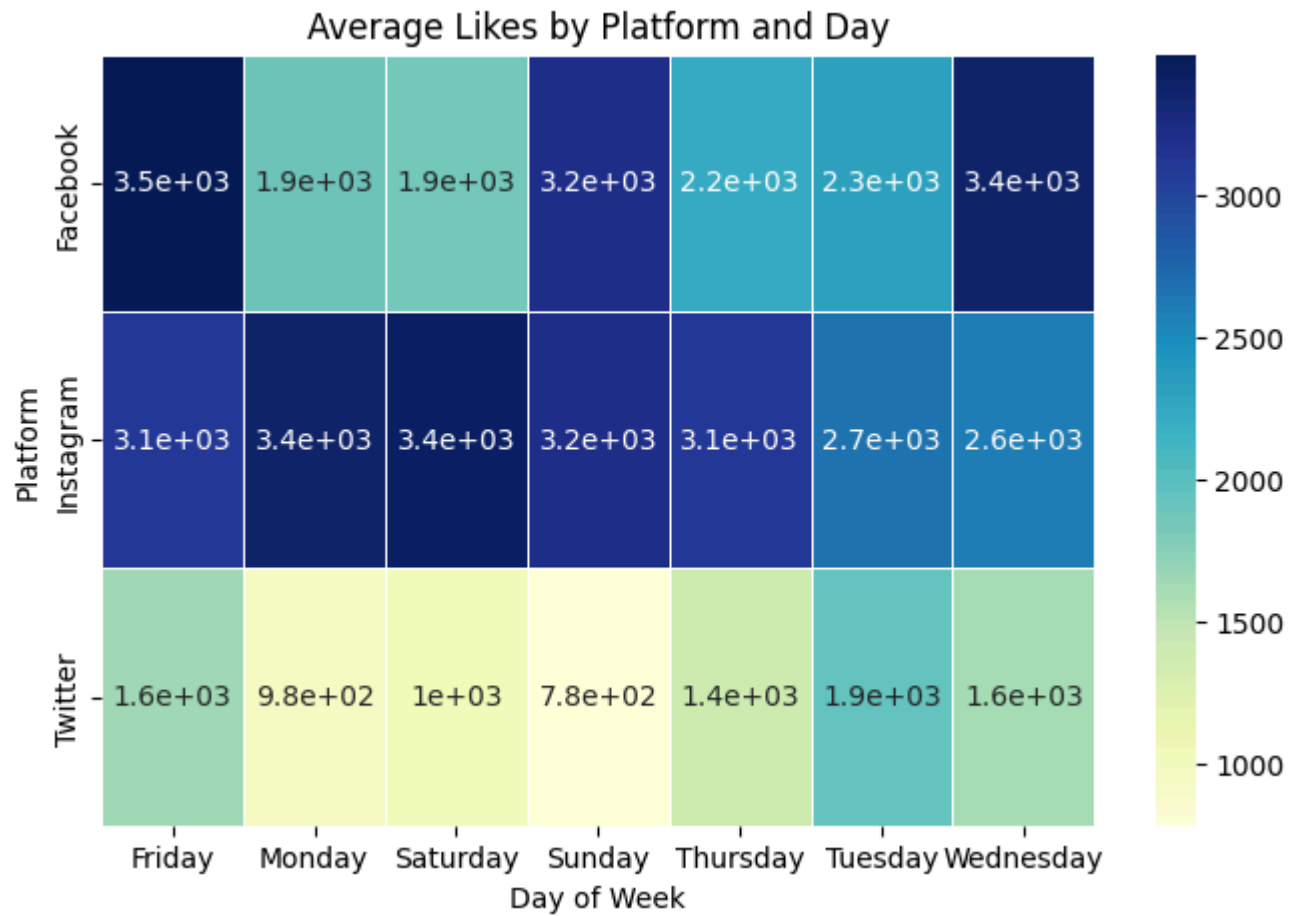
This type of heatmap is often used for business analytics dashboards.

```
In [98]: data = df.groupby(["platform", "post_day"])["likes"].mean().unstack()
plt.figure(figsize=(8,5))

sns.heatmap(
    data,
    vmin=None,
    vmax=None,
    cmap="YlGnBu",
    linewidths=0.5,
    linecolor="white",
    annot=True,
    cbar=True
)

plt.title("Average Likes by Platform and Day")
plt.xlabel("Day of Week")
plt.ylabel("Platform")

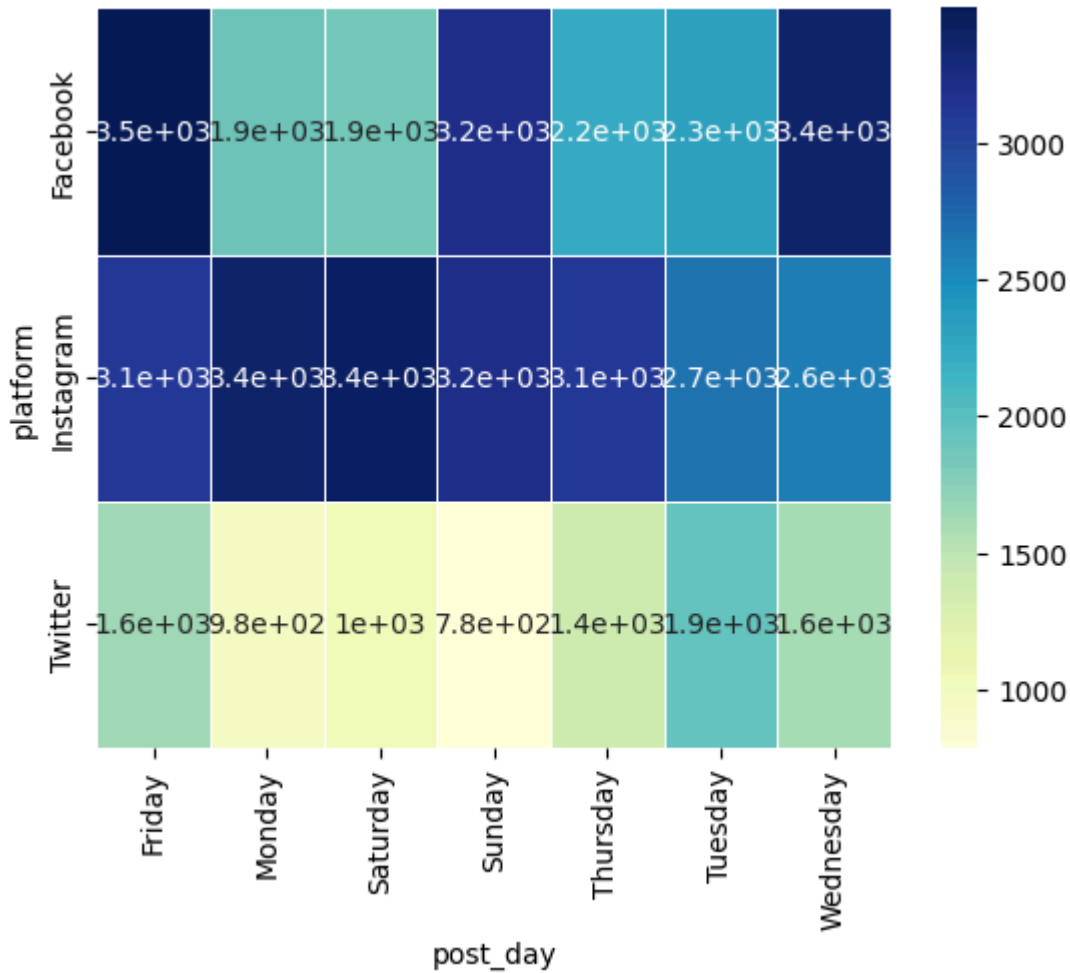
plt.show()
```



```
In [99]: data = df.groupby(["platform", "post_day"])["likes"].mean().unstack()

sns.heatmap(data, annot=True, cmap="YlGnBu", linewidths=0.5, linecolor="white", cbar=True)
```

```
Out[99]: <Axes: xlabel='post_day', ylabel='platform'>
```



Boxplot (Seaborn)

```
sns.boxplot(x, y, hue, data, color)
```

Parameter	Meaning
x	categorical variable on x-axis
y	numeric variable to analyze distribution

Parameter	Meaning
hue	subgroup category inside each x category
data	dataset used
color	color of the boxes

Element	Meaning
Middle line	median
Box	Q1–Q3 (interquartile range)
Whiskers	range of values
Dots	outliers

What is a Box Plot?

A Box Plot (Box-and-Whisker Plot) is a statistical graph used to show the distribution of numerical data and detect outliers.

It summarizes data using five key statistics.

Measure	Meaning
Minimum	Smallest value (excluding outliers)
Q1 (First Quartile)	25% of data lies below
Median (Q2)	Middle value (50%)
Q3 (Third Quartile)	75% of data lies below
Maximum	Largest value (excluding outliers)

IQR Method (Interquartile Range) Formula $Q1 = \text{quantile}(0.25)$

$Q3 = \text{quantile}(0.75)$

$IQR = Q3 - Q1$

IQR measures the spread of the middle 50% of the data.

Outlier Detection using IQR

Lower and upper limits are calculated as:

Lower Bound = $Q1 - 1.5 \times IQR$

Upper Bound = $Q3 + 1.5 \times IQR$

Rule

Values below Lower Bound → Outliers

Values above Upper Bound → Outliers

Boxplot Example 1

Platform vs Likes

Interpretation:

Platform	Observation
Instagram	highest median likes
Facebook	moderate engagement
Twitter	lowest engagement

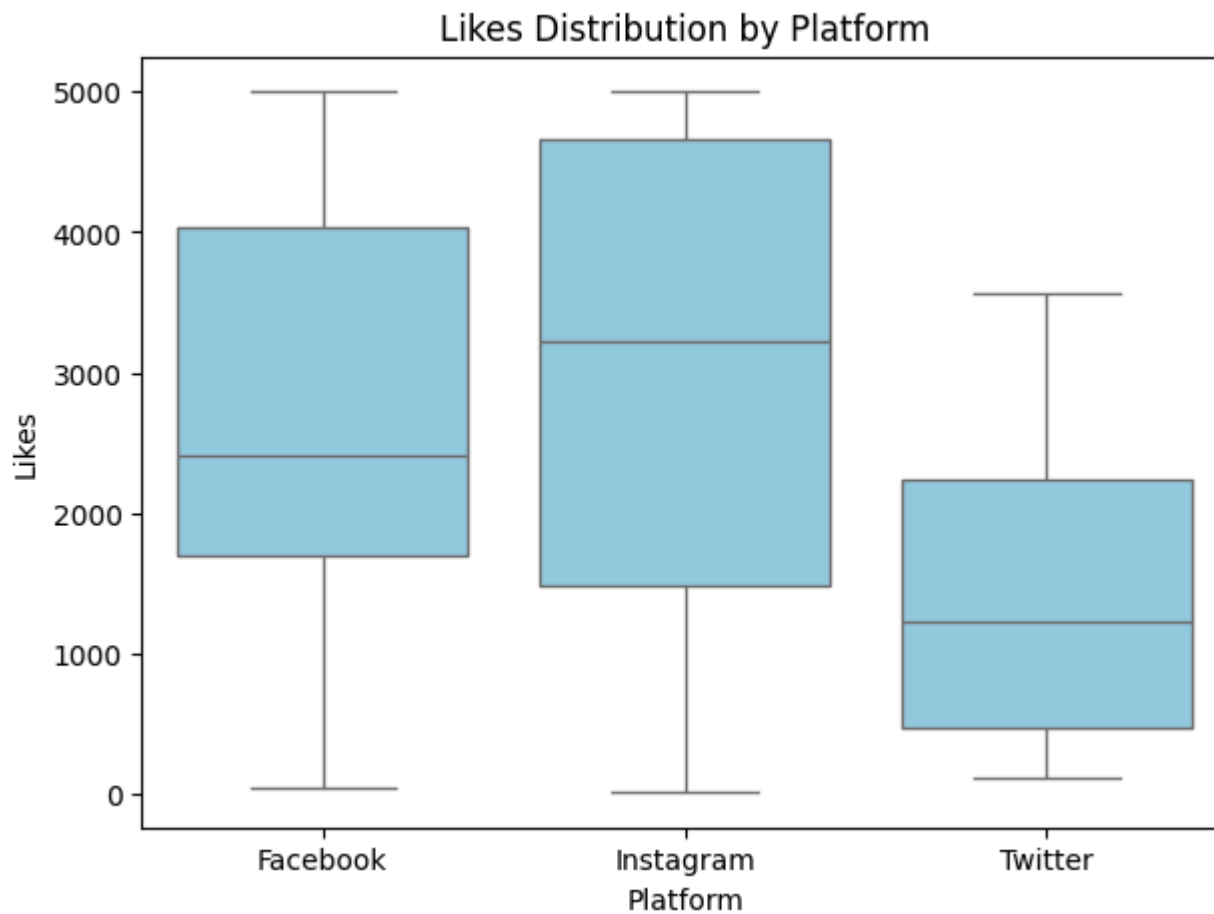
In [100...

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("social_media_engagement.csv")

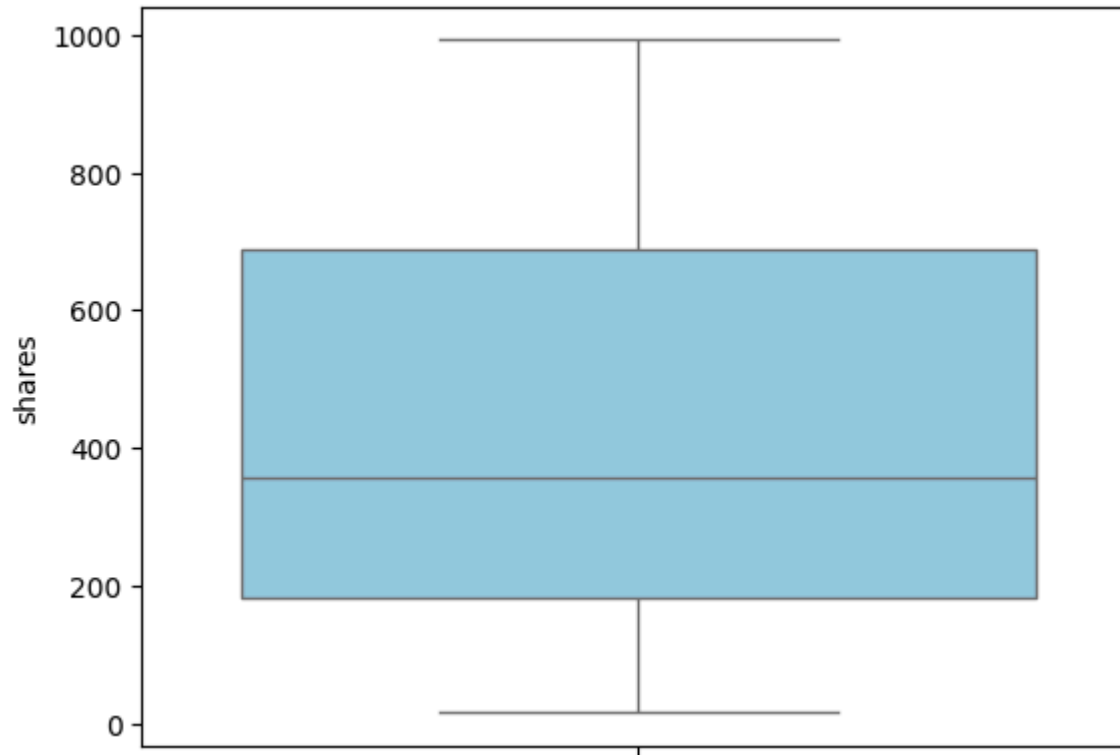
plt.figure(figsize=(7,5))
```

```
sns.boxplot(  
    x="platform",  
    y="likes",  
    data=df,  
    color="skyblue"  
)  
  
plt.title("Likes Distribution by Platform")  
plt.xlabel("Platform")  
plt.ylabel("Likes")  
  
plt.show()
```



```
In [103... sns.boxplot(  
    y="shares",  
    data=df,  
    color="skyblue"  
)
```

```
Out[103... <Axes: ylabel='shares'>
```



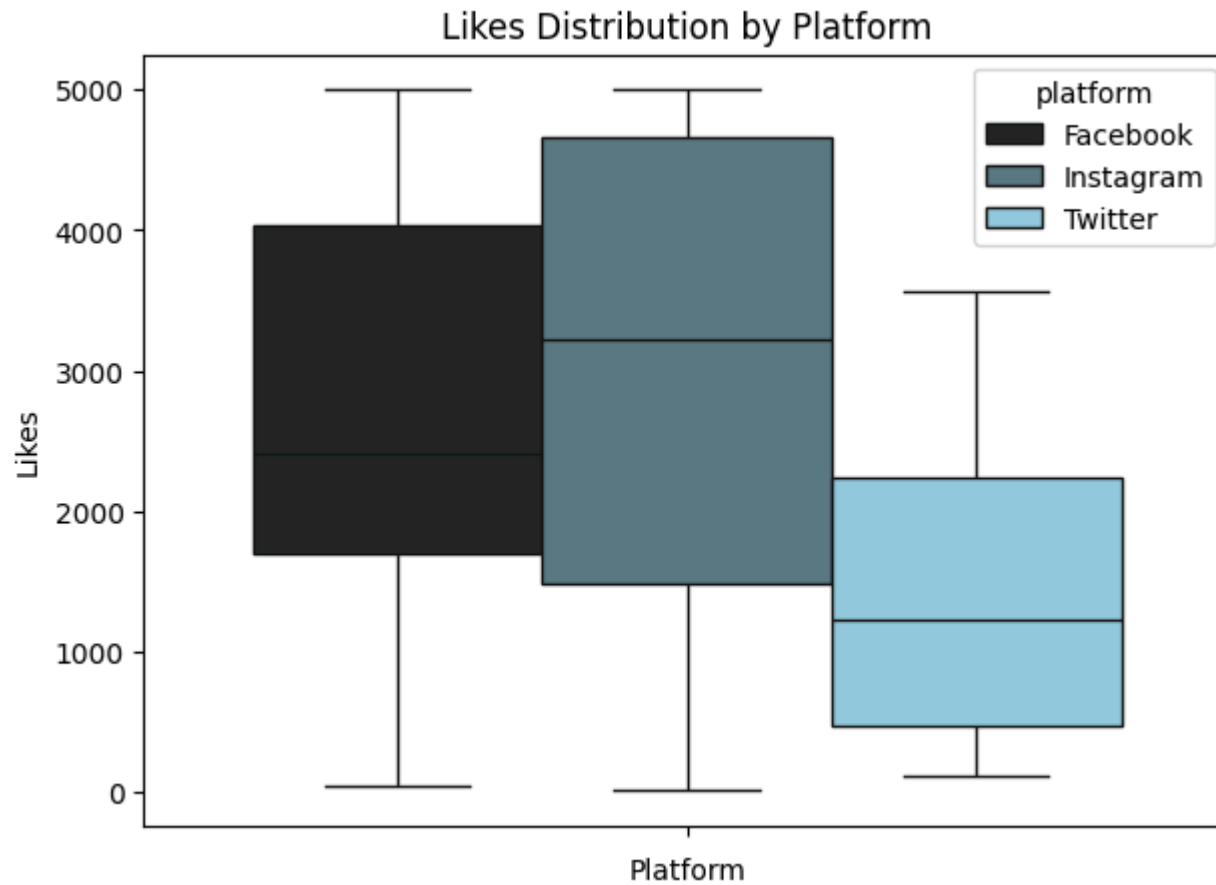
```
In [104... import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
df = pd.read_csv("social_media_engagement.csv")  
  
plt.figure(figsize=(7,5))
```

```
sns.boxplot(  
    hue="platform",  
    y="likes",  
    data=df,  
    color="skyblue"  
)  
  
plt.title("Likes Distribution by Platform")  
plt.xlabel("Platform")  
plt.ylabel("Likes")  
  
plt.show()
```

C:\Users\VISHAL\AppData\Local\Temp\ipykernel_25024\231652744.py:9: FutureWarning:

Setting a gradient palette using color= is deprecated and will be removed in v0.14.0. Set `palette='dark:skyblue'` for the same effect.

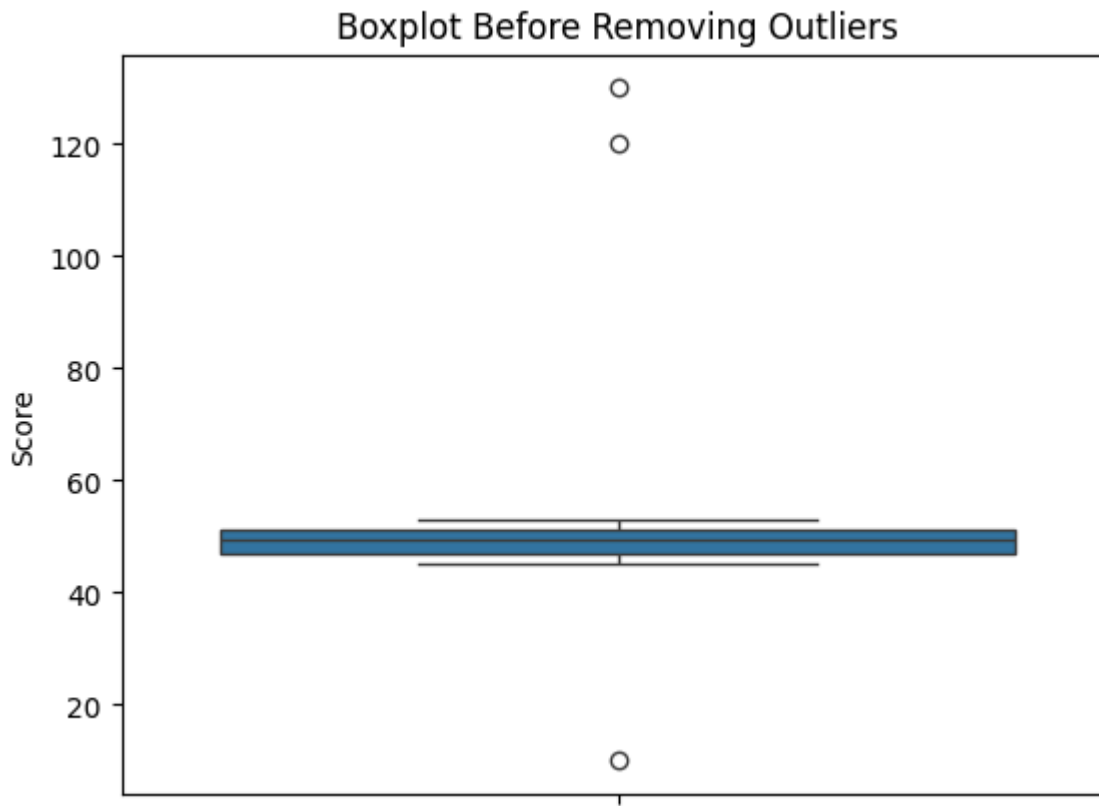
```
sns.boxplot(  
    hue="platform",  
    y="likes",  
    data=df,  
    palette="dark:skyblue"  
)
```



In [105...]

```
data = [45, 50, 52, 48, 47, 51, 49, 53, 46, 50,
        48, 47, 52, 49, 51, 50, 46, 120, 130, 10]
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({"Score": data})
sns.boxplot(y=df["Score"])
plt.title("Boxplot Before Removing Outliers")
plt.show()
```



Detect Outliers using IQR Method

In [106...]

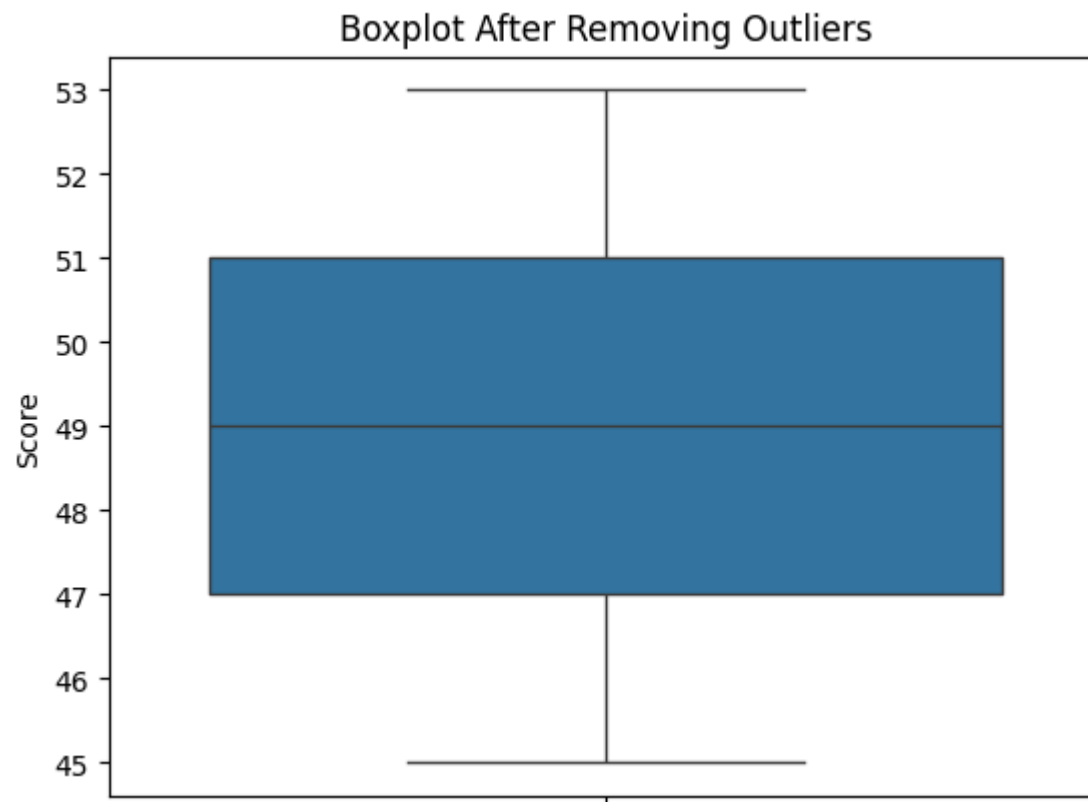
```
Q1 = df["Score"].quantile(0.25)
Q3 = df["Score"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df["Score"] < lower) | (df["Score"] > upper)]
print("Outliers:")
print(outliers)
```

Outliers:
Score
17 120
18 130
19 10

remove outlier

```
In [108... df_clean = df[(df["Score"] >= lower) & (df["Score"] <= upper)]
```

```
In [109... sns.boxplot(y=df_clean["Score"])  
plt.title("Boxplot After Removing Outliers")  
plt.show()
```



In [110...

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = [45,50,52,48,47,51,49,53,46,50,
        48,47,52,49,51,50,46,120,130,10]

df = pd.DataFrame({"Score":data})

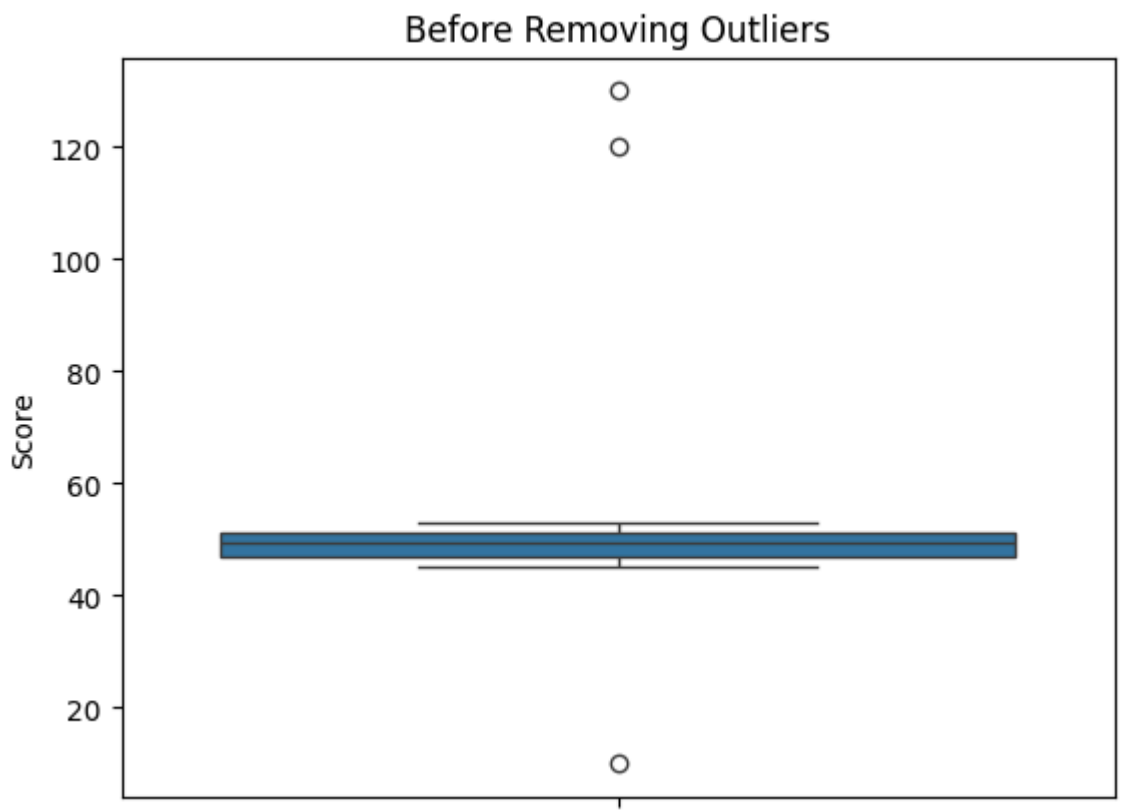
# BEFORE
sns.boxplot(y=df["Score"])
plt.title("Before Removing Outliers")
plt.show()

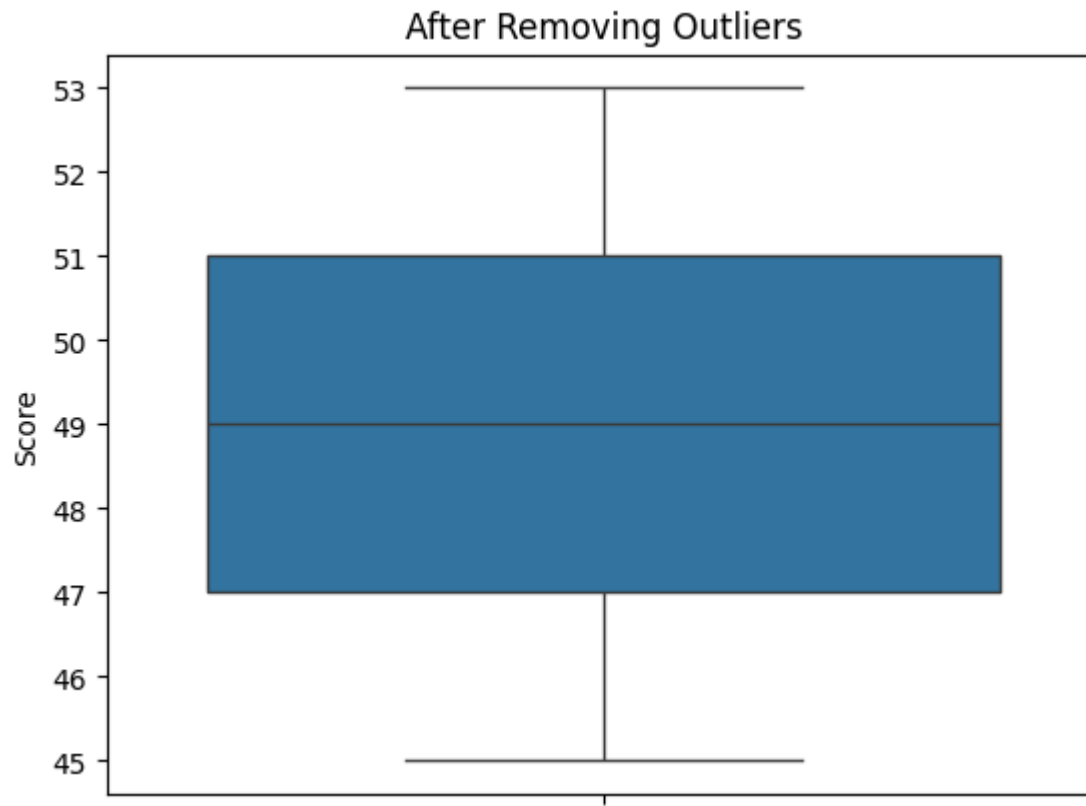
# IQR METHOD
Q1 = df["Score"].quantile(0.25)
Q3 = df["Score"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# REMOVE OUTLIERS
df_clean = df[(df["Score"] >= lower) & (df["Score"] <= upper)]

# AFTER
sns.boxplot(y=df_clean["Score"])
plt.title("After Removing Outliers")
plt.show()
```





```
In [112... df=pd.read_csv("social_media_engagement.csv")
```

Scatter plot:

`sns.scatterplot(x, y, hue, style, s, data)`

Parameter	Meaning
x	column used for x-axis values
y	column used for y-axis values
hue	separates points by color using a categorical column

Parameter	Meaning
style	changes marker shape based on category
s	controls marker size (bubble size)
data	dataset used for plotting

Basic Scatter Plot (x, y)

Interpretation:

Each point = one post

X-axis = likes

Y-axis = shares

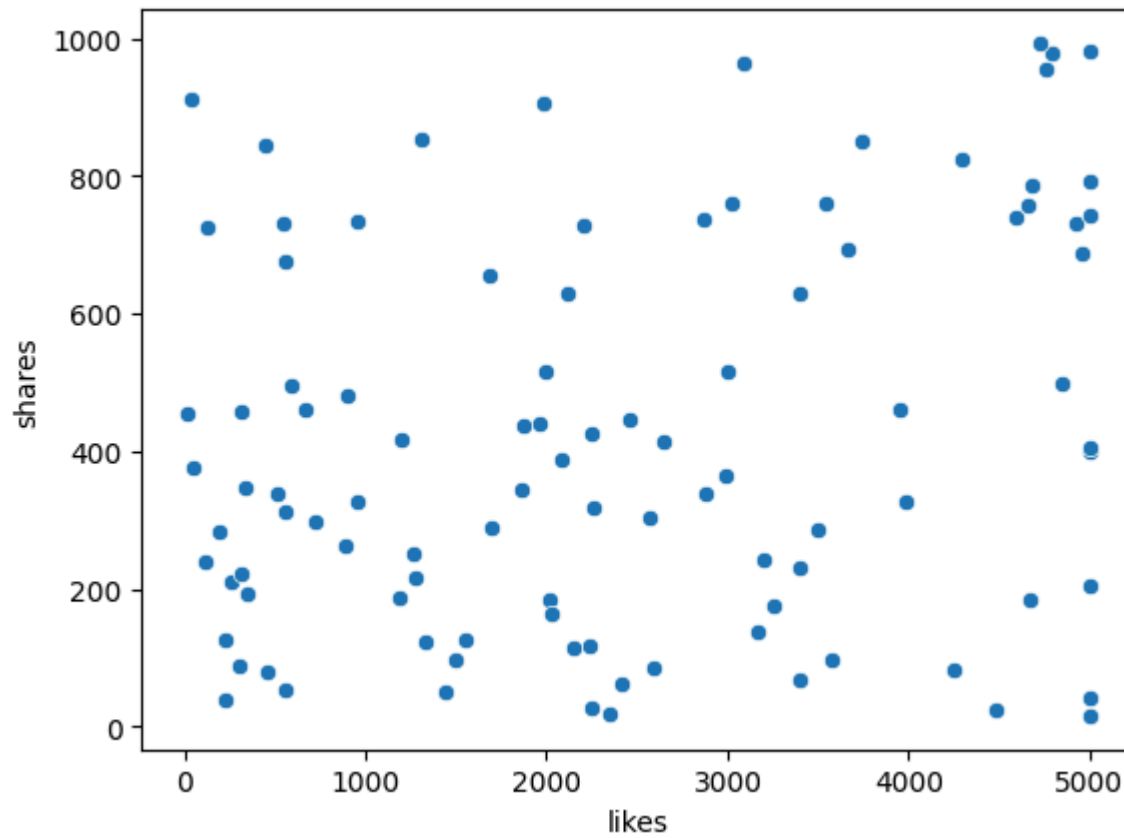
Insight: Posts with more likes often get more shares.

In [113...

```
sns.scatterplot(x="likes", y="shares", data=df)
```

Out[113...

```
<Axes: xlabel='likes', ylabel='shares'>
```



Scatter Plot with hue | Parameter | Purpose | | ----- | ----- | | `hue="platform"` | separates points by platform |

Platforms:

Facebook

Instagram

Twitter

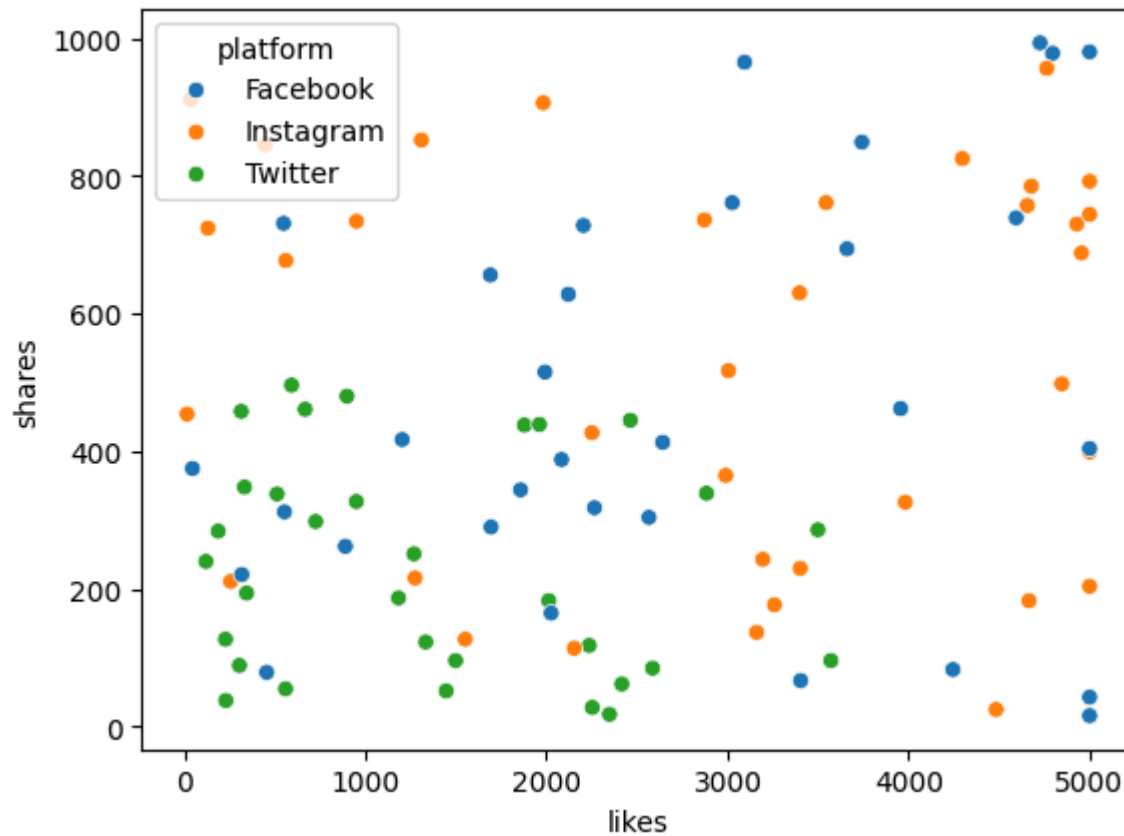
Insight:

Instagram posts cluster in higher engagement zones

Twitter posts often have lower likes and shares

```
In [114...] sns.scatterplot(x="likes", y="shares", hue="platform", data=df)
```

```
Out[114...] <Axes: xlabel='likes', ylabel='shares'>
```



Scatter Plot with style

Parameter	Purpose
<code>style="post_type"</code>	marker shape changes for each post type

Post types:

image

carousel

poll

video

text

Insight:

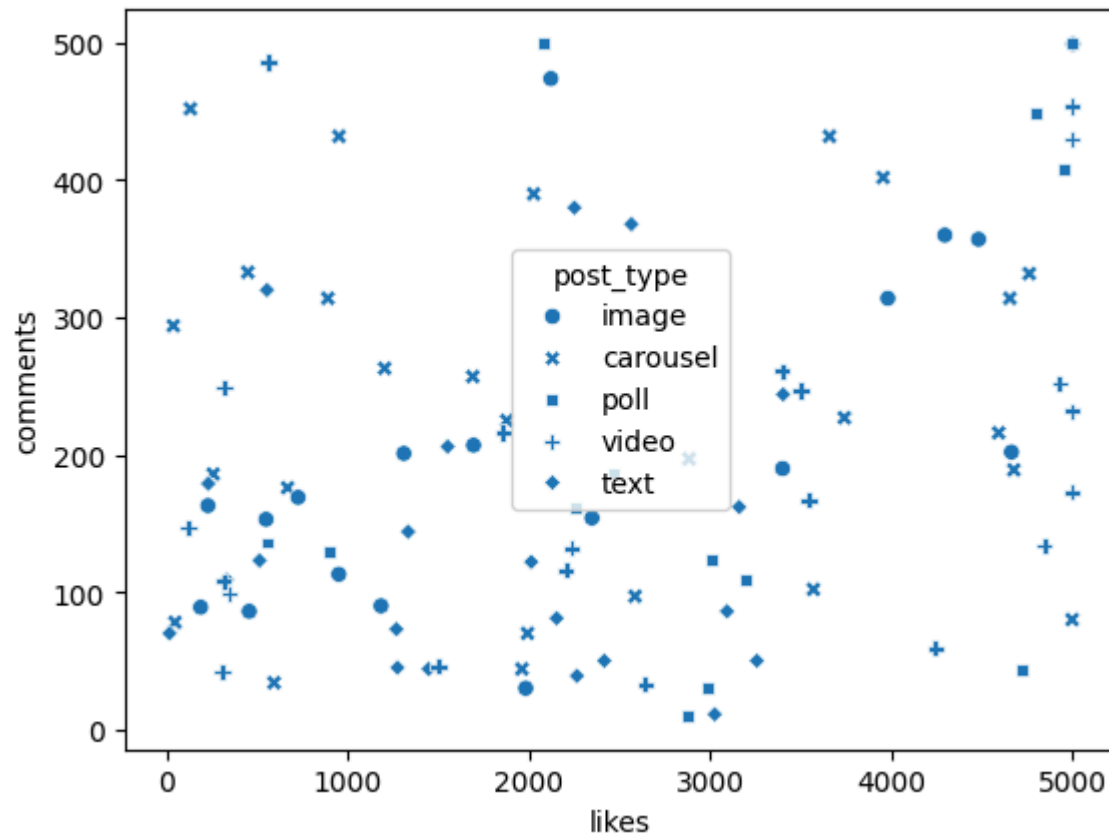
Some post types generate higher comments

In [115...

```
sns.scatterplot(x="likes", y="comments", style="post_type", data=df)
```

Out[115...

```
<Axes: xlabel='likes', ylabel='comments'>
```



Bubble Scatter Plot

Parameter	Purpose
size="comments"	bubble size based on number of comments

Interpretation:

Large circles → many comments

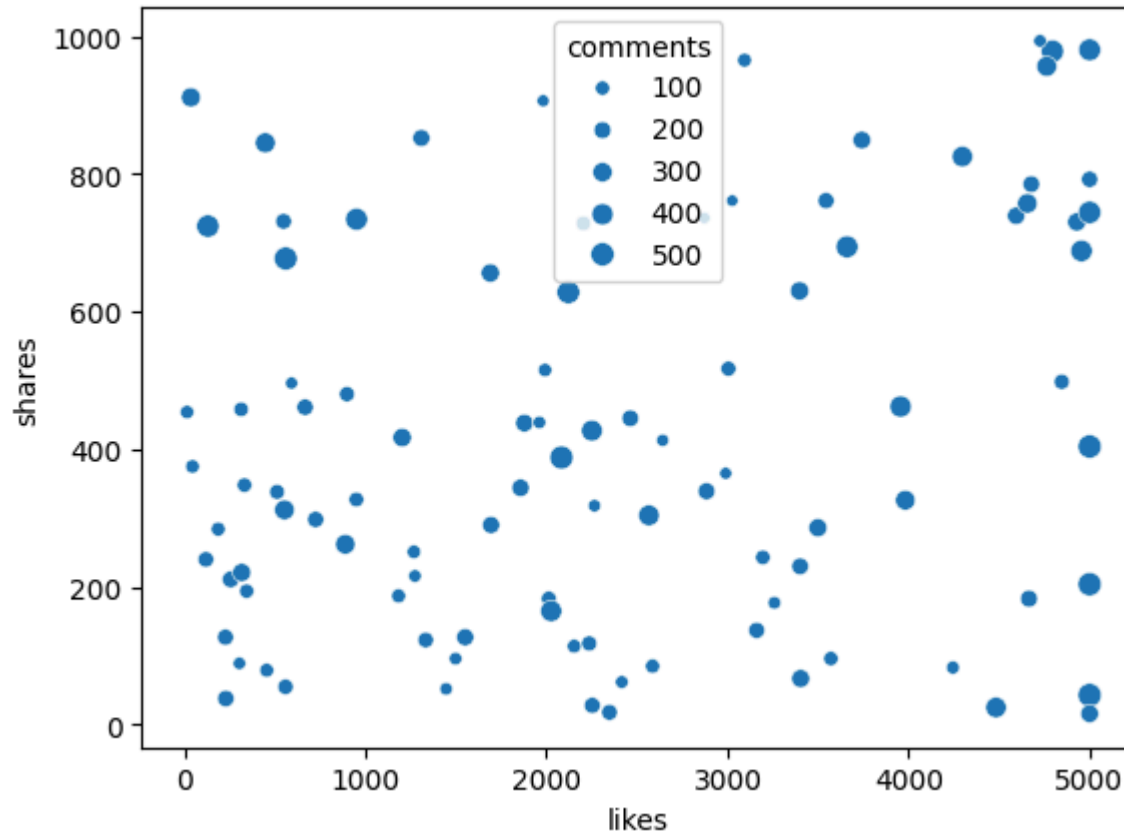
Small circles → fewer comments

Insight:

Posts with many likes often generate bigger engagement bubbles

```
In [116... sns.scatterplot(x="likes", y="shares", size="comments", data=df)
```

```
Out[116... <Axes: xlabel='likes', ylabel='shares'>
```



```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Parameter	Meaning
hue="platform"	color represents platform
style="post_type"	marker shape represents post type
size="comments"	bubble size represents comment count

This single plot shows three dimensions of engagement:

color → platform

shape → post type

size → comments

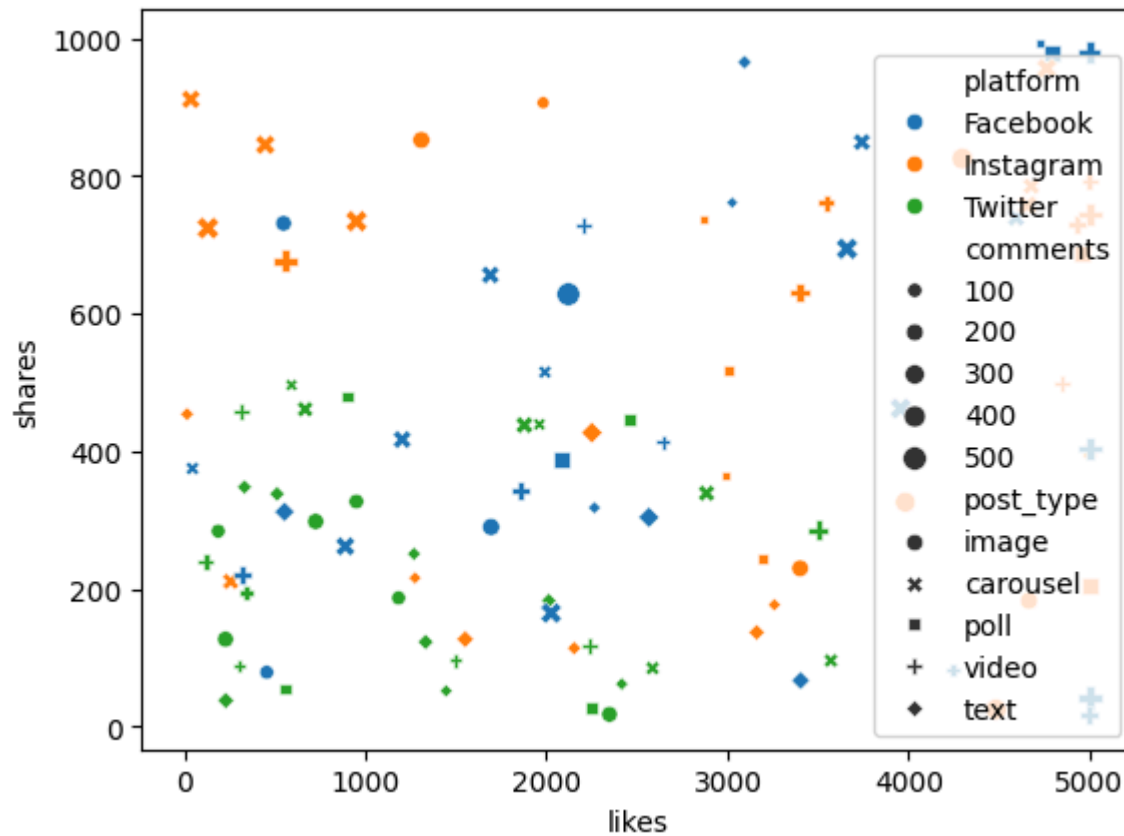
Metric	Observation
Likes vs Shares	positive trend
Instagram	highest engagement cluster
Twitter	lowest engagement cluster
Bubble size	large comments on high-like posts

In [117...

```
sns.scatterplot(  
    x="likes",  
    y="shares",  
    hue="platform",  
    style="post_type",  
    size="comments",  
    data=df  
)
```

Out[117...

```
<Axes: xlabel='likes', ylabel='shares'>
```



1. What is Plotly?

Plotly is a Python library used to create interactive graphs and dashboards.

Unlike Matplotlib or Seaborn, Plotly graphs are:

Interactive

Zoomable

Hoverable

Clickable

Web-based

Users can hover on points to see exact values, zoom into regions, and toggle data.

Commonly used in:

Data Analysis

Business Dashboards

Machine Learning Visualization

Web Applications (Dash)

Library name: plotly

Why Plotly is Used

Module	Use
plotly.express (px)	Simple plotting
plotly.graph_objects (go)	Advanced customization

Feature	Explanation
Interactive charts	Hover to see values
Zooming	Zoom into specific region
Tooltips	Show extra information
Web dashboards	Used in Dash framework
Beautiful visualizations	Modern UI

Feature	Matplotlib	Seaborn	Plotly
Interactive	✗	✗	✓
Hover values	✗	✗	✓
Zoom	✗	✗	✓
Dashboard support	✗	✗	✓
Best for	static plots	statistical plots	dashboards

Plotly Charts – Basic Requirement & Comparison

Chart	Basic Requirement	Used For	Example Code
Bar Chart	Category + Numeric value	Compare values between categories	<code>px.bar(data, x="Category", y="Value")</code>
Scatter Plot	Two numeric variables	Relationship / correlation between variables	<code>px.scatter(data, x="Var1", y="Var2")</code>
Line Chart	Ordered data (usually time) + numeric value	Trend over time	<code>px.line(data, x="Time", y="Value")</code>
Histogram	One numeric column	Distribution of data	<code>px.histogram(data, x="Value")</code>
Pie Chart	Category + numeric value	Proportion / percentage	<code>px.pie(data, names="Category", values="Value")</code>

1. Bar Chart

- Basic Requirement

One categorical column

One numeric column

- Purpose

Compare values between groups.

2. Scatter Plot

- Basic Requirement

Two numeric variables

- Purpose

Show relationship or correlation.

3. Line Chart

- Basic Requirement

Ordered variable (time or sequence)

- Numeric value

Purpose

Show trend over time.

4. Histogram

- Basic Requirement

One numeric column

- Purpose

Show frequency distribution of data

5. Pie Chart

- Basic Requirement

Category column

Numeric values

- Purpose

Show percentage share of categories.

```
In [13]: import numpy as np
import pandas as pd
import plotly.express as px
import plotly.io as pio
pio.renderers.default = "notebook"
```

```
In [14]: df=pd.read_csv("social_media_engagement.csv")
```

```
In [15]: df
```

```
Out[15]:
```

	post_id	platform	post_type	post_time	likes	comments	shares	post_day	sentiment_score
0	1	Facebook	image	8/17/2023 14:45	2121	474	628	Thursday	positive
1	2	Facebook	carousel	5/14/2023 0:45	3660	432	694	Sunday	neutral
2	3	Instagram	poll	2/21/2023 16:15	4955	408	688	Tuesday	negative
3	4	Twitter	image	11/16/2023 0:45	1183	90	187	Thursday	negative
4	5	Twitter	video	5/23/2023 0:30	3499	247	286	Tuesday	positive
...	...	...	...	...	...	...	...	...	...
95	96	Instagram	carousel	7/12/2023 17:45	36	294	911	Wednesday	positive
96	97	Twitter	video	10/27/2023 23:45	314	108	458	Friday	neutral
97	98	Twitter	text	8/5/2023 8:45	229	179	38	Saturday	positive
98	99	Instagram	poll	12/29/2023 12:15	5000	500	204	Friday	positive
99	100	Instagram	image	6/6/2023 21:00	4483	357	25	Tuesday	neutral

100 rows × 9 columns

Bar Chart Types

Simple Bar Chart – compares one value per category (default behavior).

Vishal Acharya

Grouped Bar Chart (`barmode="group"`) – bars of different categories are shown side-by-side for comparison.

Stacked Bar Chart (`barmode="stack"`) – bars are stacked on top of each other to show contribution to a total.

Diverging Bar Chart (`barmode="relative"`) – positive and negative values extend in opposite directions from zero.

px.bar() Parameters

x – column used for x-axis categories.

y – column used for numerical values determining bar height.

width – sets the figure width in pixels.

height – sets the figure height in pixels.

title – adds a title to the chart.

color – separates bars using different colors based on a column.

hover_data – displays additional columns when hovering over bars.

pattern_shape – adds pattern styles (/, x, etc.) to bars based on category.

barmode – controls bar layout such as group, stack, or relative.

text_auto – automatically shows values on the bars.

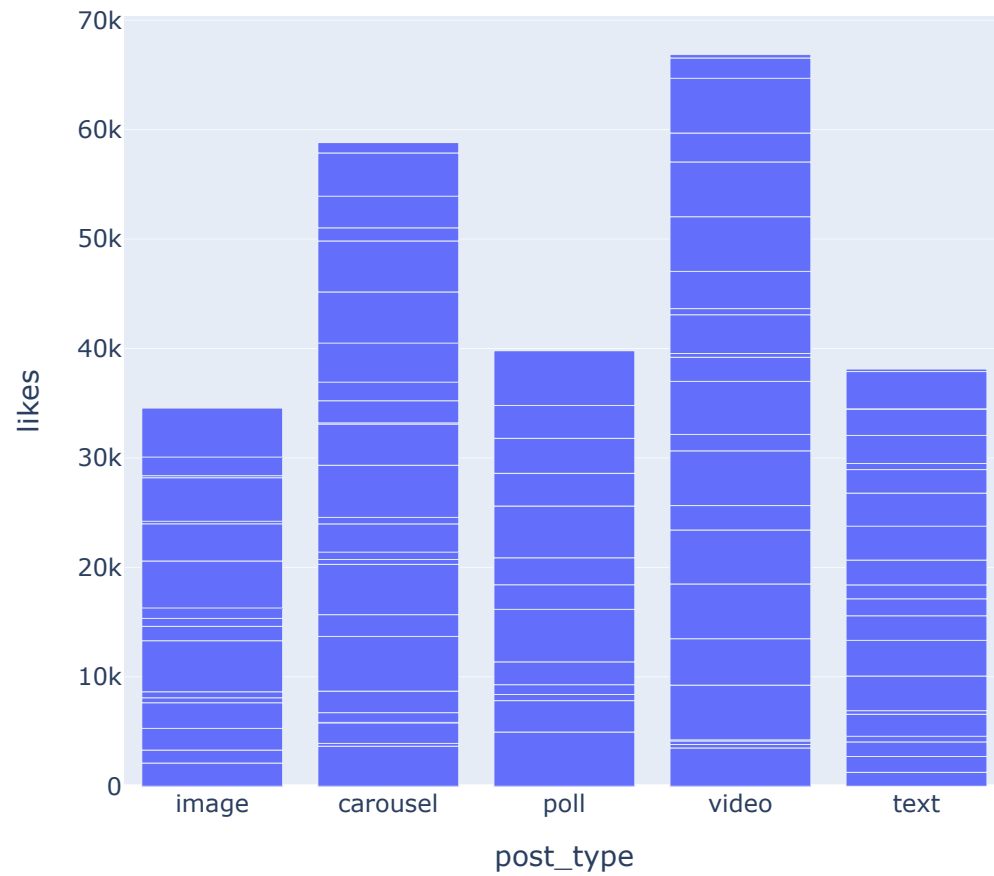
text – displays custom text labels from a column on bars.

orientation – changes bar direction to vertical (v) or horizontal (h).

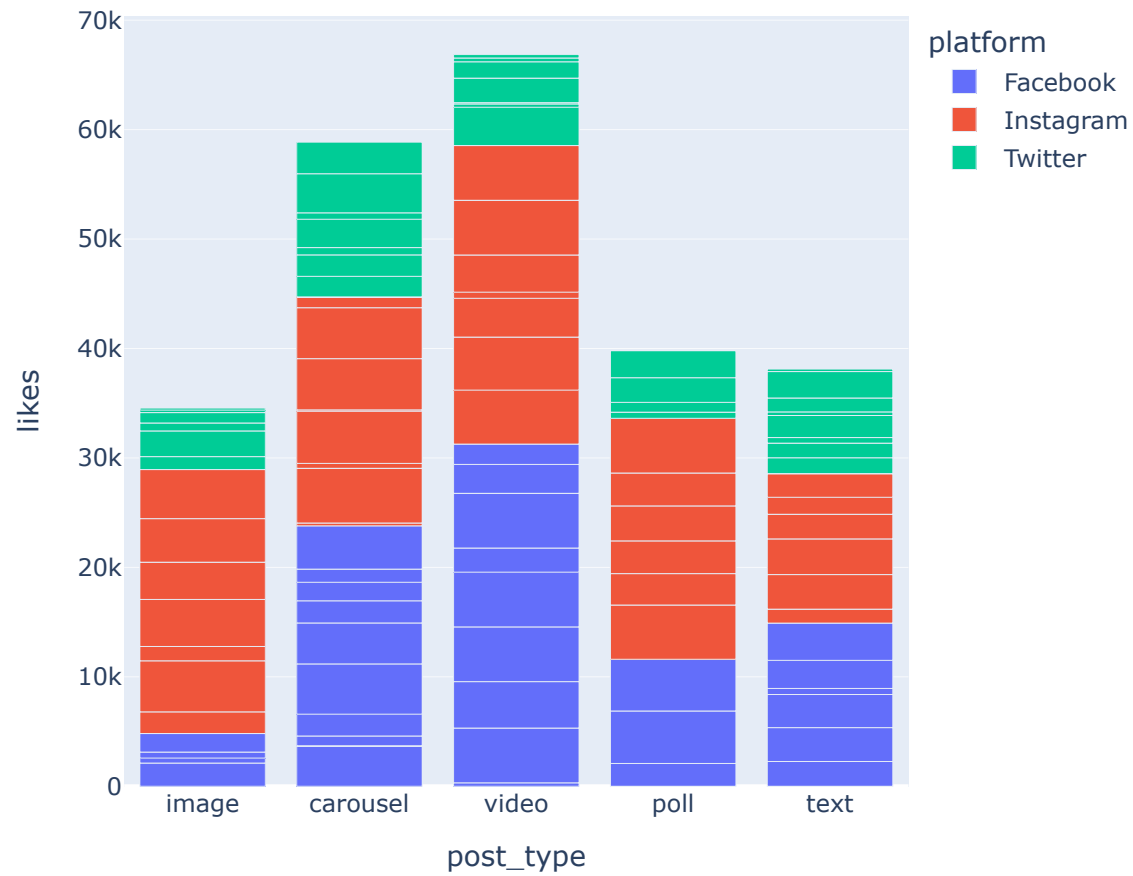
color_discrete_map – assigns specific colors to particular category values.(dictionary)

color_discrete_sequence – defines a list of colors used for bars.

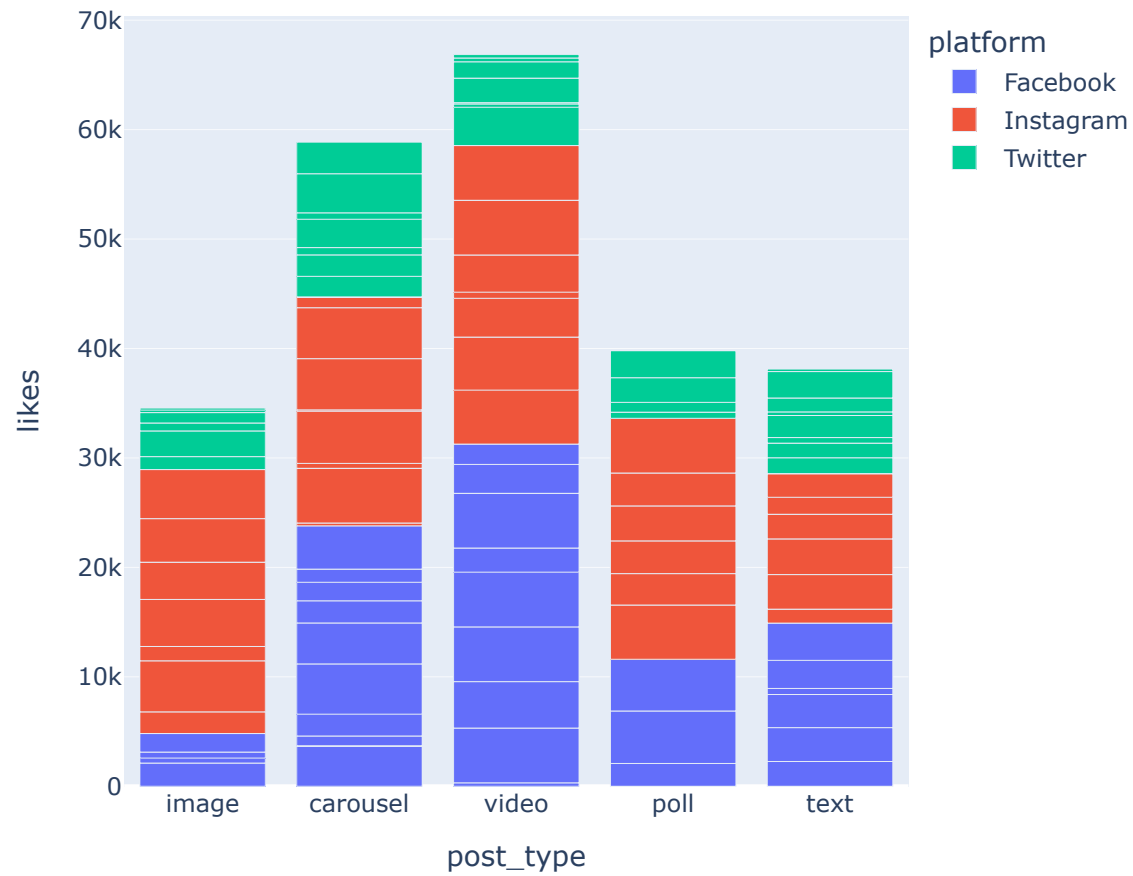
```
In [16]: fig=px.bar(df,x="post_type",y="likes",width=600)
fig.show()
```



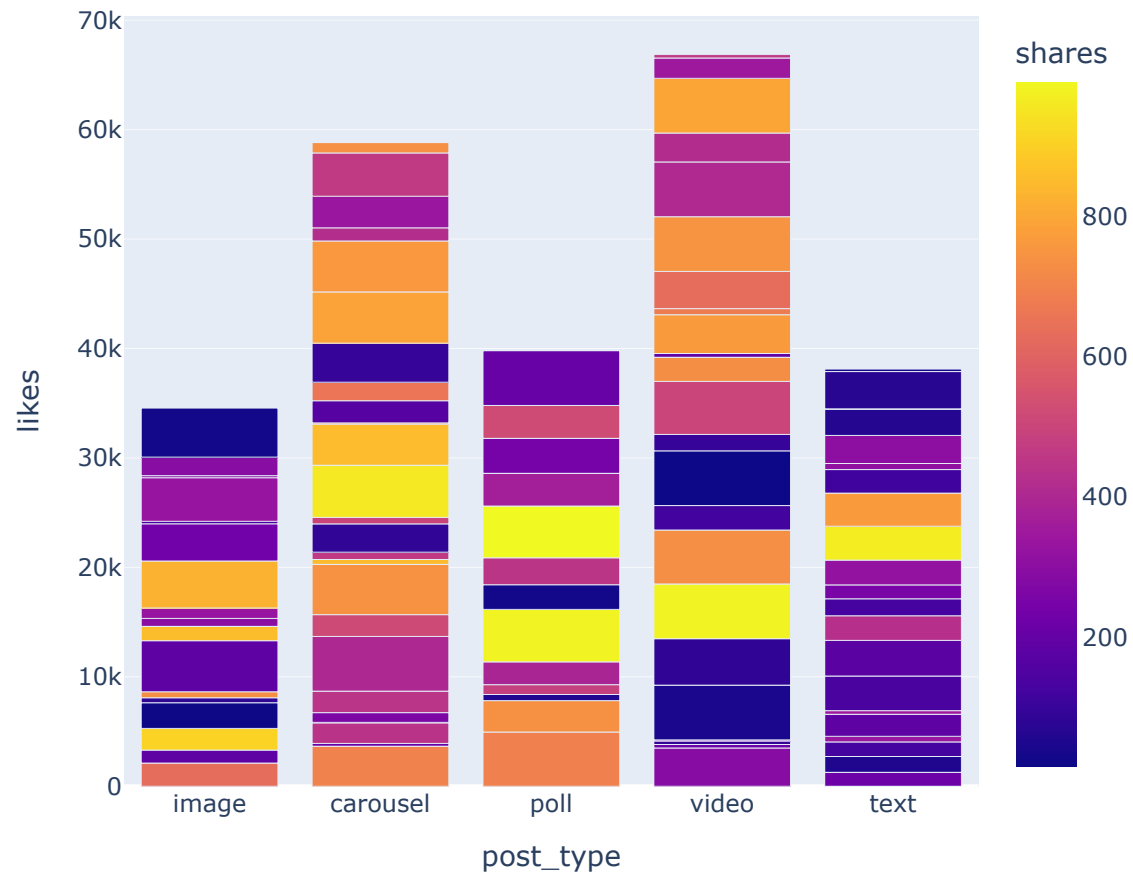
```
In [17]: fig = px.bar(df, x="post_type", y="likes", color="platform", width=600)
fig.show()
```



```
In [18]: fig = px.bar(df, x="post_type", y="likes", color="platform", width=600, hover_data=["post_day", "sentiment_score"])
fig.show()
```



```
In [19]: fig = px.bar(df, x="post_type", y="likes", color="platform", width=600, hover_data=["post_type", "likes"])
fig.show()
```



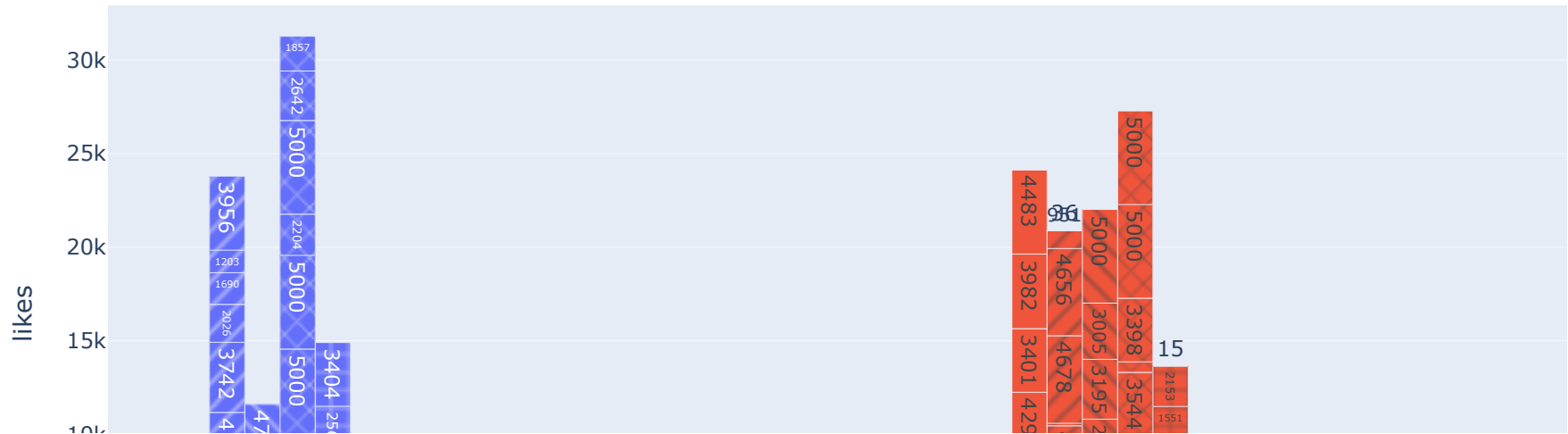
```
In [20]: import pandas as pd
import plotly.express as px

df = pd.read_csv("social_media_engagement.csv")

fig = px.bar(
    df,
    x="platform",           # correct column name
```

```
y="likes",  
color="platform",  
pattern_shape="post_type",    # pattern by post type  
barmode="group",  
title="Total Likes by Platform (Pattern = Post Type)",  
text_auto=True  
)  
  
fig.show()
```

Total Likes by Platform (Pattern = Post Type)



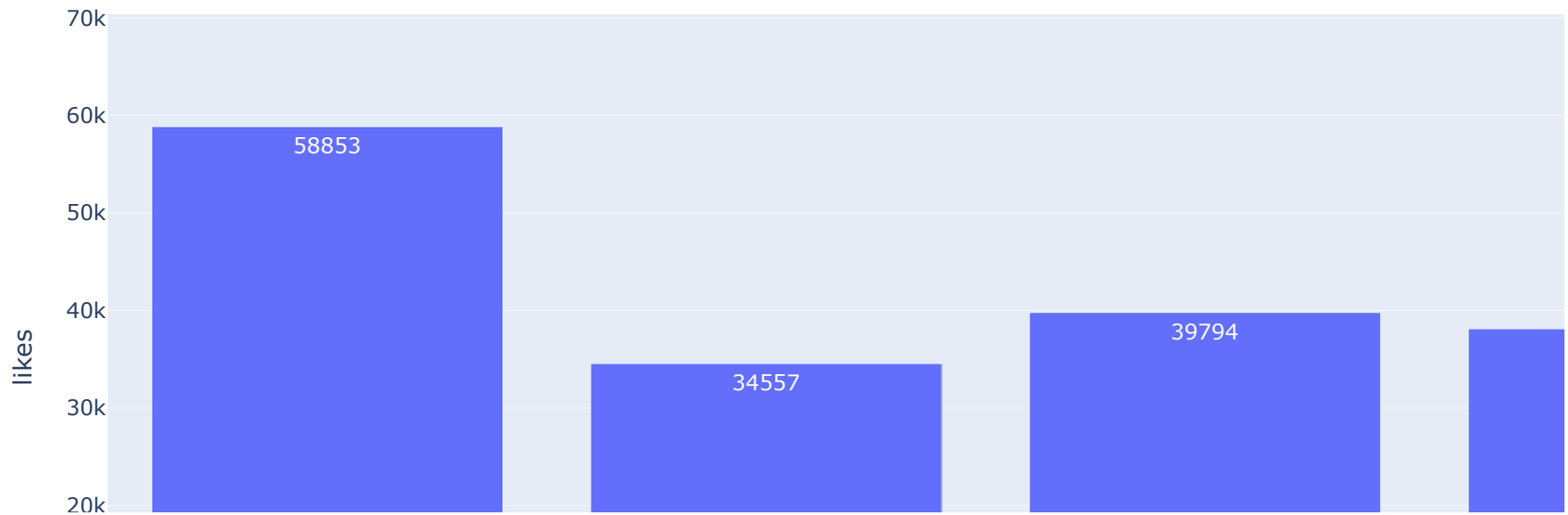
```
In [21]: data=df.groupby("post_type")["likes"].sum().reset_index()
```

```
In [22]: data
```

Out[22]:

	post_type	likes
0	carousel	58853
1	image	34557
2	poll	39794
3	text	38121
4	video	66856

```
In [23]: fig=px.bar(data,x="post_type",y="likes",text="likes")  
fig.show()
```



```
In [24]: data=df.groupby("post_type")["likes"].sum().reset_index().sort_values("likes")
fig=px.bar(data,y="post_type",x="likes",orientation="h")
fig.show()
```

Vishal Acharya



Group bar chart

In [25]:

```
df
```

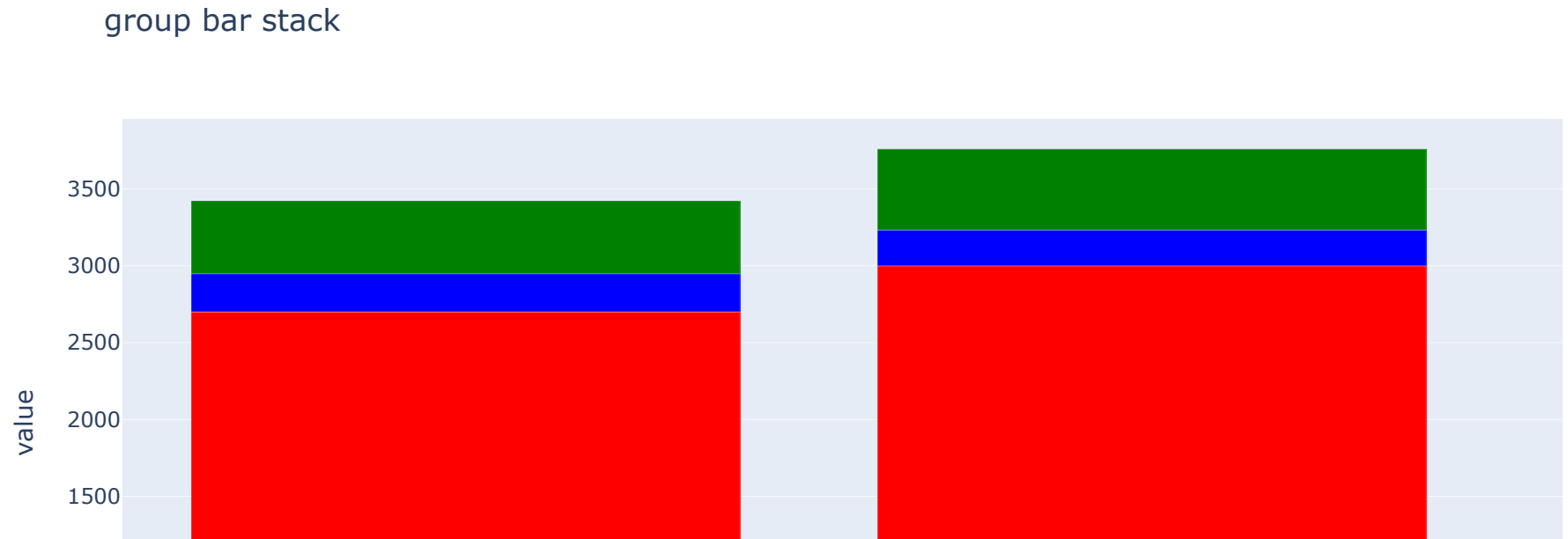
Out[25]:

	post_id	platform	post_type	post_time	likes	comments	shares	post_day	sentiment_score
0	1	Facebook	image	8/17/2023 14:45	2121	474	628	Thursday	positive
1	2	Facebook	carousel	5/14/2023 0:45	3660	432	694	Sunday	neutral
2	3	Instagram	poll	2/21/2023 16:15	4955	408	688	Tuesday	negative
3	4	Twitter	image	11/16/2023 0:45	1183	90	187	Thursday	negative
4	5	Twitter	video	5/23/2023 0:30	3499	247	286	Tuesday	positive
...	...	...	...	...	...	...	...	...	...
95	96	Instagram	carousel	7/12/2023 17:45	36	294	911	Wednesday	positive
96	97	Twitter	video	10/27/2023 23:45	314	108	458	Friday	neutral
97	98	Twitter	text	8/5/2023 8:45	229	179	38	Saturday	positive
98	99	Instagram	poll	12/29/2023 12:15	5000	500	204	Friday	positive
99	100	Instagram	image	6/6/2023 21:00	4483	357	25	Tuesday	neutral

100 rows × 9 columns

```
In [26]: data=df.groupby("platform")[["likes","comments","shares"]].mean().reset_index()
```

```
In [27]: fig=px.bar(data,x="platform",y=["likes","comments","shares"],barmode="stack",title="group bar stack",color_discrete_map={"like  
fig.show()
```



```
In [28]: fig=px.bar(data,x="platform",y=["likes","comments","shares"],barmode="group",title="group",color_discrete_map={"likes":"red","comments":"blue","shares":"green"},fig.show()
```



diverging bar

```
In [29]: data["comments"]=data["comments"]*-1
```

```
In [30]: fig=px.bar(data,x=["comments","likes"],y="platform",orientation="h",barmode="relative",color_discrete_sequence=["red","black"])  
fig.show()
```

Vishal Acharya



```
In [31]: %%writefile bar.csv
match_id,player,match_format,ground,run,opposite_team,ball,4s,6s
1,Virat Kohli,ODI,Ahmedabad,85,Australia,92,8,2
2,Rohit Sharma,T20,Mumbai,60,England,38,6,4
3,Shubman Gill,ODI,Delhi,110,Australia,105,12,1
4,Virat Kohli,T20,Chennai,45,England,30,4,2
5,Rohit Sharma,Test,Ahmedabad,132,South Africa,210,14,1
6,Shubman Gill,ODI,Mumbai,78,Australia,85,7,3
```

```
7, Virat Kohli, T20, Delhi, 55, South Africa, 32, 5, 3
8, Rohit Sharma, ODI, Chennai, 95, England, 101, 9, 2
9, Shubman Gill, T20, Ahmedabad, 70, Australia, 40, 6, 3
10, Virat Kohli, Test, Mumbai, 120, South Africa, 180, 13, 2
11, Rohit Sharma, ODI, Delhi, 88, England, 90, 10, 1
12, Shubman Gill, T20, Chennai, 39, South Africa, 28, 3, 1
13, Virat Kohli, ODI, Ahmedabad, 102, Australia, 98, 11, 2
14, Rohit Sharma, T20, Mumbai, 48, England, 27, 5, 2
15, Shubman Gill, Test, Delhi, 140, South Africa, 220, 15, 2
16, Virat Kohli, ODI, Chennai, 67, Australia, 72, 6, 1
17, Rohit Sharma, T20, Ahmedabad, 59, South Africa, 35, 5, 3
18, Shubman Gill, ODI, Mumbai, 91, England, 88, 9, 2
19, Virat Kohli, T20, Delhi, 73, Australia, 42, 7, 4
20, Rohit Sharma, Test, Chennai, 125, South Africa, 205, 12, 3
```

Overwriting bar.csv

```
In [32]: import numpy as np
import pandas as pd
import plotly.express as px
import plotly.io as pio
pio.renderers.default = "notebook"
```

```
In [33]: df=pd.read_csv("bar.csv")
```

```
In [34]: df
```

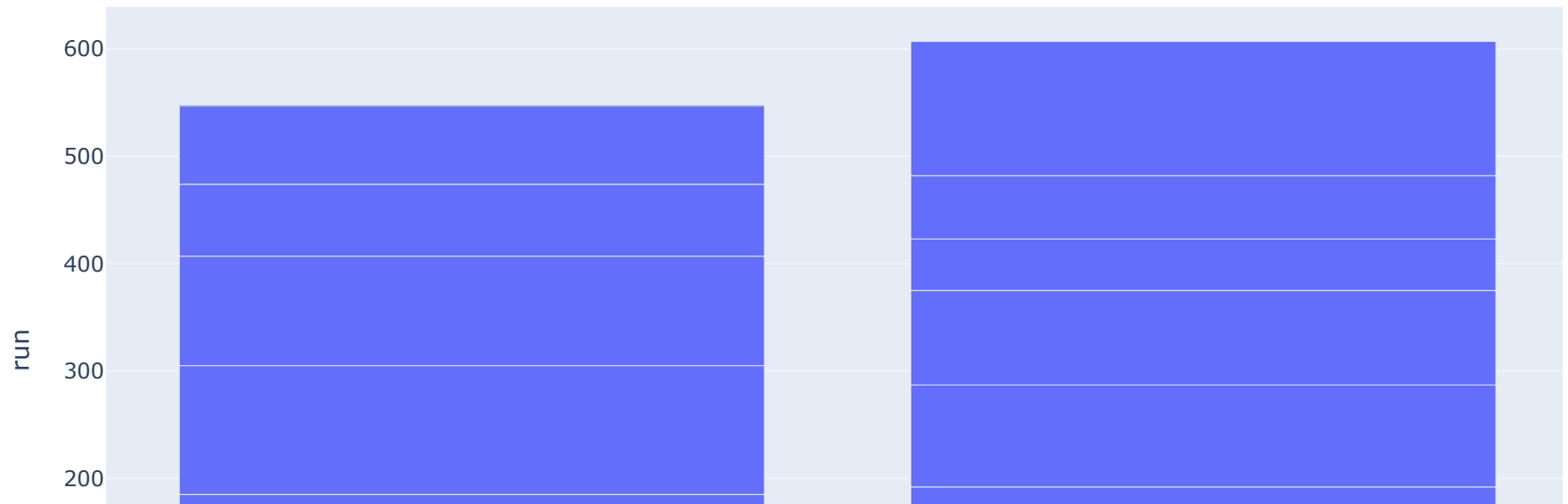
Out[34]:

	match_id	player	match_format	ground	run	opposite_team	ball	4s	6s
0	1	Virat Kohli	ODI	Ahmedabad	85	Australia	92	8	2
1	2	Rohit Sharma	T20	Mumbai	60	England	38	6	4
2	3	Shubman Gill	ODI	Delhi	110	Australia	105	12	1
3	4	Virat Kohli	T20	Chennai	45	England	30	4	2
4	5	Rohit Sharma	Test	Ahmedabad	132	South Africa	210	14	1
5	6	Shubman Gill	ODI	Mumbai	78	Australia	85	7	3
6	7	Virat Kohli	T20	Delhi	55	South Africa	32	5	3
7	8	Rohit Sharma	ODI	Chennai	95	England	101	9	2
8	9	Shubman Gill	T20	Ahmedabad	70	Australia	40	6	3
9	10	Virat Kohli	Test	Mumbai	120	South Africa	180	13	2
10	11	Rohit Sharma	ODI	Delhi	88	England	90	10	1
11	12	Shubman Gill	T20	Chennai	39	South Africa	28	3	1
12	13	Virat Kohli	ODI	Ahmedabad	102	Australia	98	11	2
13	14	Rohit Sharma	T20	Mumbai	48	England	27	5	2
14	15	Shubman Gill	Test	Delhi	140	South Africa	220	15	2
15	16	Virat Kohli	ODI	Chennai	67	Australia	72	6	1
16	17	Rohit Sharma	T20	Ahmedabad	59	South Africa	35	5	3
17	18	Shubman Gill	ODI	Mumbai	91	England	88	9	2
18	19	Virat Kohli	T20	Delhi	73	Australia	42	7	4
19	20	Rohit Sharma	Test	Chennai	125	South Africa	205	12	3

In [35]: `fig=px.bar(df,x="player",y="run")`

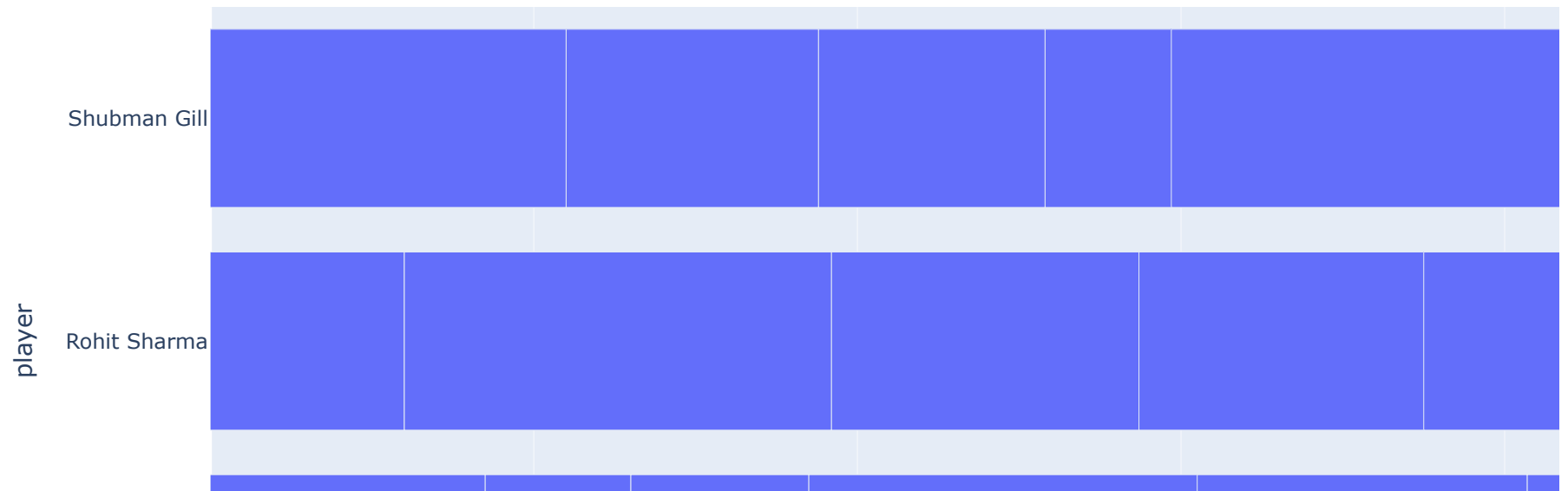
Vishal Acharya

```
fig.show()
```



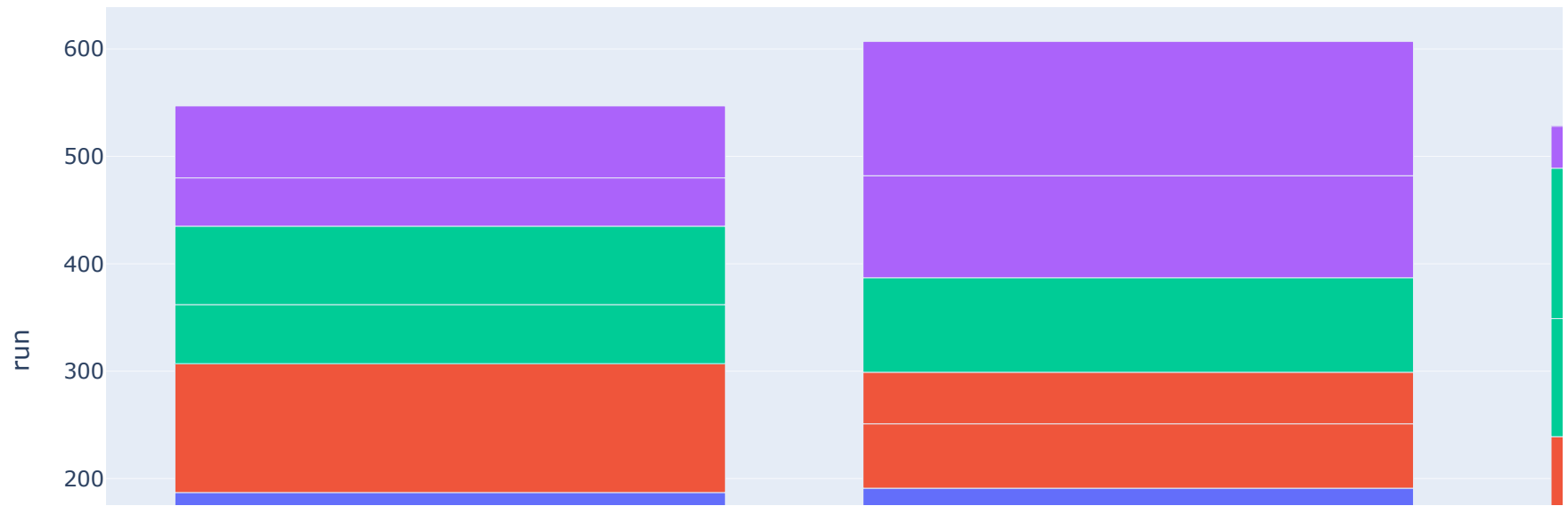
```
In [36]: fig=px.bar(df,y="player",x="run",orientation="h")  
fig.show()
```

Vishal Acharya



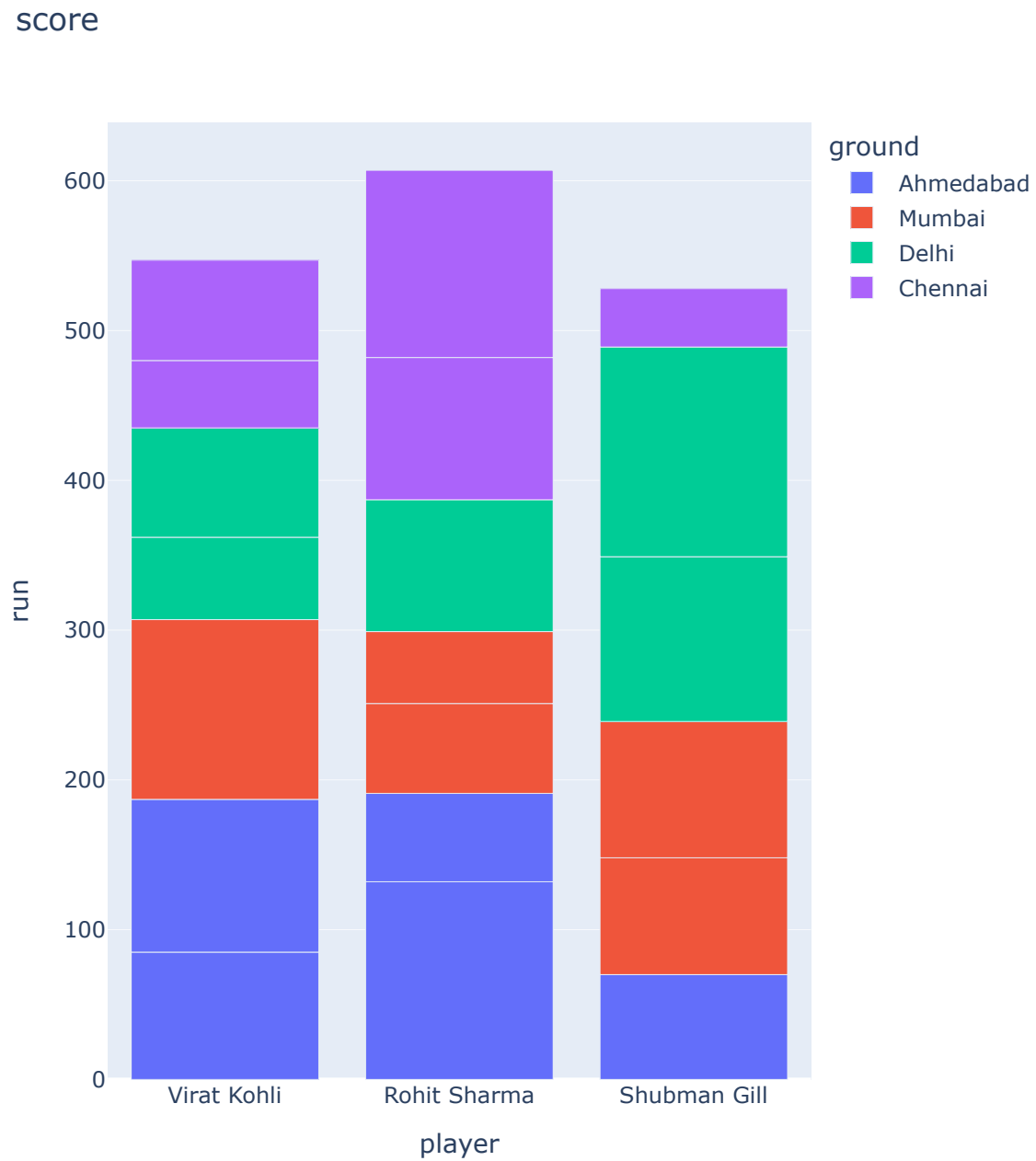
```
In [37]: fig=px.bar(df,x="player",y="run",color="ground")
fig.show()
```

Vishal Acharya



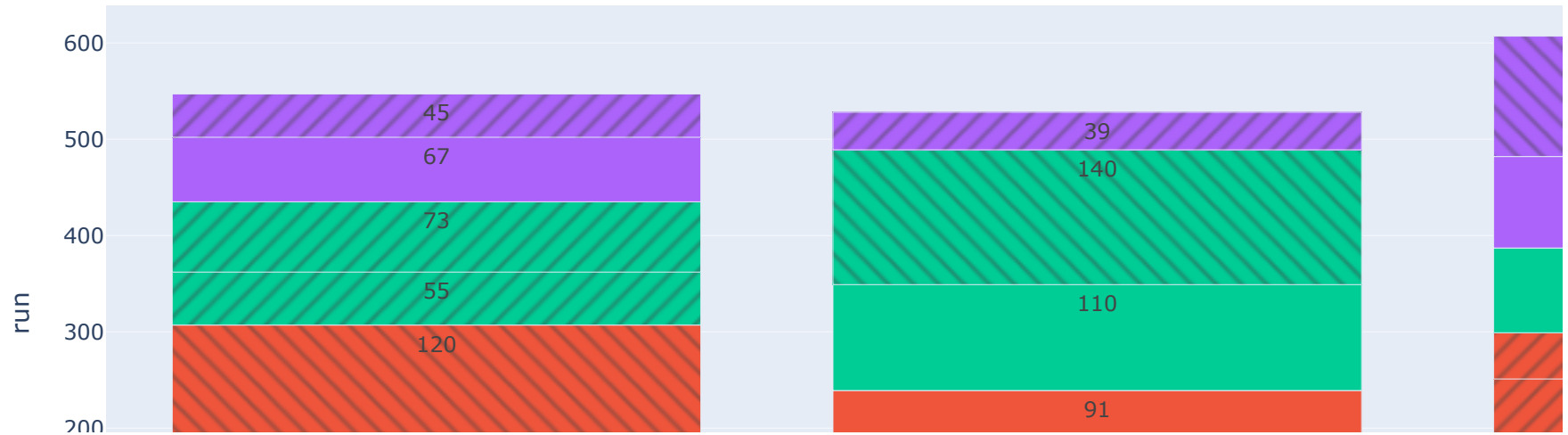
```
In [38]: fig=px.bar(df,x="player",y="run",color="ground",hover_data=["ball","opposite_team"],width=600,height=700,title="score")  
fig.show()
```

Vishal Acharya



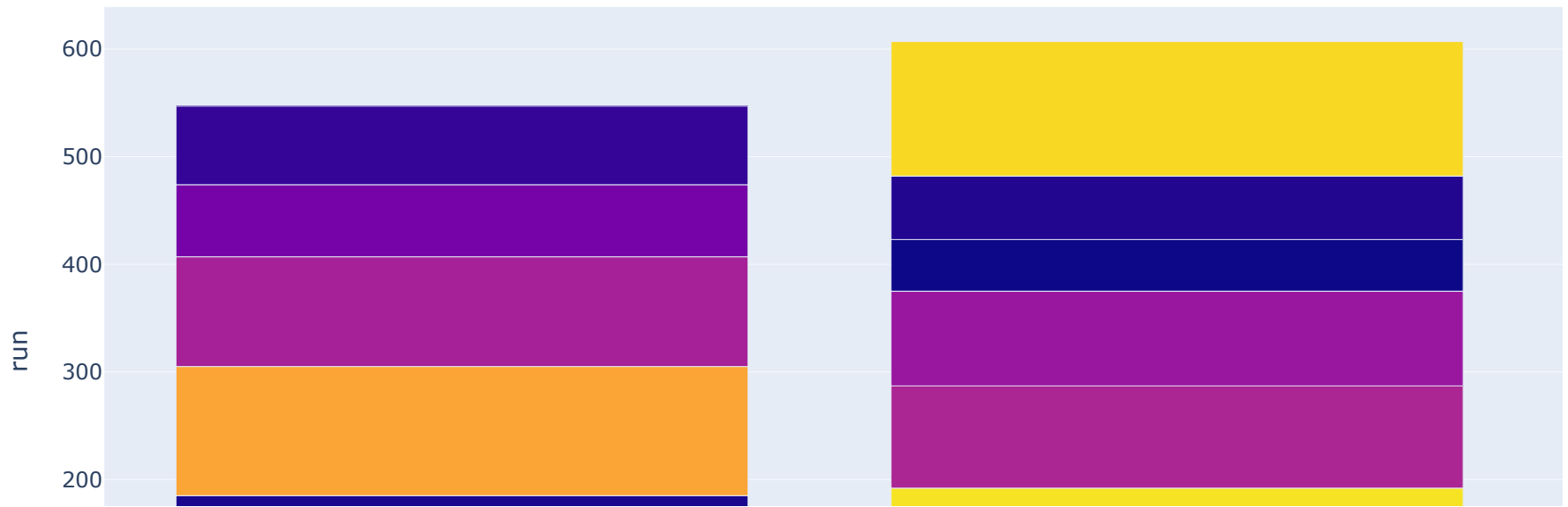
```
In [39]: fig = px.bar(  
    df,  
    x="player",  
    y="run",  
    color="ground",  
    pattern_shape="match_format",  
    title="Total Likes by Platform (Pattern = Post Type)",  
    text_auto=True  
)  
  
fig.show()
```

Total Likes by Platform (Pattern = Post Type)



```
In [40]: fig=px.bar(df,x="player",y="run",color="ball")
fig.show()
```

Vishal Acharya



```
In [41]: data1=df.groupby("player")["run"].sum().reset_index()
```

```
In [42]: data1
```

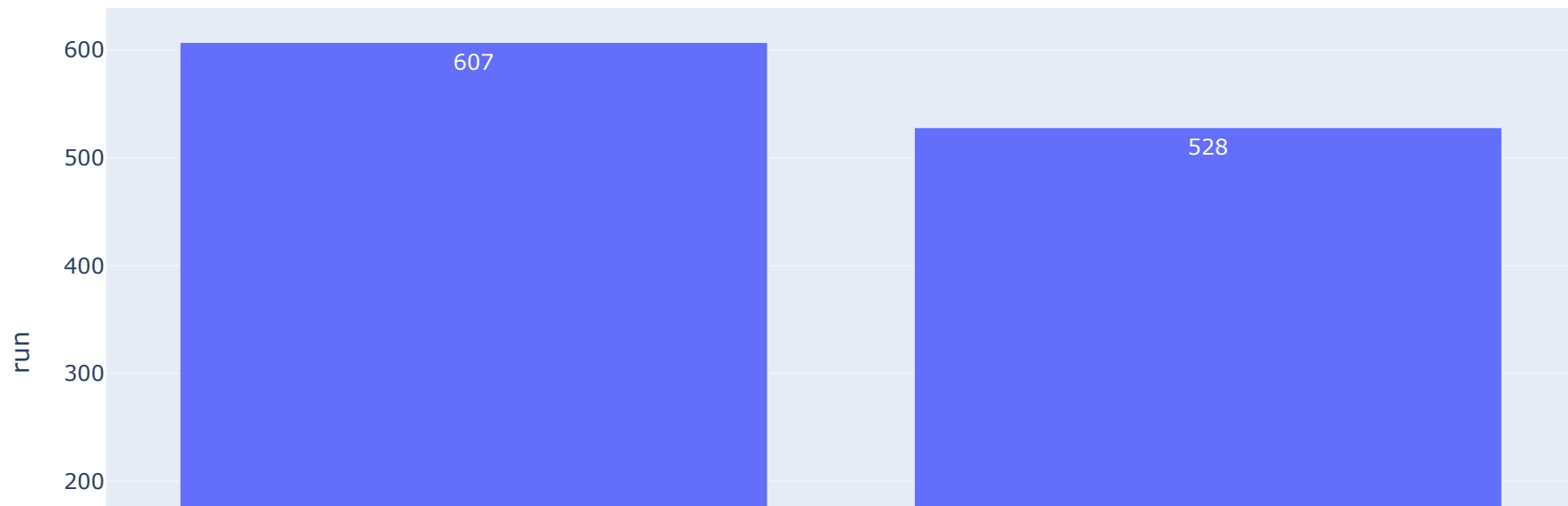
Out[42]:

	player	run
0	Rohit Sharma	607
1	Shubman Gill	528
2	Virat Kohli	547

In [43]:

```
fig=px.bar(data1,x="player",y="run",text="run")  
fig.show()
```

Vishal Acharya



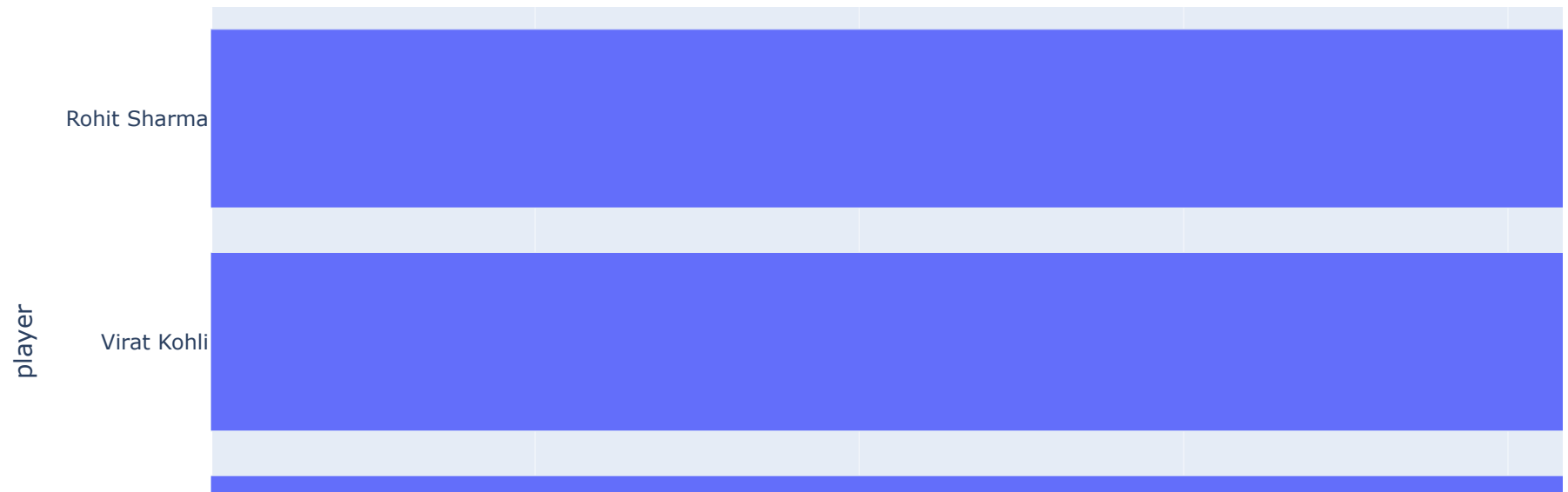
```
In [44]: fig=px.bar(data1,y="player",x="run",text="run",orientation="h")  
fig.show()
```

Vishal Acharya



```
In [45]: data1=df.groupby("player")["run"].sum().reset_index().sort_values("run")
fig=px.bar(data1,y="player",x="run",text="run",orientation="h")
fig.show()
```

Vishal Acharya



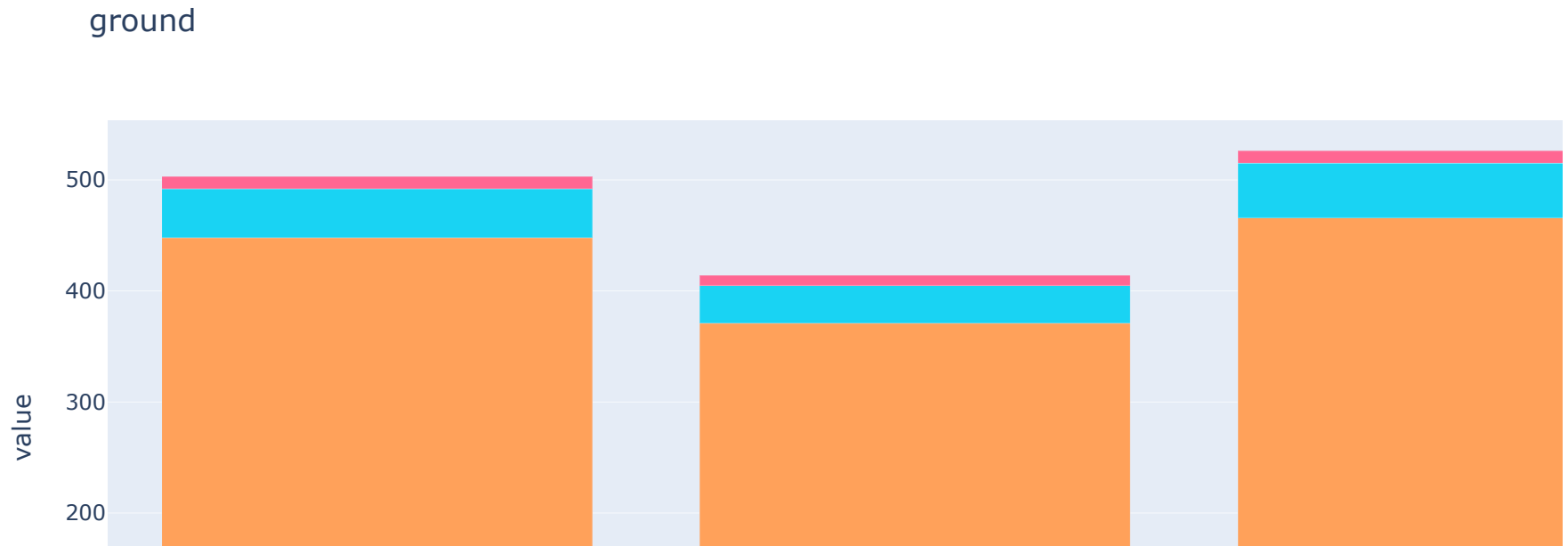
Stack Bar chart

```
In [46]: data=df.groupby("ground")[["run", "4s", "6s"]].sum().reset_index()
data
```

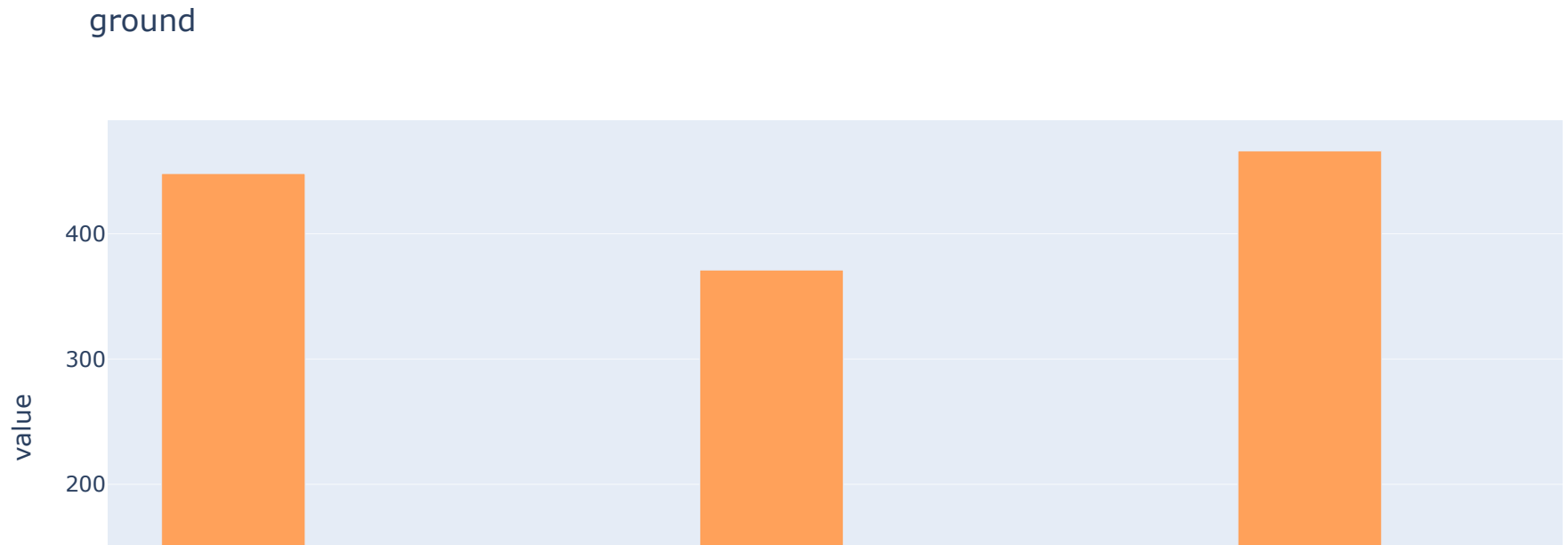
Out[46]:

	ground	run	4s	6s
0	Ahmedabad	448	44	11
1	Chennai	371	34	9
2	Delhi	466	49	11
3	Mumbai	397	40	13

```
In [47]: fig=px.bar(data,x="ground",y=["run","4s","6s"],barmode="stack",title="ground",  
              color_discrete_map={"Ahmedabad":"red","Chennai":"orange","Delhi":"blue","Mumbai":"green"})  
fig.show()
```



```
In [48]: fig=px.bar(data,x="ground",y=["run","4s","6s"],barmode="group",title="ground",  
                color_discrete_map={"Ahmedabad":"red","Chennai":"orange","Delhi":"blue","Mumbai":"green"})  
fig.show()
```



Diverging Bar

```
In [49]: data=df.groupby("ground")[["run","ball","4s"]].sum().reset_index()  
data
```

Out[49]:

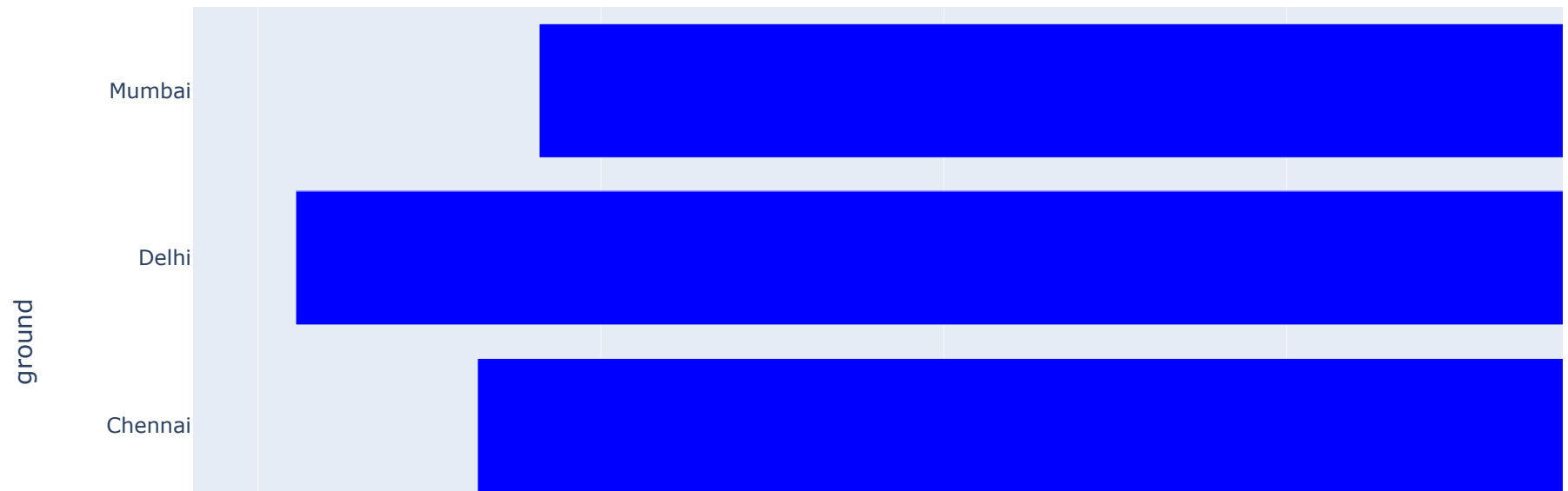
	ground	run	ball	4s
0	Ahmedabad	448	475	44
1	Chennai	371	436	34
2	Delhi	466	489	49
3	Mumbai	397	418	40

```
In [50]: data["ball"]=data["ball"]*(-1)
data
```

Out[50]:

	ground	run	ball	4s
0	Ahmedabad	448	-475	44
1	Chennai	371	-436	34
2	Delhi	466	-489	49
3	Mumbai	397	-418	40

```
In [51]: fig=px.bar(data,x=["ball","4s"],y="ground",orientation="h",barmode="relative",color_discrete_sequence=["blue","red"])
fig.show()
```



1 2D Scatter Plot

Shows relationship between two numerical variables (x and y).

Types

Simple Scatter – only x and y points.

Group Scatter – points colored by category.

Bubble Scatter – points with different sizes.

Group-Bubble Scatter – color + size together.

2 3D Scatter Plot

Shows relationship between three variables (x, y, z) in 3-dimensional space.

3 Scatter Matrix

Shows pairwise scatter plots of multiple variables in a matrix format for correlation analysis

px.scatter() Parameters

x – column used for x-axis values.

y – column used for y-axis values.

width – sets figure width in pixels.

height – sets figure height in pixels.

title – adds chart title.

color – colors points based on categorical or numeric column (group scatter).

size – controls marker size based on a column (bubble scatter).

color_discrete_map – assigns specific colors to category values.

color_discrete_sequence – provides list of colors used for points.

symbol – uses different marker shapes for categories.

facet_col – splits chart into multiple columns of subplots by category.

facet_row – splits chart into multiple rows of subplots by category.

Scatter plot

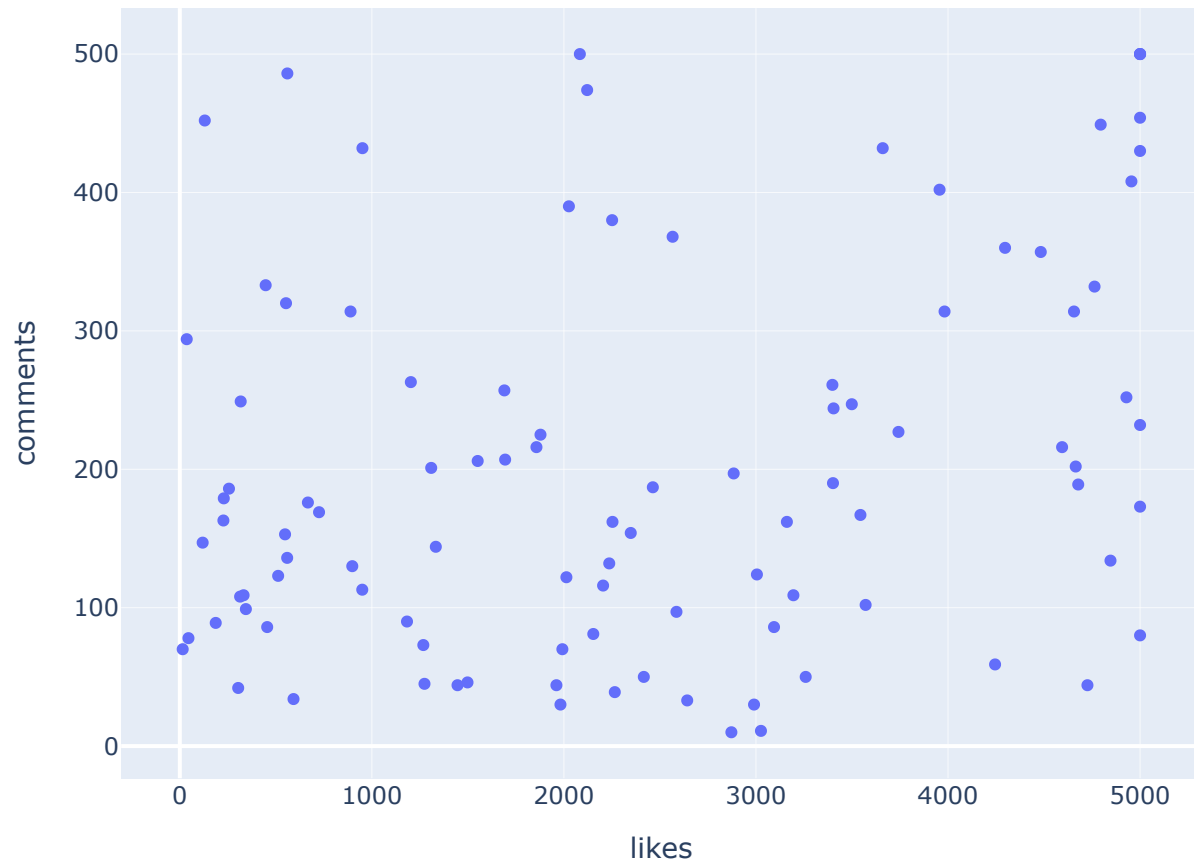
```
In [55]: df=pd.read_csv("social_media_engagement.csv")  
df
```

Out[55]:

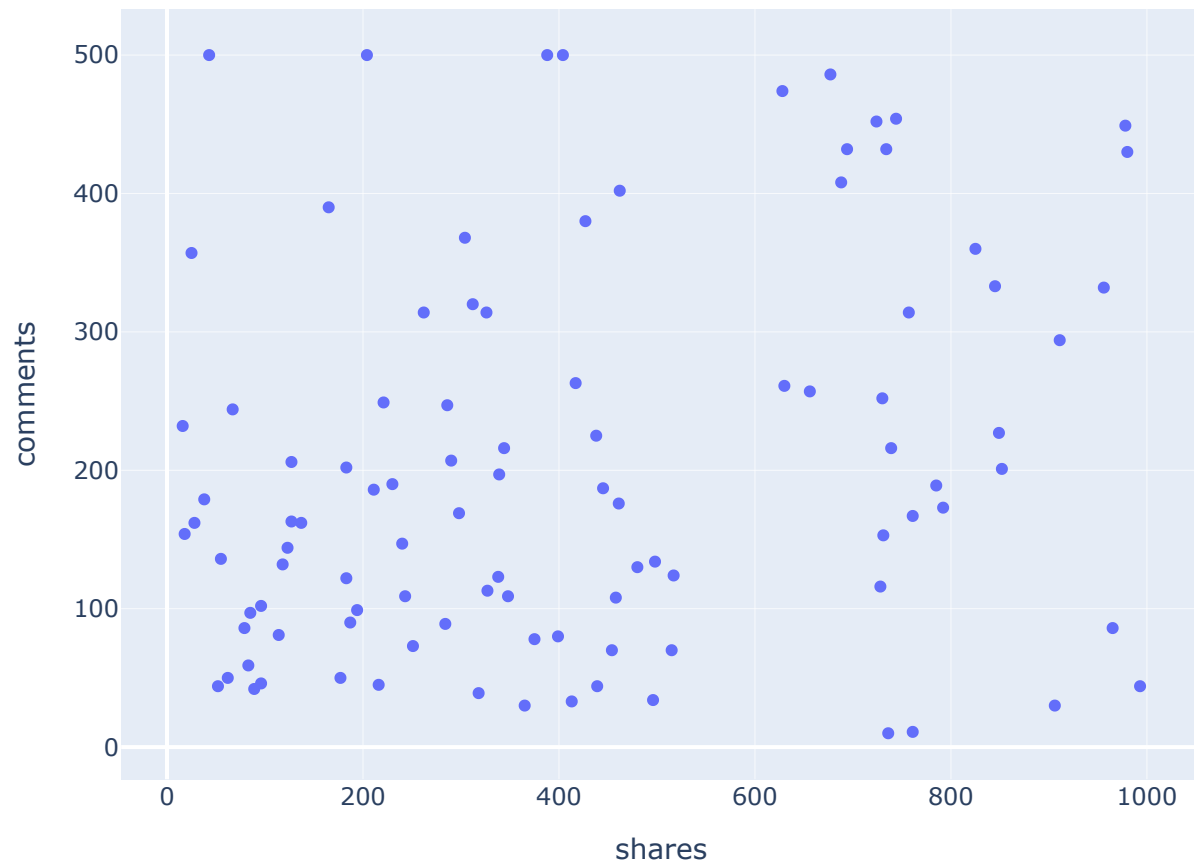
	post_id	platform	post_type	post_time	likes	comments	shares	post_day	sentiment_score
0	1	Facebook	image	8/17/2023 14:45	2121	474	628	Thursday	positive
1	2	Facebook	carousel	5/14/2023 0:45	3660	432	694	Sunday	neutral
2	3	Instagram	poll	2/21/2023 16:15	4955	408	688	Tuesday	negative
3	4	Twitter	image	11/16/2023 0:45	1183	90	187	Thursday	negative
4	5	Twitter	video	5/23/2023 0:30	3499	247	286	Tuesday	positive
...	...	...	...	...	...	...	...	...	...
95	96	Instagram	carousel	7/12/2023 17:45	36	294	911	Wednesday	positive
96	97	Twitter	video	10/27/2023 23:45	314	108	458	Friday	neutral
97	98	Twitter	text	8/5/2023 8:45	229	179	38	Saturday	positive
98	99	Instagram	poll	12/29/2023 12:15	5000	500	204	Friday	positive
99	100	Instagram	image	6/6/2023 21:00	4483	357	25	Tuesday	neutral

100 rows × 9 columns

```
In [56]: fig=px.scatter(df,x="likes",y="comments",width=700)
fig.show()
```



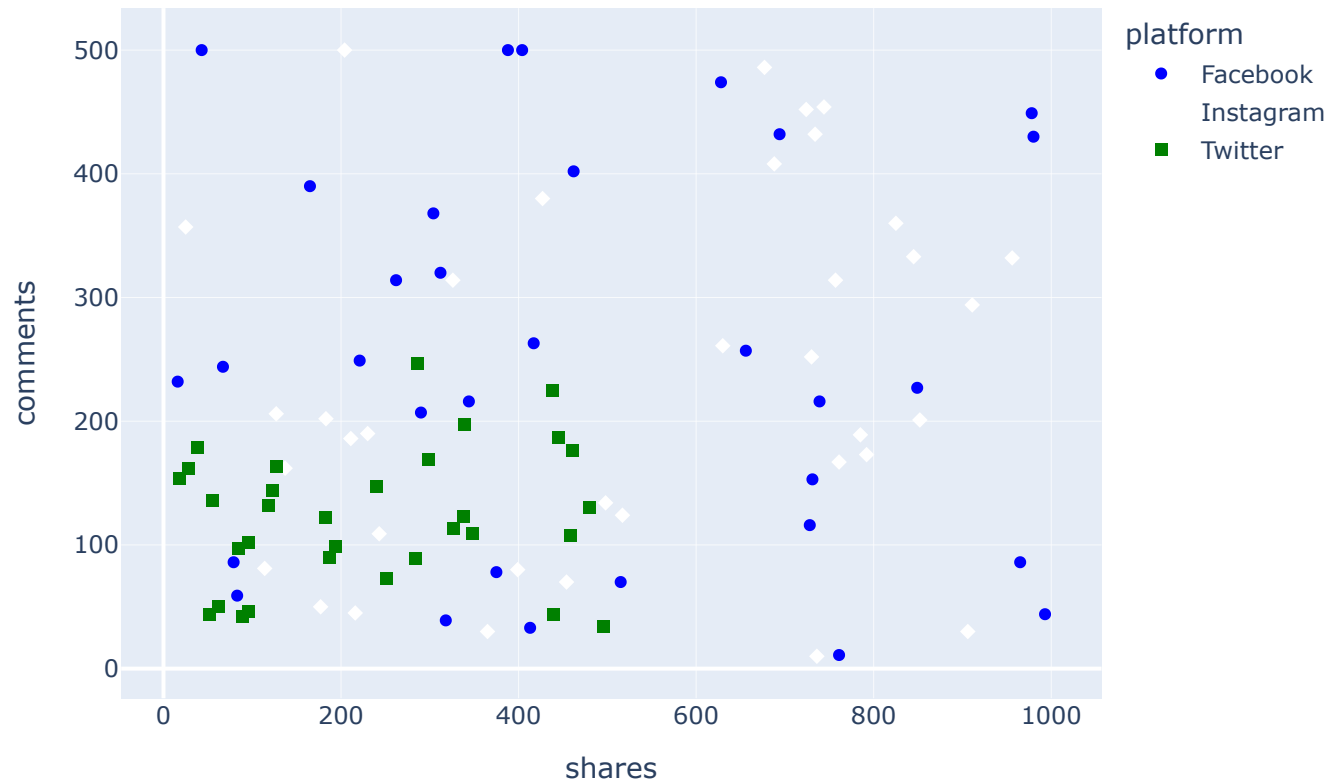
```
In [58]: fig=px.scatter(df,x="shares",y="comments",width=700)
fig.show()
```



Group Scatter Chart

```
In [59]: fig=px.scatter(df,x="shares",y="comments",color="platform",width=700,title="scatter_plot",  
                    color_discrete_map={"Facebook":"blue","Instagram":"white","Twitter":"green"},symbol="platform")  
fig.show()
```

scatter_plot



```
In [60]: df
```

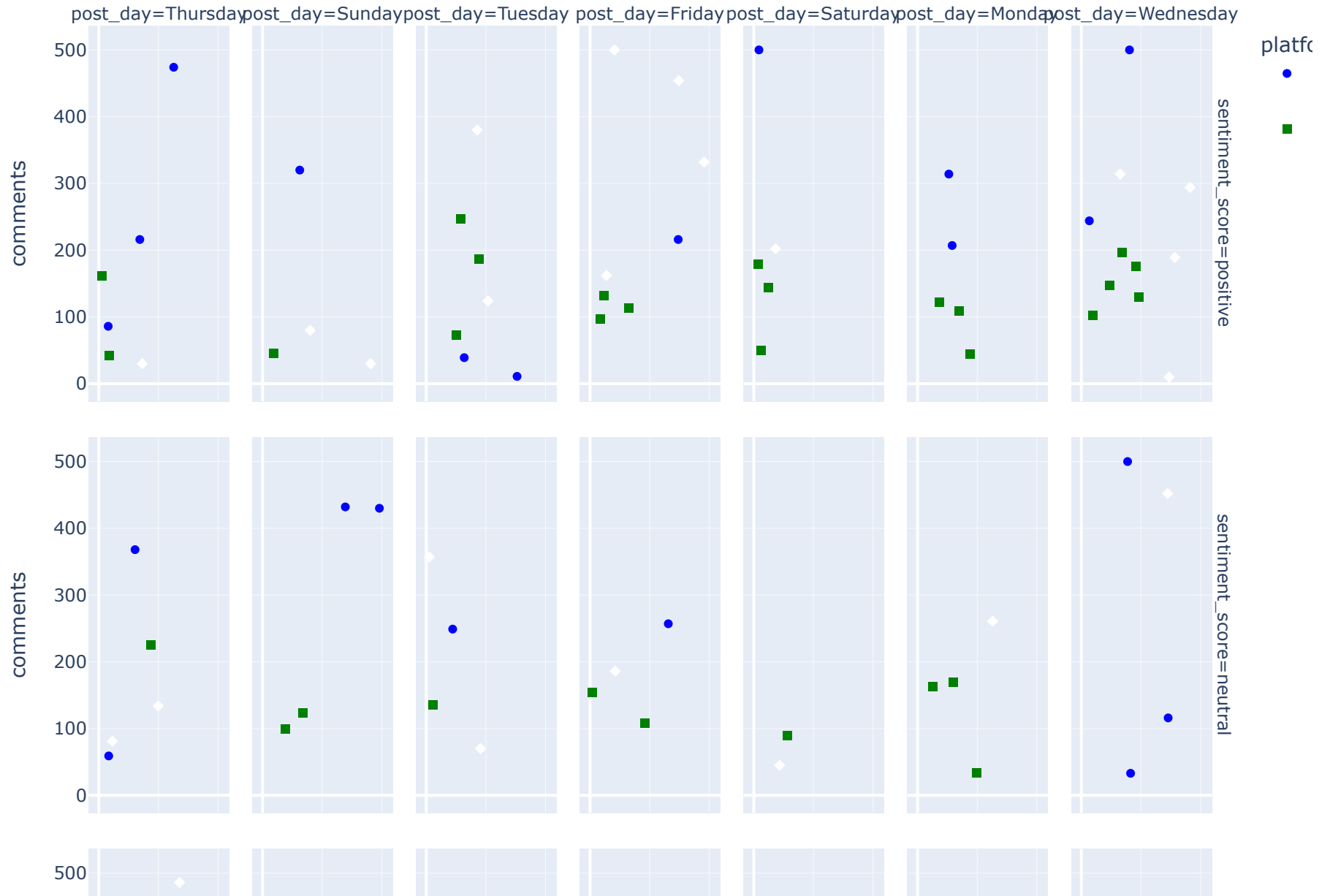
Out[60]:

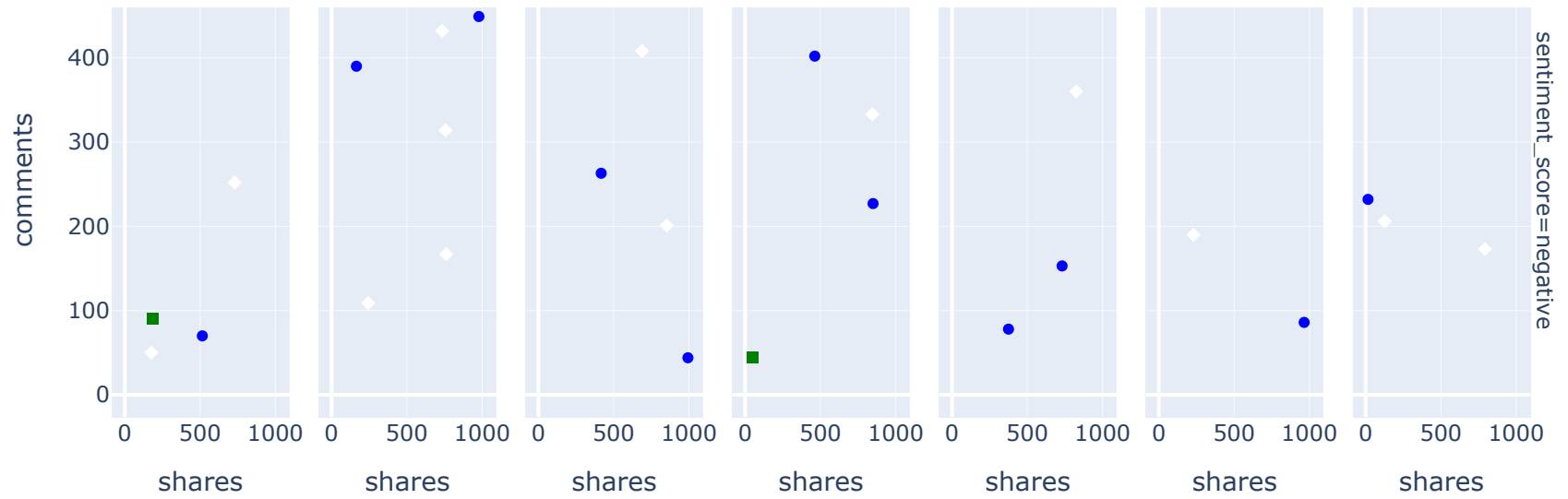
	post_id	platform	post_type	post_time	likes	comments	shares	post_day	sentiment_score
0	1	Facebook	image	8/17/2023 14:45	2121	474	628	Thursday	positive
1	2	Facebook	carousel	5/14/2023 0:45	3660	432	694	Sunday	neutral
2	3	Instagram	poll	2/21/2023 16:15	4955	408	688	Tuesday	negative
3	4	Twitter	image	11/16/2023 0:45	1183	90	187	Thursday	negative
4	5	Twitter	video	5/23/2023 0:30	3499	247	286	Tuesday	positive
...	...	...	...	...	...	...	...	...	...
95	96	Instagram	carousel	7/12/2023 17:45	36	294	911	Wednesday	positive
96	97	Twitter	video	10/27/2023 23:45	314	108	458	Friday	neutral
97	98	Twitter	text	8/5/2023 8:45	229	179	38	Saturday	positive
98	99	Instagram	poll	12/29/2023 12:15	5000	500	204	Friday	positive
99	100	Instagram	image	6/6/2023 21:00	4483	357	25	Tuesday	neutral

100 rows × 9 columns

```
In [61]: fig=px.scatter(df,x="shares",y="comments",color="platform",width=1000,height=1000,title="scatter_plot",facet_col="post_day",
                color_discrete_map={"Facebook":"blue","Instagram":"white","Twitter":"green"},symbol="platform",
                facet_row="sentiment_score"
            )
fig.show()
```

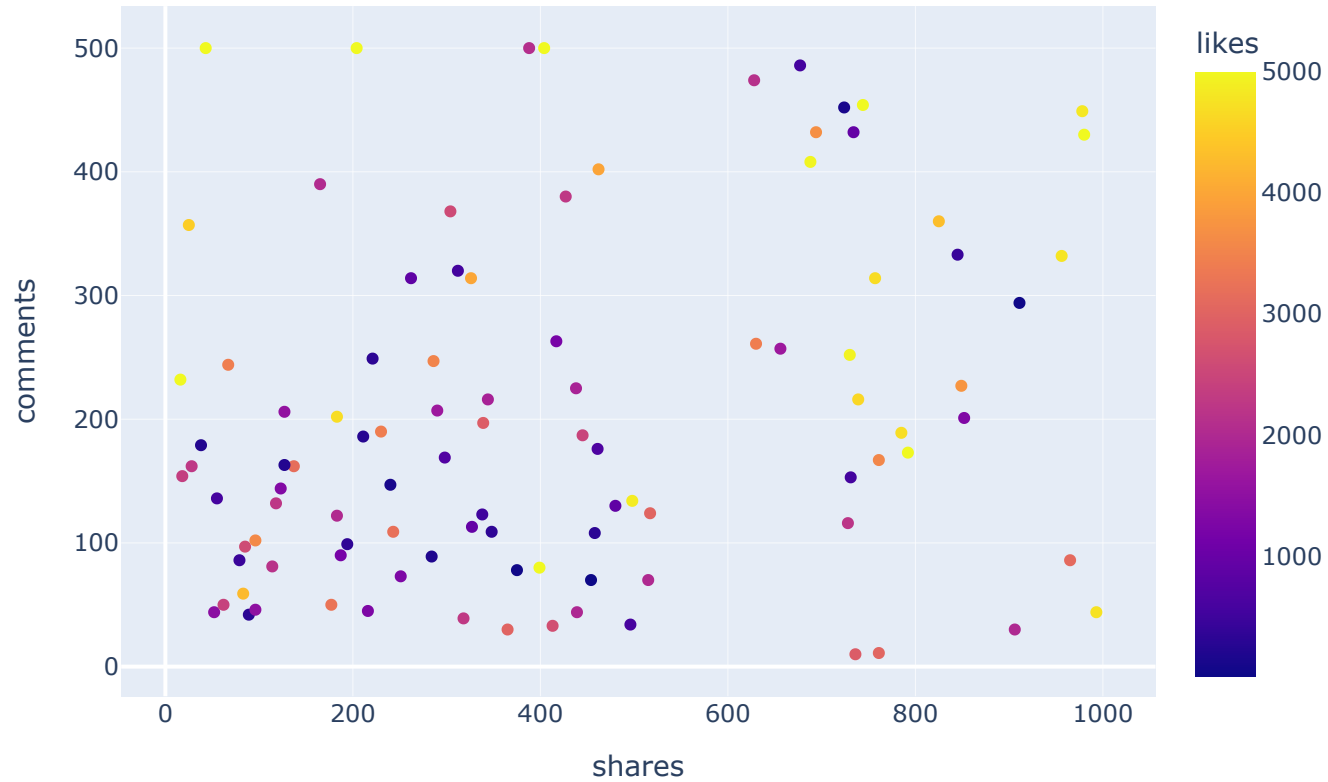
scatter_plot





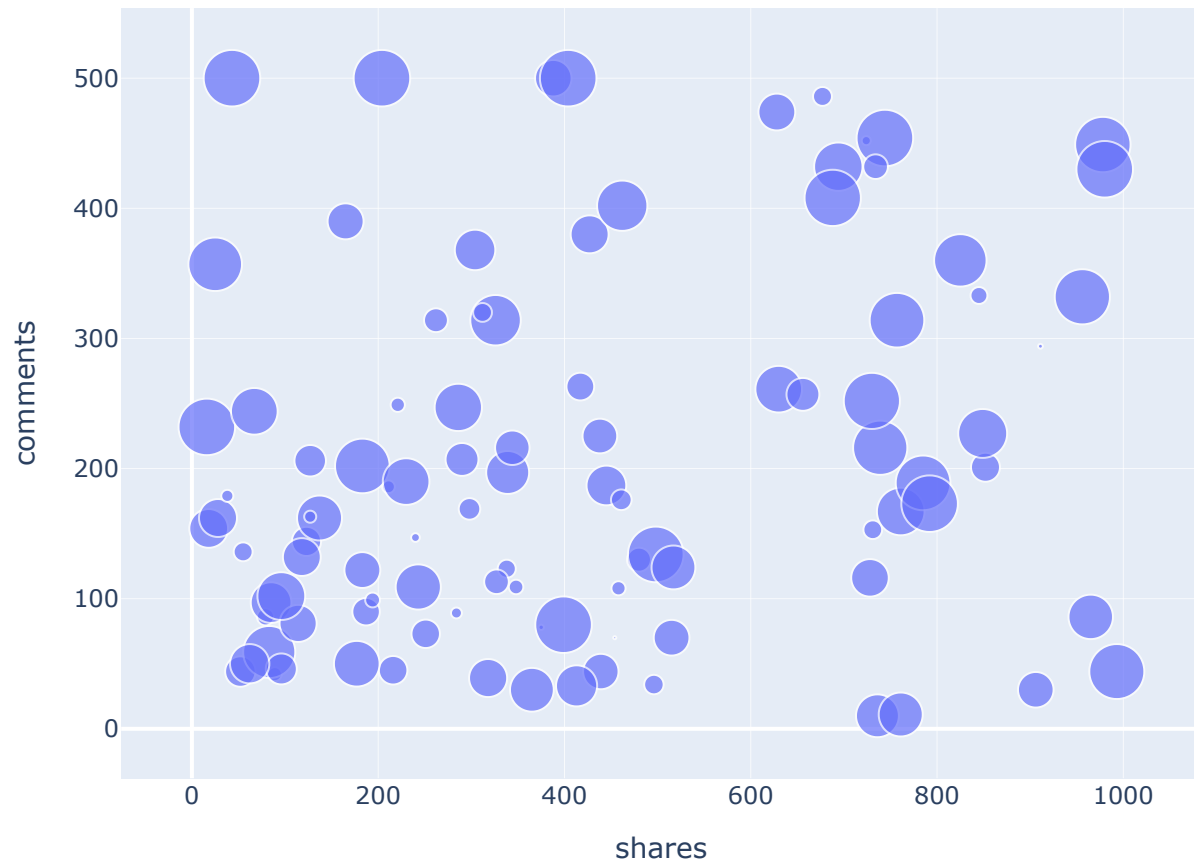
```
In [62]: fig=px.scatter(df,x="shares",y="comments",color="likes",width=700,title="scatter_plot")  
fig.show()
```

scatter_plot

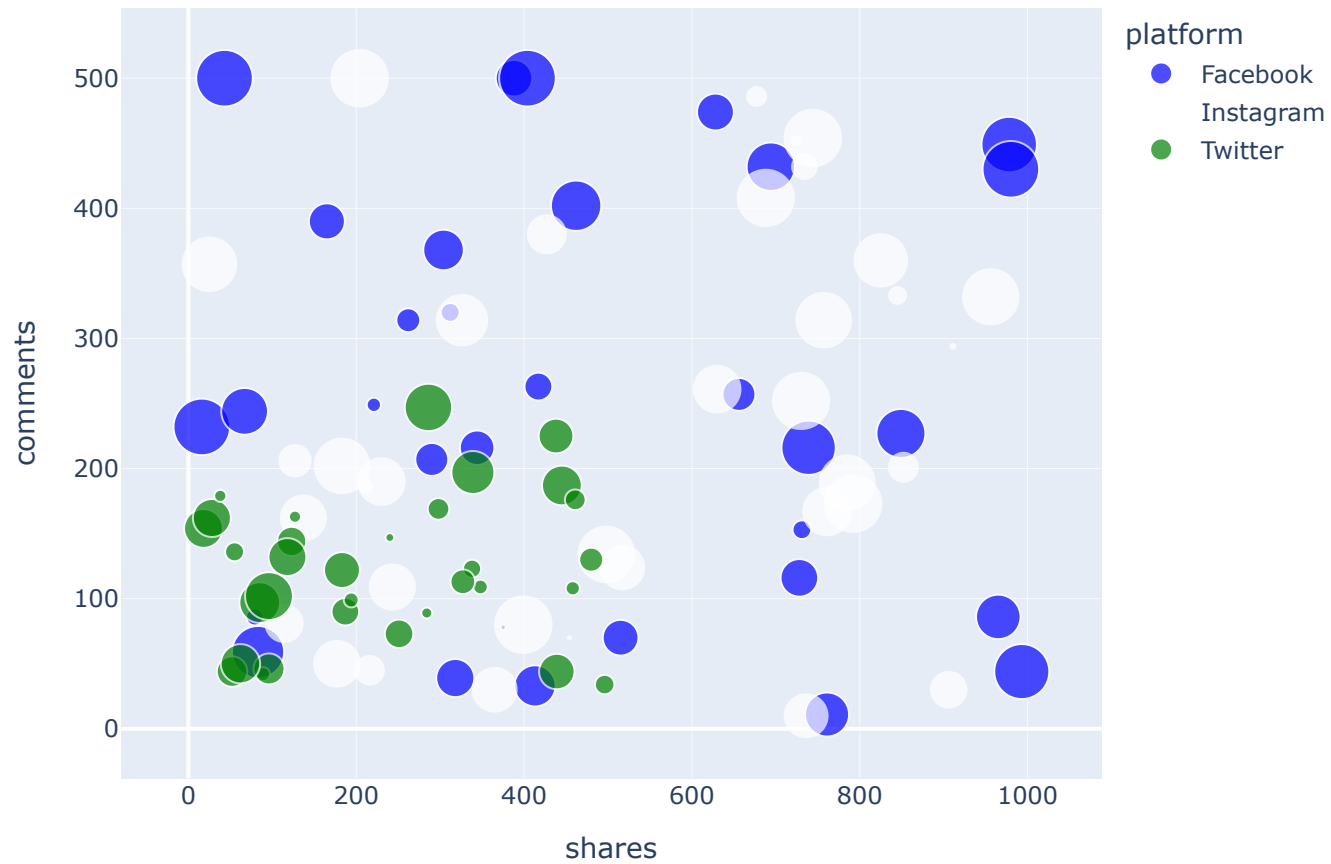


Bubble Char

```
In [63]: fig=px.scatter(df,x="shares",y="comments",width=700,size="likes")
fig.show()
```



```
In [64]: fig=px.scatter(df,x="shares",y="comments",color="platform",width=700,size="likes",  
                      color_discrete_map={"Facebook":"blue","Instagram":"white","Twitter":"green"})  
fig.show()
```



px.scatter_3d() Parameters

df – DataFrame containing the dataset.

x – column for x-axis variable.

y – column for y-axis variable.

z – column for z-axis variable (3rd dimension).

color – colors points by category or numeric value.

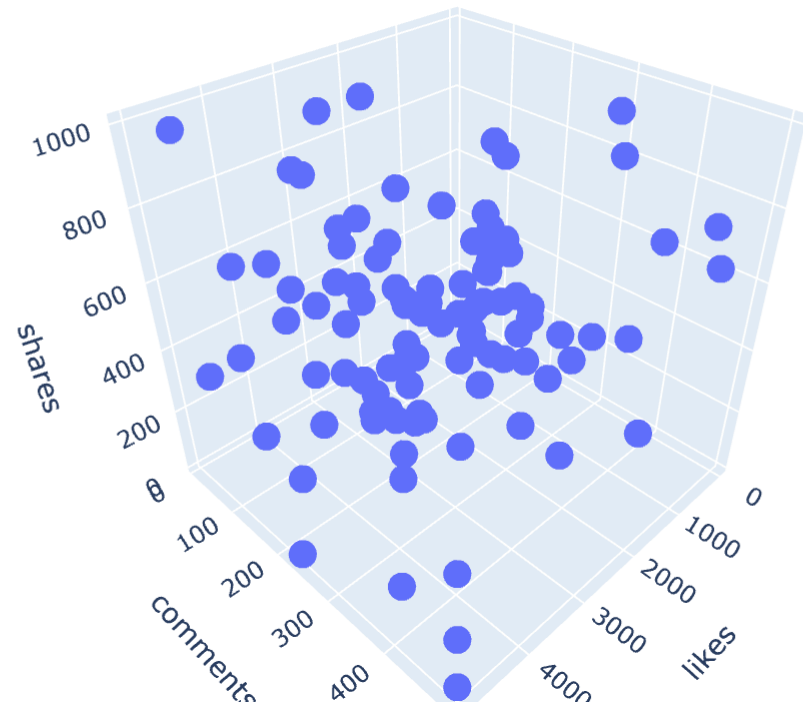
size – controls marker size based on a column.

title – adds title to the 3D scatter plot.

3d scatter PLOT

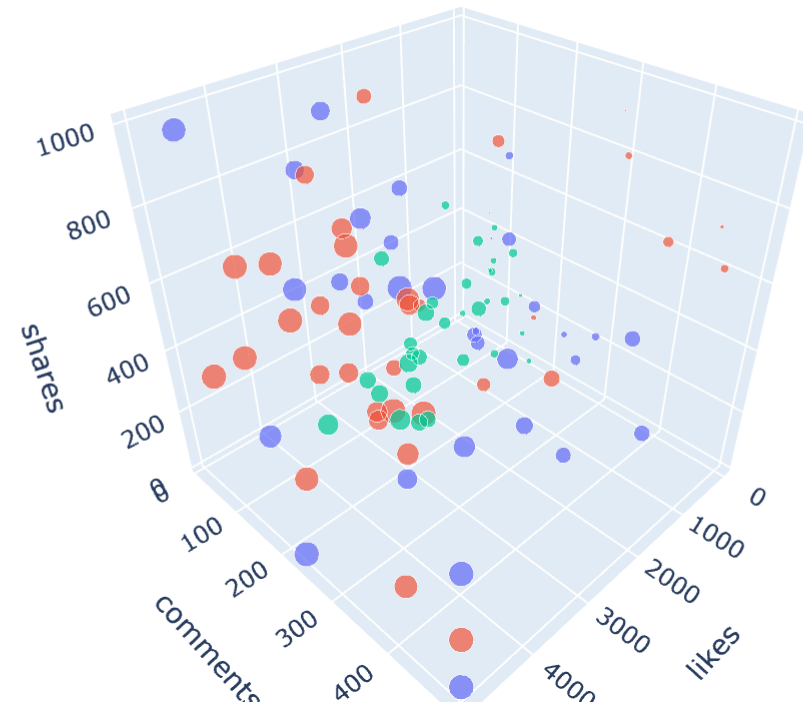
```
In [65]: fig=px.scatter_3d(df,x="likes",y="comments",z="shares")  
fig.show()
```

Vishal Acharya

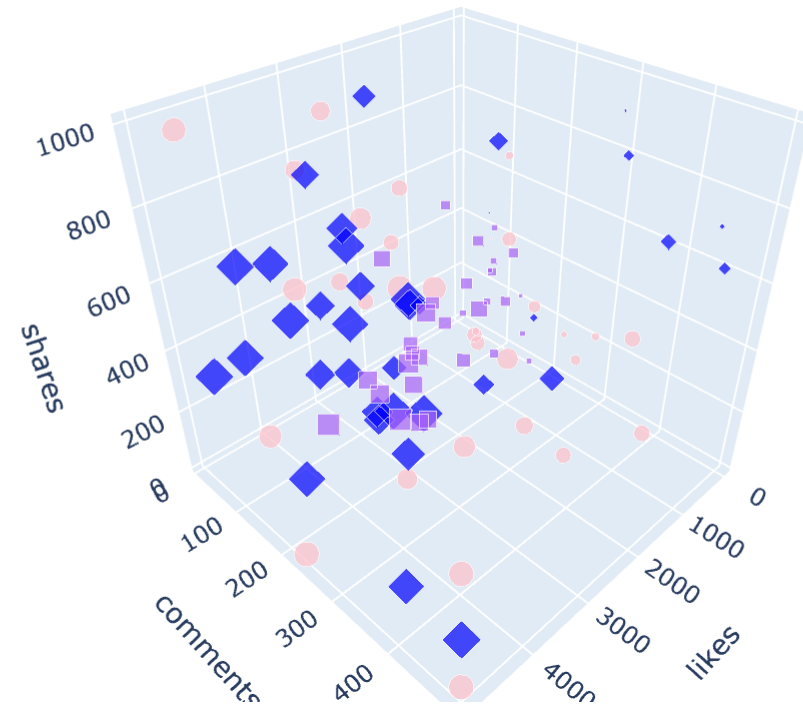


```
In [66]: fig=px.scatter_3d(df,x="likes",y="comments",z="shares",color="platform",size="likes")
fig.show()
```

Vishal Acharya



```
In [67]: fig=px.scatter_3d(df,x="likes",y="comments",z="shares",color="platform",size="likes",symbol="platform",  
color_discrete_map={"Facebook":"pink","Instagram":"blue","Twitter":"yellow"})  
fig.show()
```



px.scatter_matrix() Parameters

df – dataset used for the scatter matrix.

dimensions – list of columns used to create pairwise scatter plots.

title – title of the scatter matrix figure.

color – colors points by category column.

height – sets figure height.

width – sets figure width.

In [68]:

```
df
```

Out[68]:

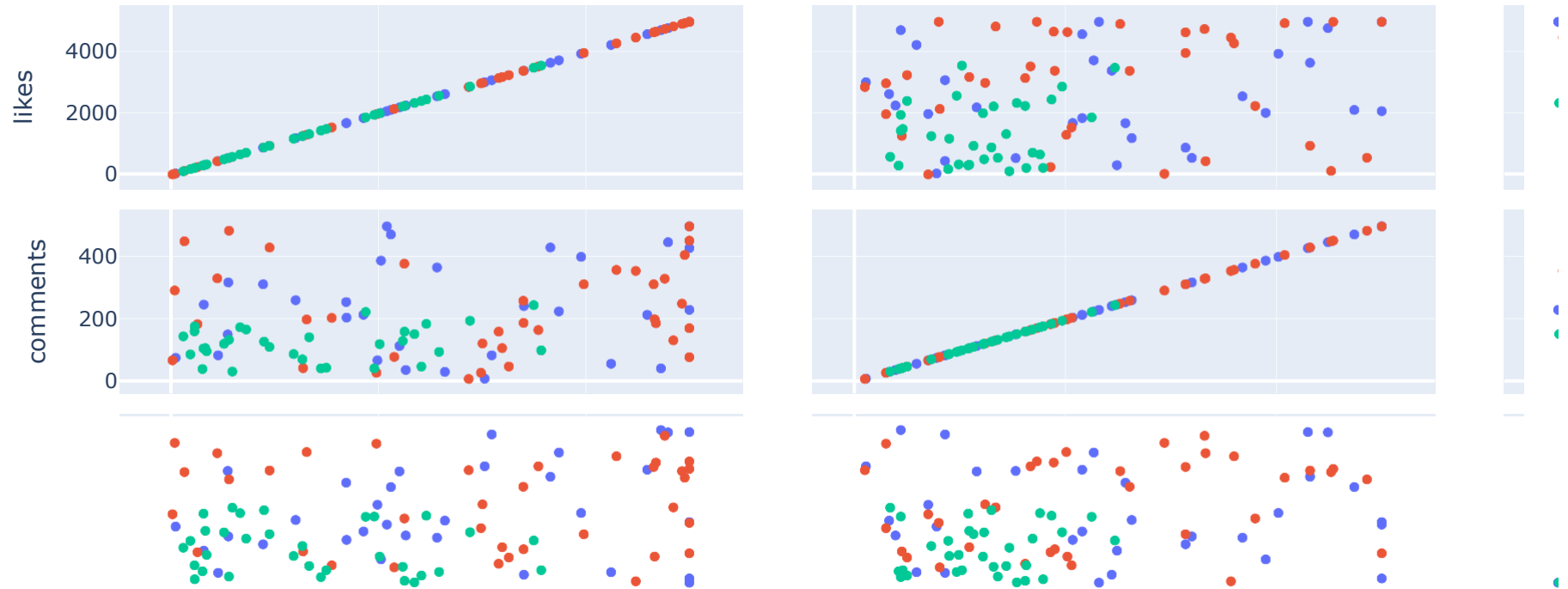
	post_id	platform	post_type	post_time	likes	comments	shares	post_day	sentiment_score
0	1	Facebook	image	8/17/2023 14:45	2121	474	628	Thursday	positive
1	2	Facebook	carousel	5/14/2023 0:45	3660	432	694	Sunday	neutral
2	3	Instagram	poll	2/21/2023 16:15	4955	408	688	Tuesday	negative
3	4	Twitter	image	11/16/2023 0:45	1183	90	187	Thursday	negative
4	5	Twitter	video	5/23/2023 0:30	3499	247	286	Tuesday	positive
...	...	...	...	...	...	...	...	...	...
95	96	Instagram	carousel	7/12/2023 17:45	36	294	911	Wednesday	positive
96	97	Twitter	video	10/27/2023 23:45	314	108	458	Friday	neutral
97	98	Twitter	text	8/5/2023 8:45	229	179	38	Saturday	positive
98	99	Instagram	poll	12/29/2023 12:15	5000	500	204	Friday	positive
99	100	Instagram	image	6/6/2023 21:00	4483	357	25	Tuesday	neutral

100 rows × 9 columns

In [69]:

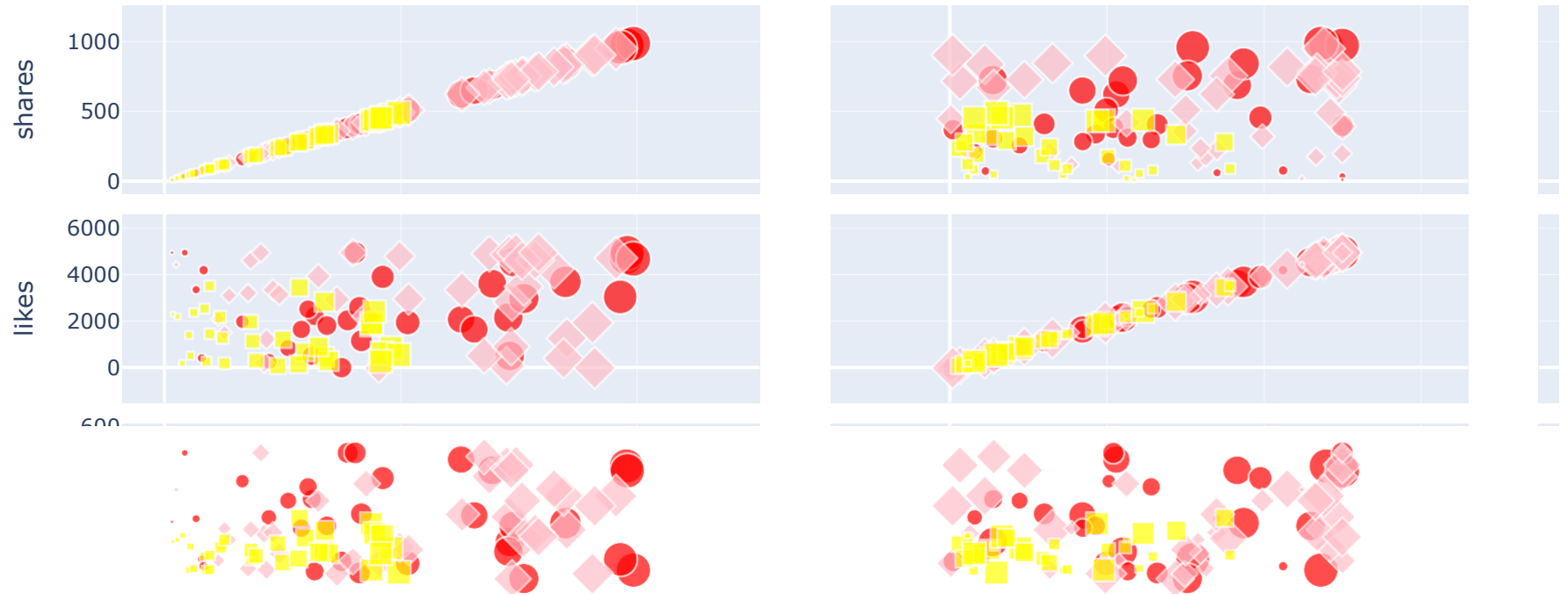
```
px.scatter_matrix(df,dimensions=["likes","comments","shares"],title="scatter matrix",color="platform")
```

scatter matrix



```
In [70]: px.scatter_matrix(df,dimensions=["likes","comments","shares"],title="scatter matrix",color="platform",size="shares",
symbol="platform",color_discrete_sequence=["red","pink","yellow"])
```

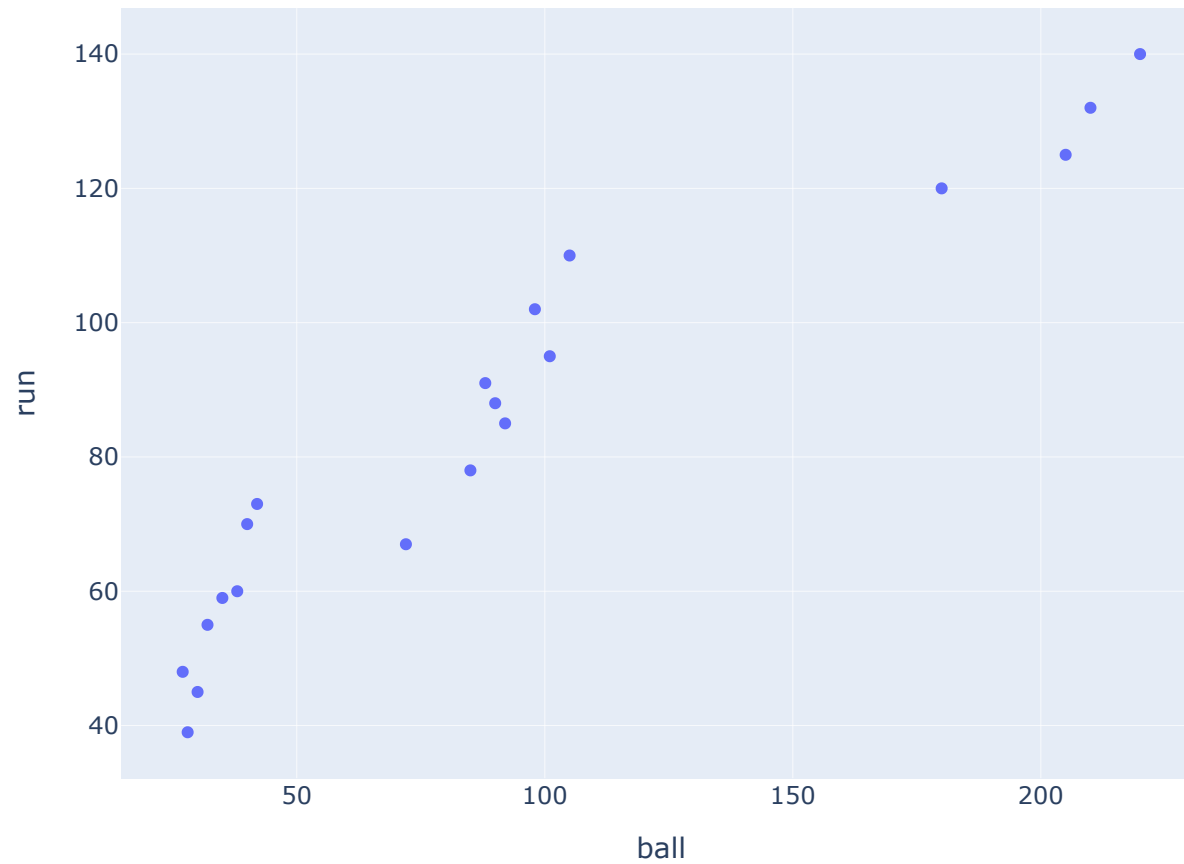
scatter matrix



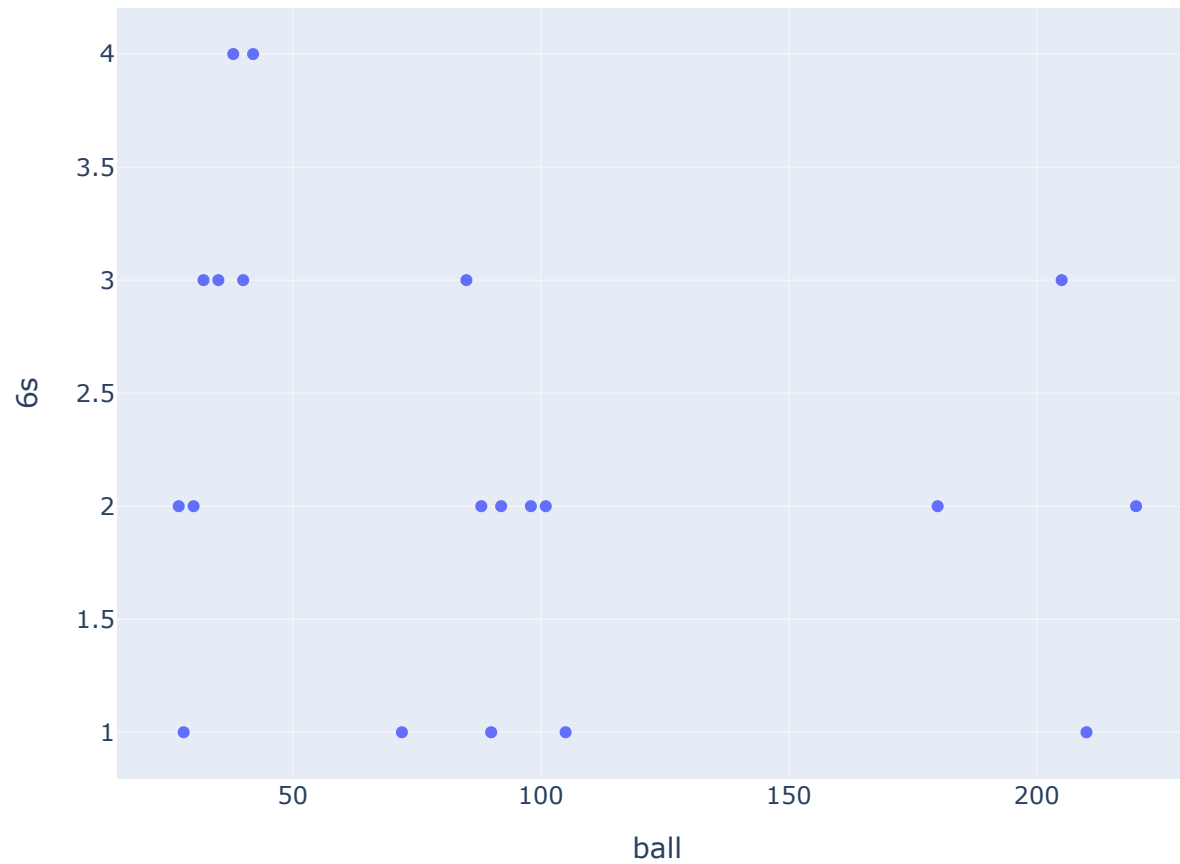
```
In [77]: df=pd.read_csv("bar.csv")
```

```
In [72]: fig=px.scatter(df,x="ball",y="run",width=700)
fig.show()
```

Vishal Acharya

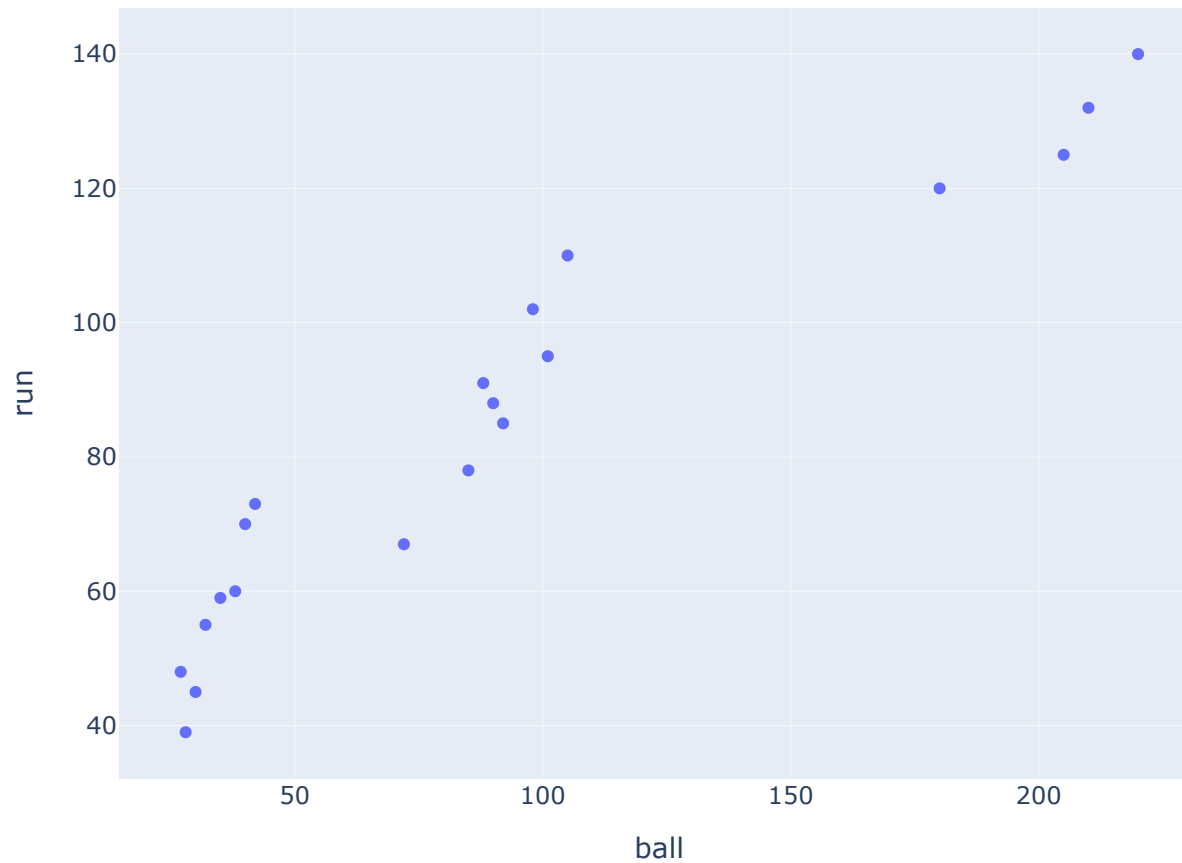


```
In [73]: fig=px.scatter(df,x="ball",y="6s",width=700)
fig.show()
```



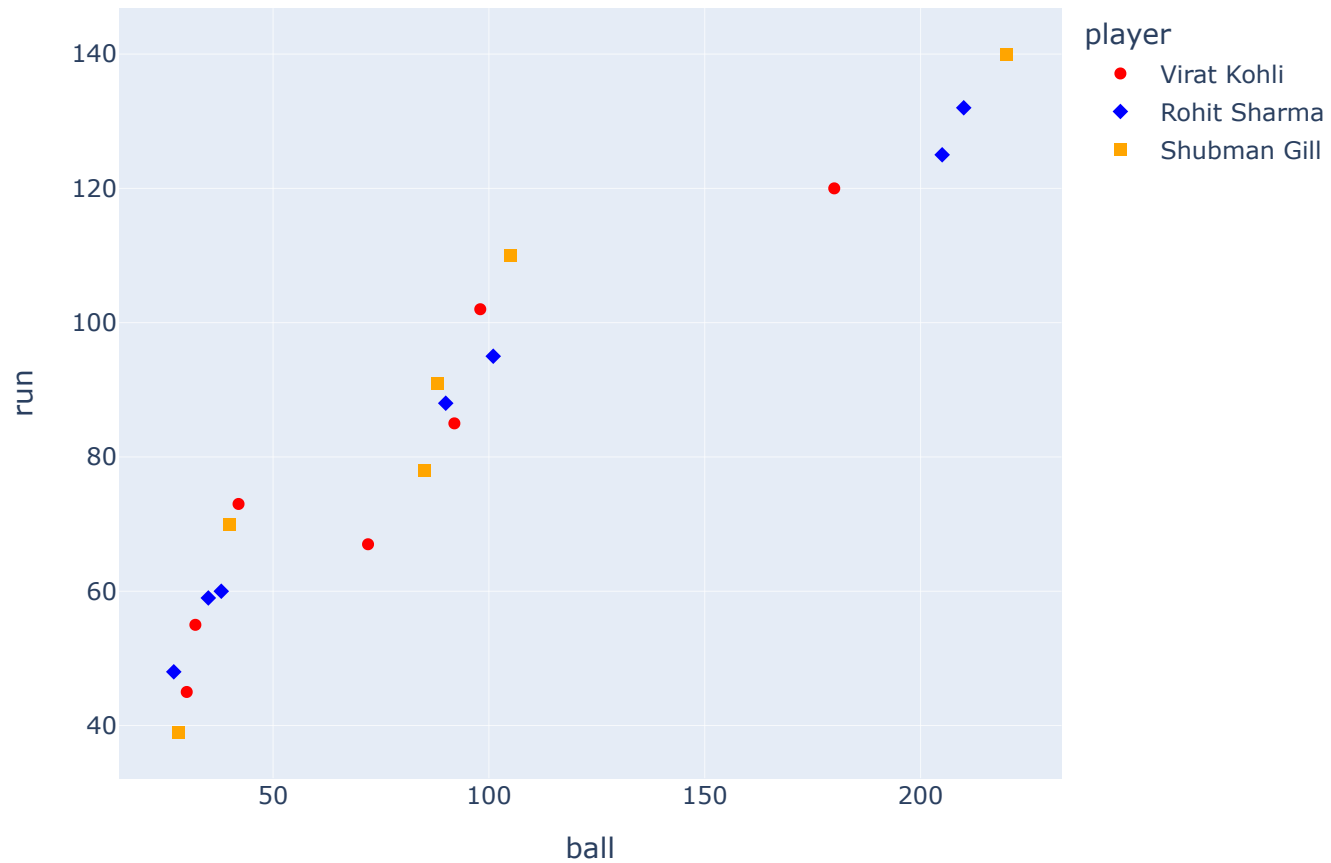
```
In [78]: fig=px.scatter(df,x="ball",y="run",width=700)  
fig.show()
```

Vishal Acharya



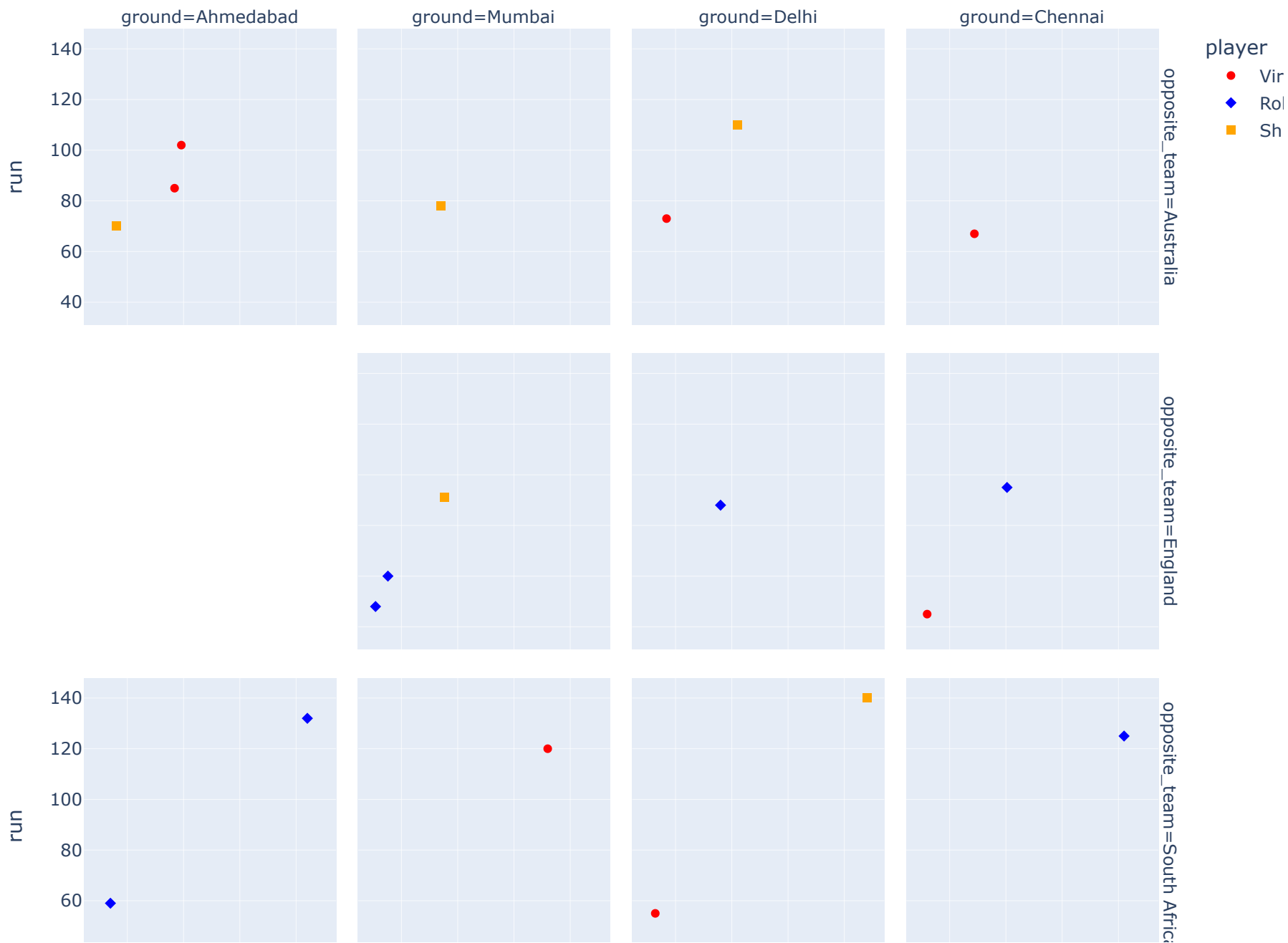
```
In [79]: fig=px.scatter(df,x="ball",y="run",color="player",width=700,  
                        color_discrete_map={"Virat Kohli":"red","Rohit Sharma":"blue","Shubman Gill":"orange"},symbol="player")  
fig.show()
```

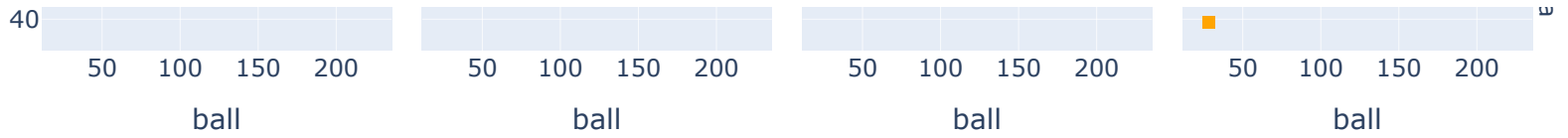
Vishal Acharya



```
In [80]: fig=px.scatter(df,x="ball",y="run",color="player",width=1000,height=800,  
                      color_discrete_map={"Virat Kohli":"red","Rohit Sharma":"blue","Shubman Gill":"orange"},symbol="player",  
                      facet_col="ground",facet_row="opposite_team")  
fig.show()
```

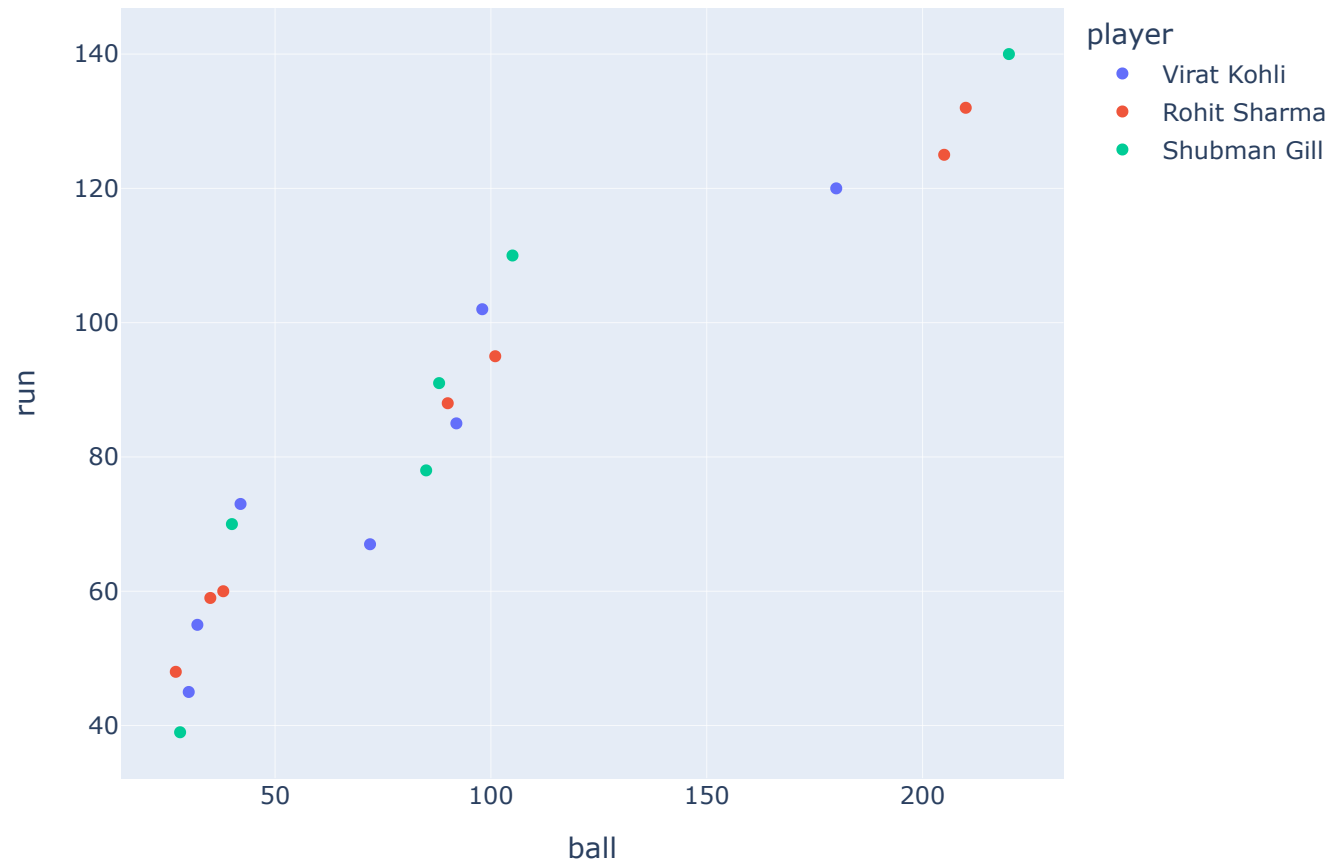
Vishal Acharya





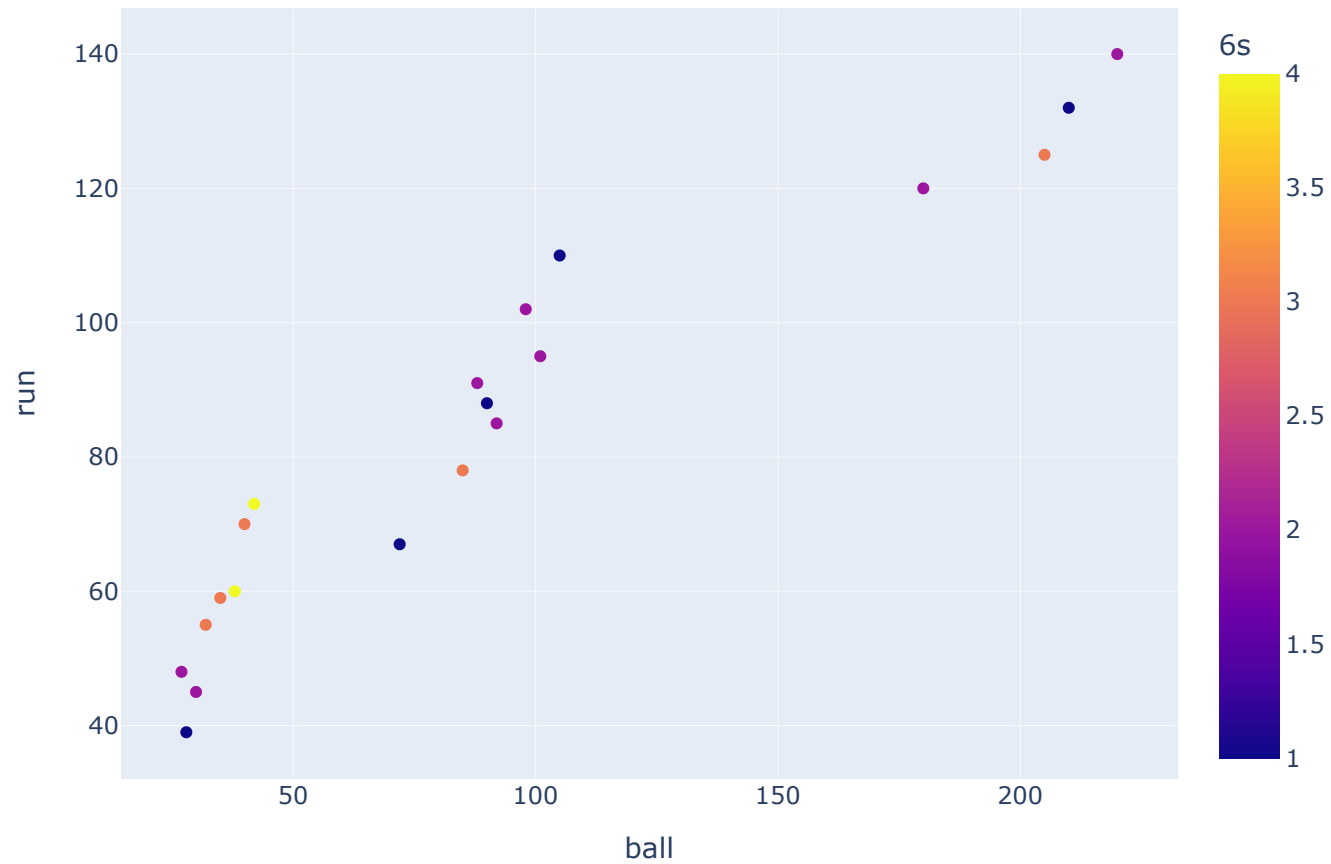
```
In [81]: fig=px.scatter(df,x="ball",y="run",width=700,color="player")  
fig.show()
```

Vishal Acharya



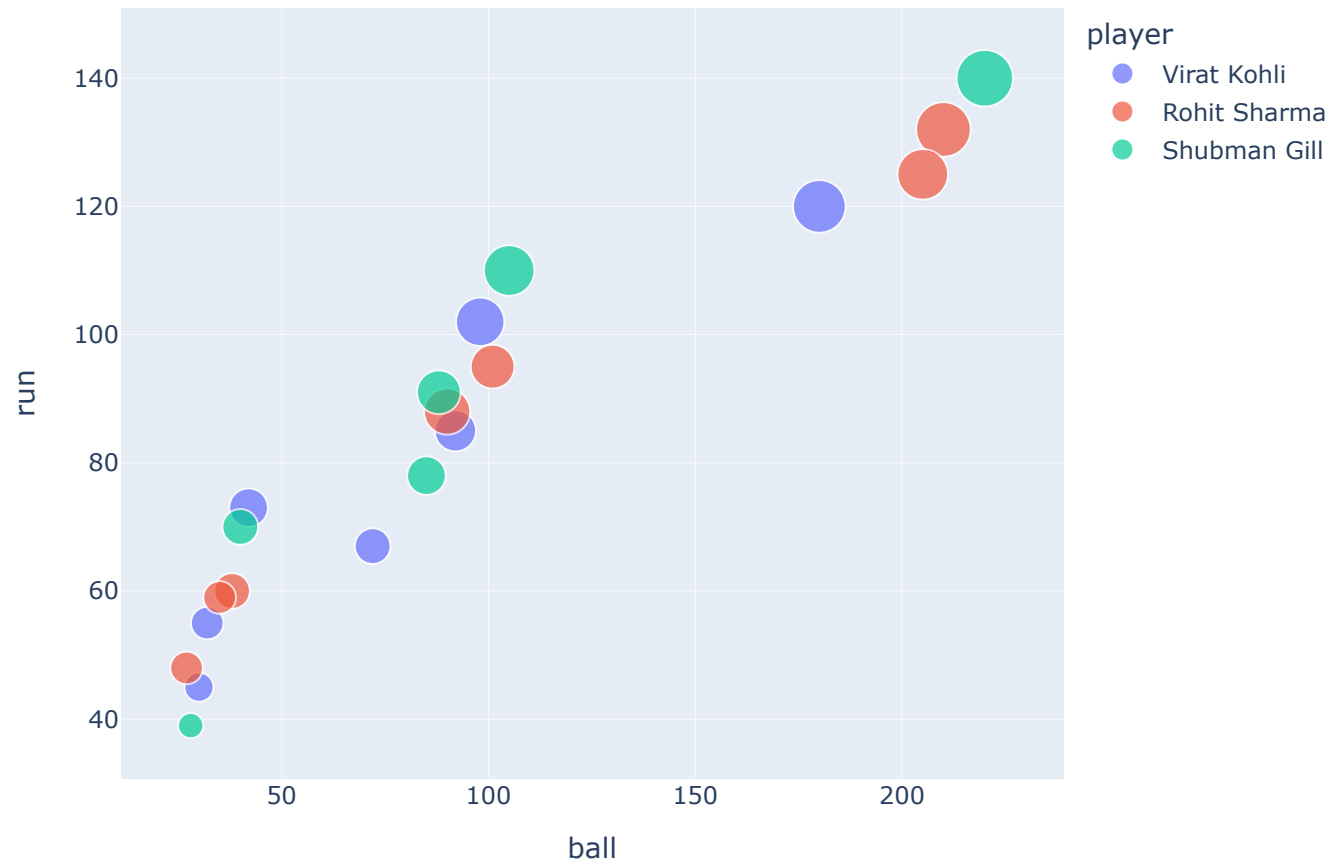
```
In [82]: fig=px.scatter(df,x="ball",y="run",width=700,color="6s")  
fig.show()
```

Vishal Acharya



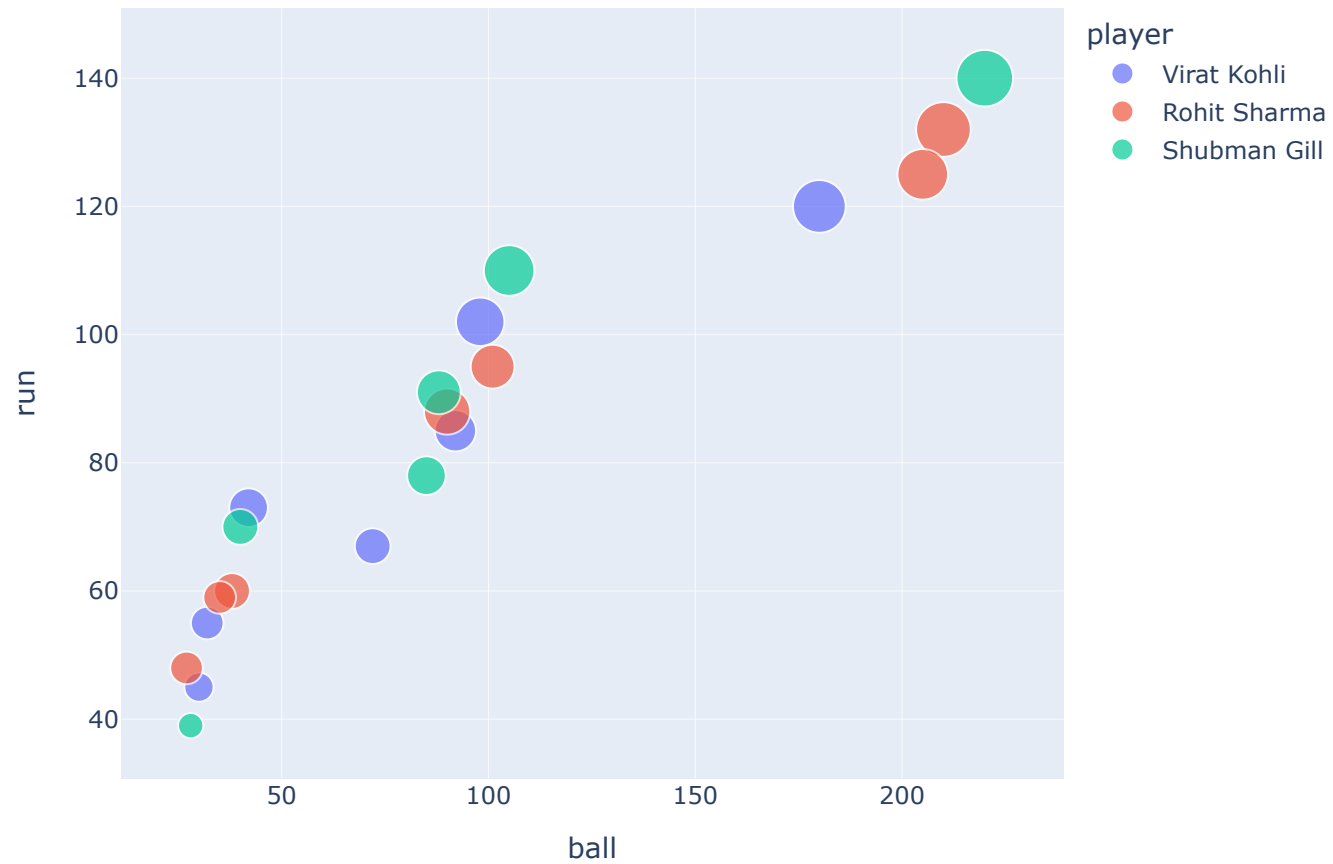
```
In [83]: fig=px.scatter(df,x="ball",y="run",width=700,color="player",size="4s")
fig.show()
```

Vishal Acharya



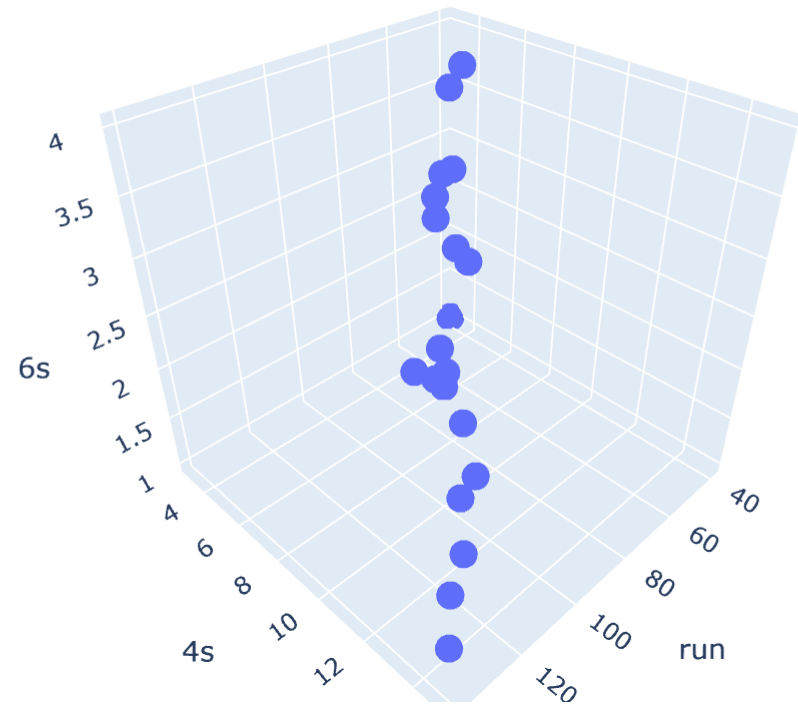
```
In [84]: fig=px.scatter(df,x="ball",y="run",width=700,color="player",size="4s")  
fig.show()
```

Vishal Acharya



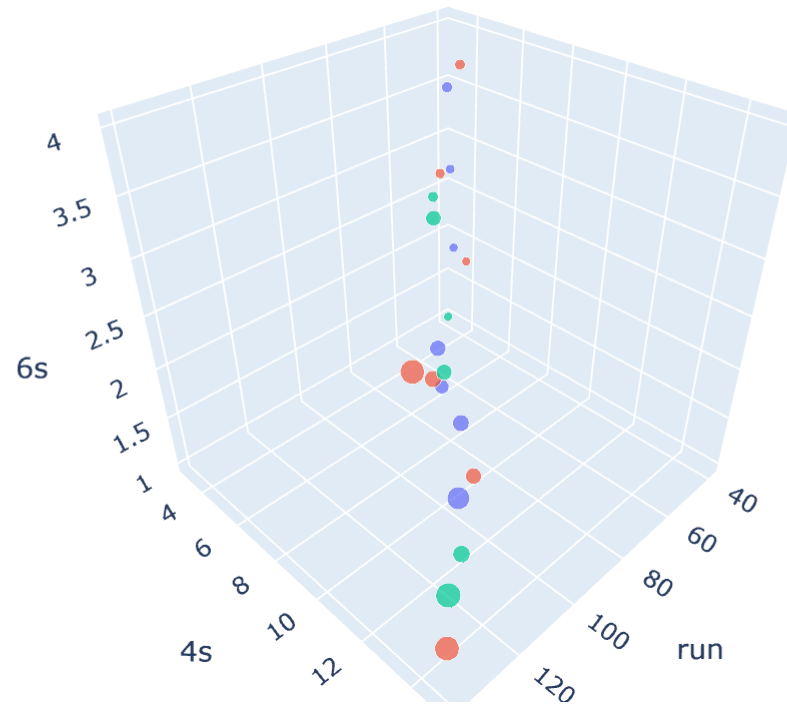
```
In [86]: px.scatter_3d(df, x="run", y="4s", z="6s")
```

Vishal Acharya



```
In [87]: px.scatter_3d(df,x="run",y="4s",z="6s",color="player",size="ball")
```

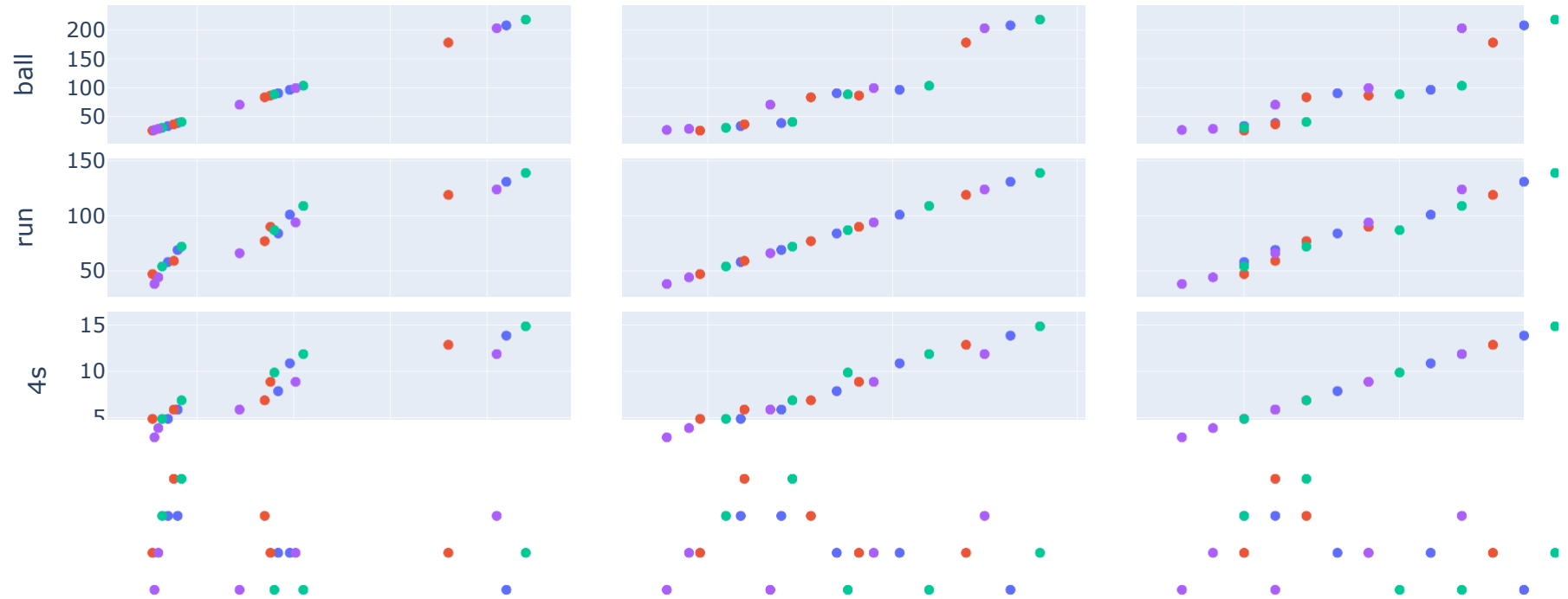
Vishal Acharya



```
In [88]: px.scatter_matrix(df,dimensions=["ball","run","4s","6s"],title="scattter matrix",color="ground")
```

Vishal Acharya

scattter matrix



```
In [259... %%writefile multi_line_cricket.csv
match_id,ground,player,run,ball
1,Ahmedabad,Kohli,78,52
2,Chennai,Kohli,65,48
3,Delhi,Kohli,92,60
4,Mumbai,Kohli,55,40
5,Ahmedabad,Kohli,88,58
```

```
6,Ahmedabad,Rohit,45,30
7,Chennai,Rohit,70,50
8,Delhi,Rohit,110,65
9,Mumbai,Rohit,60,42
10,Ahmedabad,Rohit,72,49

11,Ahmedabad,Hardik,35,25
12,Chennai,Hardik,50,32
13,Delhi,Hardik,67,40
14,Mumbai,Hardik,80,45
15,Ahmedabad,Hardik,90,55

16,Chennai,Kohli,74,51
17,Delhi,Rohit,85,54
18,Mumbai,Hardik,66,38
19,Ahmedabad,Rohit,95,60
20,Chennai,Hardik,77,44
```

Writing multi_line_cricket.csv

Line chart

Line Chart Types in Plotly

1. Single Line Chart

Shows trend of one variable over another variable (usually time or sequence).

Example: Sales over months.

```
px.line(df, x="Month", y="Sales")
```

2. Multi Line Chart

Shows multiple lines in one chart to compare categories.

Example: Sales of different regions over months.

```
px.line(df, x="Month", y="Sales", color="Region")
```

color separates the lines by category.

Chart	Purpose
Simple Line Chart	Shows trend of one variable
Multi Line Chart	Compares trends of multiple categories

In [263...

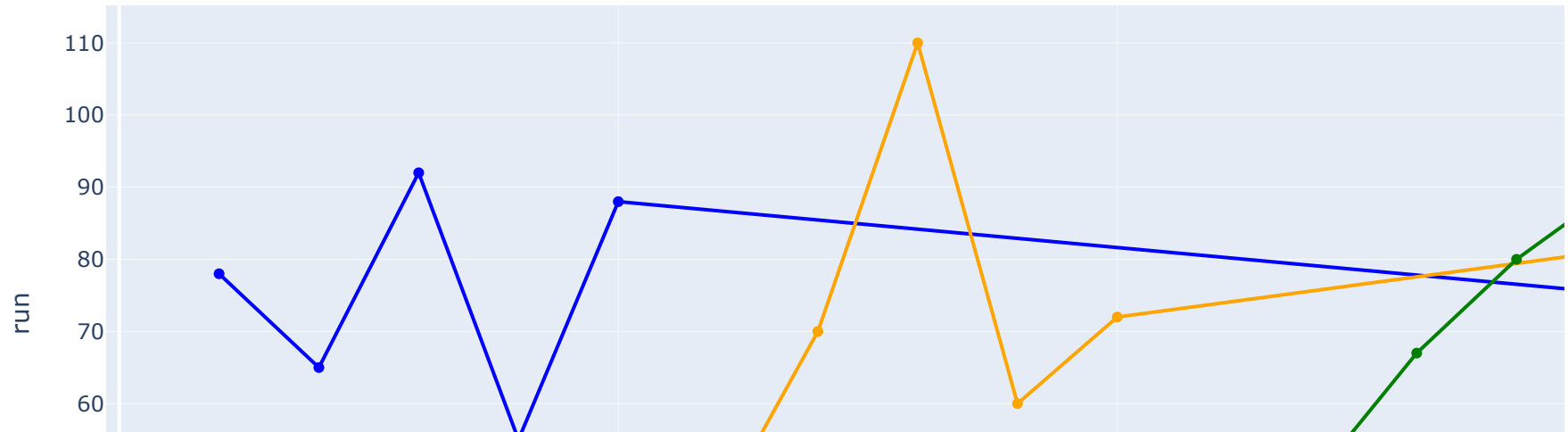
```
import pandas as pd
import plotly.express as px

# Load CSV
df = pd.read_csv("multi_line_cricket.csv")

# Multi-line chart
fig = px.line(df,
              x="match_id",
              y="run",
              color="player",
              markers=True,
              hover_data=["ground", "ball"],
              color_discrete_map={
                  "Kohli": "blue",
                  "Rohit": "orange",
                  "Hardik": "green"
              },
              title="Multi-Line Chart: Player Runs Across Matches")

fig.show()
```

Multi-Line Chart: Player Runs Across Matches



In [269...]

```
px.line(df,x="match_id",y="run")
```

Vishal Acharya



```
In [89]: import pandas as pd
import plotly.express as px
```

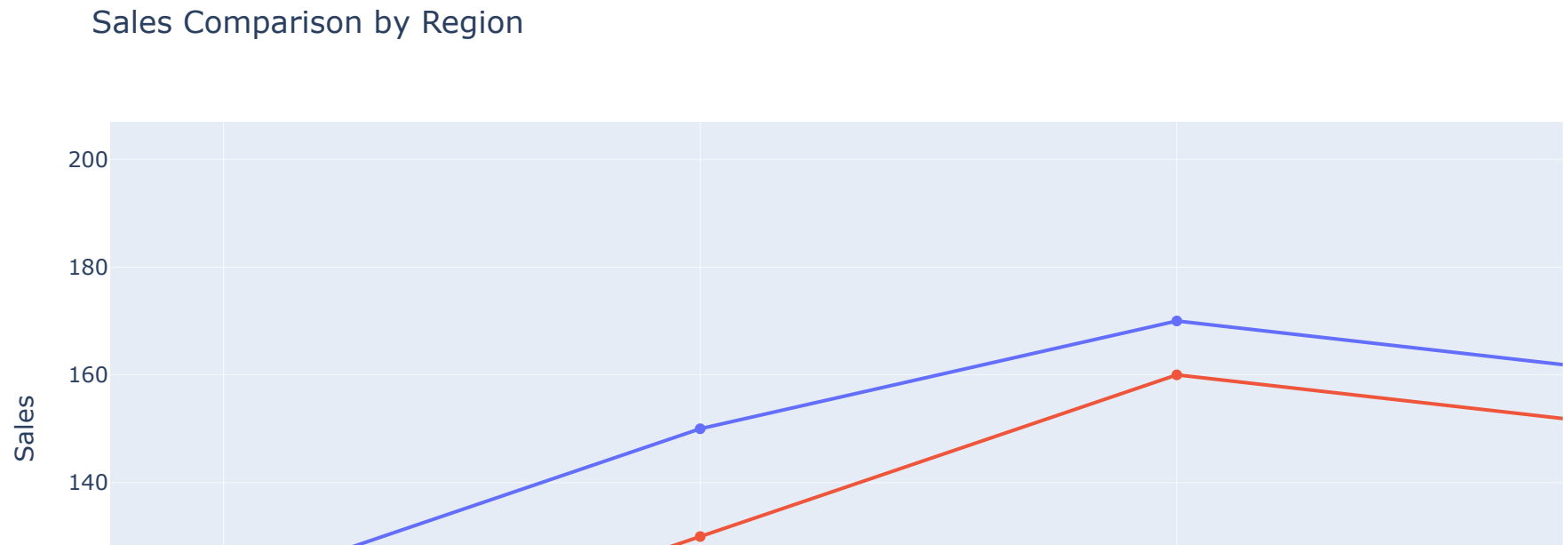
```
df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [120, 150, 170, 160, 200]
})
```

```
fig = px.line(  
    df,  
    x="Month",  
    y="Sales",  
    title="Monthly Sales Trend",  
    markers=True  
)  
  
fig.show()
```



```
In [90]: df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [120, 150, 170, 160, 200, 100, 130, 160, 150, 180],
    "Region": ["East", "East", "East", "East", "East",
               "West", "West", "West", "West", "West"]
})
fig = px.line(
    df,
```

```
x="Month",  
y="Sales",  
color="Region",  
title="Sales Comparison by Region",  
markers=True  
)  
  
fig.show()
```



Histogram

A Histogram shows the distribution of numerical data by grouping values into bins (ranges) and counting how many values fall in each bin.

Example: distribution of ages, marks, salary, etc.

```
px.histogram(df, x, y, nbins, color, histfunc, pattern_shape, text_auto, color_discrete_map, color_discrete_sequence)
```

Parameters

df – DataFrame containing the dataset.

x – column whose distribution is shown on the x-axis.

y – optional column used for aggregation values when using histfunc.

nbins – defines the number of bins (interval groups) in the histogram.

color – separates bars by category using different colors.

histfunc – aggregation function applied to y (e.g., count, sum, avg, min, max).

pattern_shape – adds patterns to histogram bars based on category.

text_auto – automatically displays values on bars.

color_discrete_map – assigns specific colors to particular category values.

color_discrete_sequence – defines a list of colors used in the histogram.

Parameter	Use
x	variable distribution
nbins	number of bins

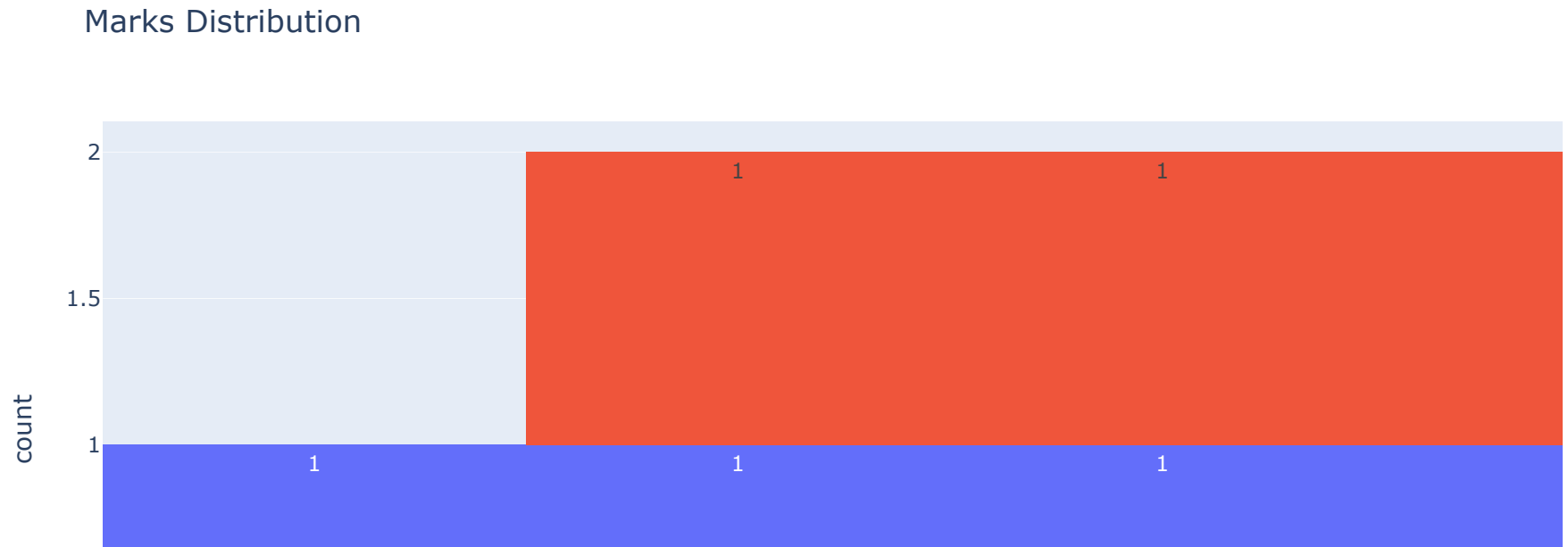
Parameter	Use
color	category grouping
histfunc	aggregation function
pattern_shape	bar pattern style
text_auto	show values on bars
color_discrete_map	manual color mapping
color_discrete_sequence	color list for bars

```
In [91]: import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    "Marks": [55, 60, 65, 70, 75, 80, 85, 90, 95],
    "Department": ["CS", "IT", "CS", "IT", "CS", "IT", "CS", "IT", "CS"]
})

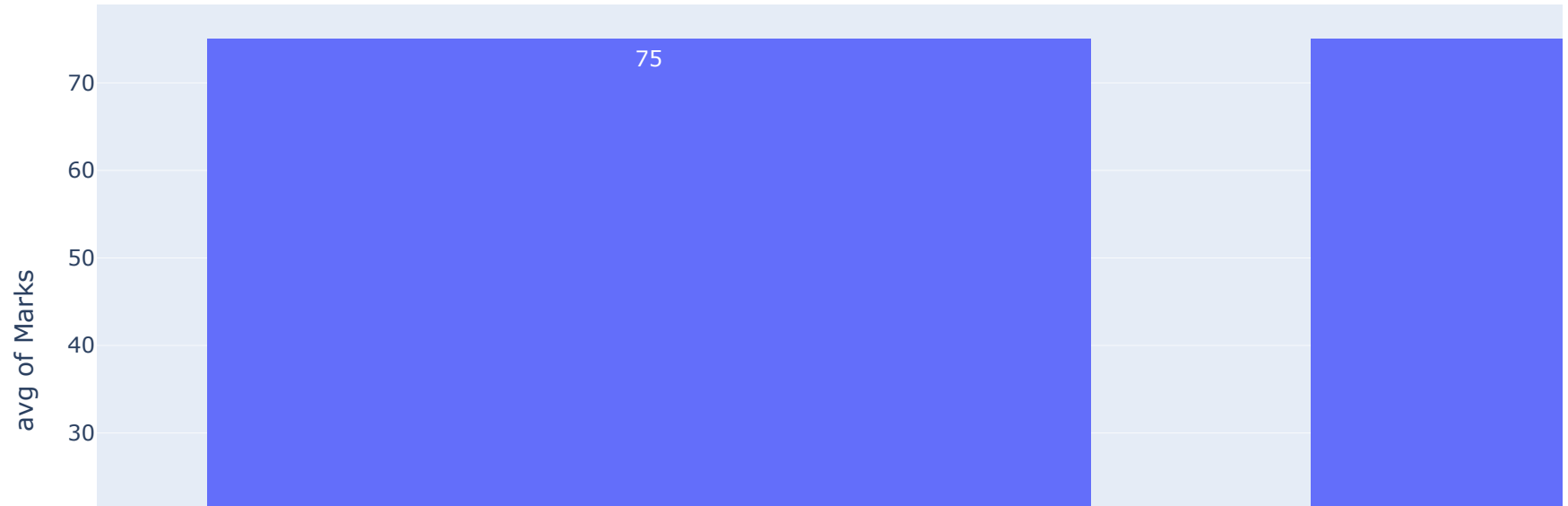
fig = px.histogram(
    df,
    x="Marks",
    nbins=5,
    color="Department",
    text_auto=True,
    title="Marks Distribution"
)

fig.show()
```



```
In [92]: fig = px.histogram(  
    df,  
    x="Department",  
    y="Marks",  
    histfunc="avg",  
    text_auto=True  
)
```

```
fig.show()
```



```
In [428... df=px.data.tips()
```

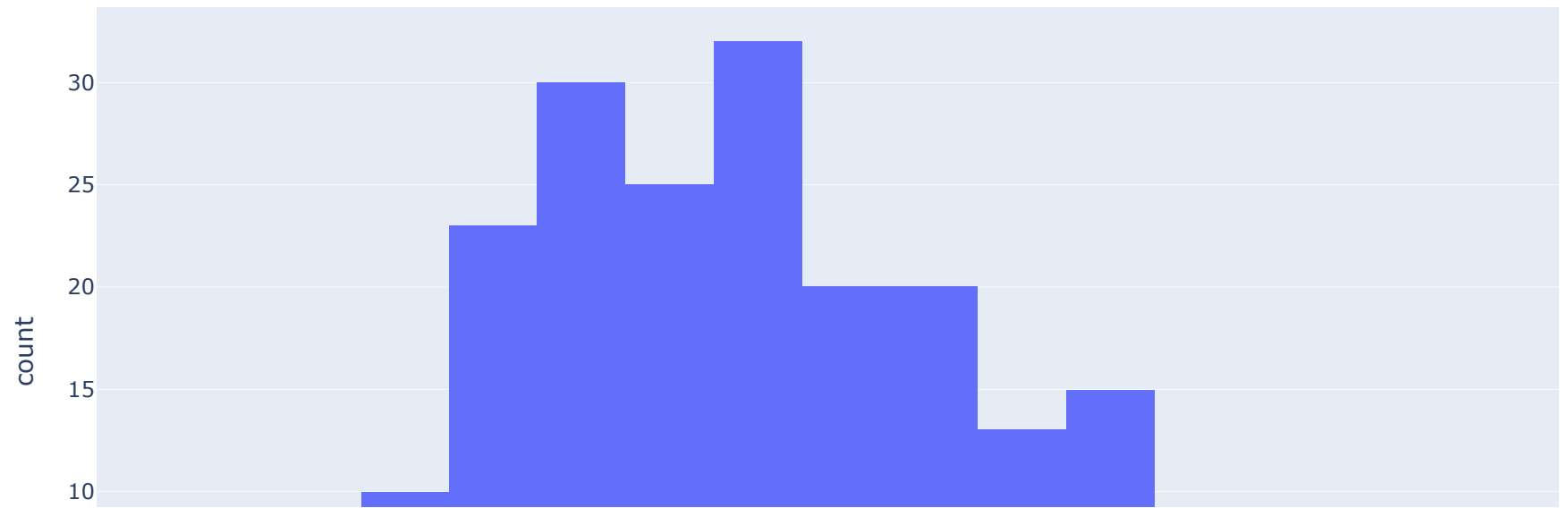
```
In [274... df.head()
```

Out[274...

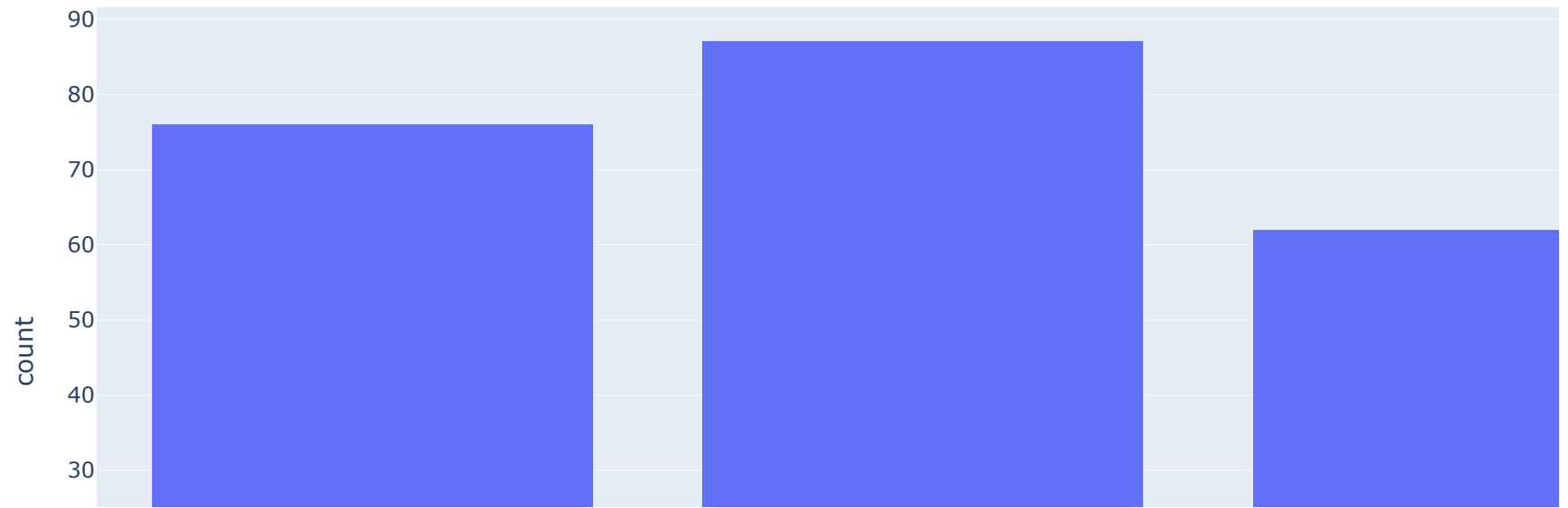
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

In [280...

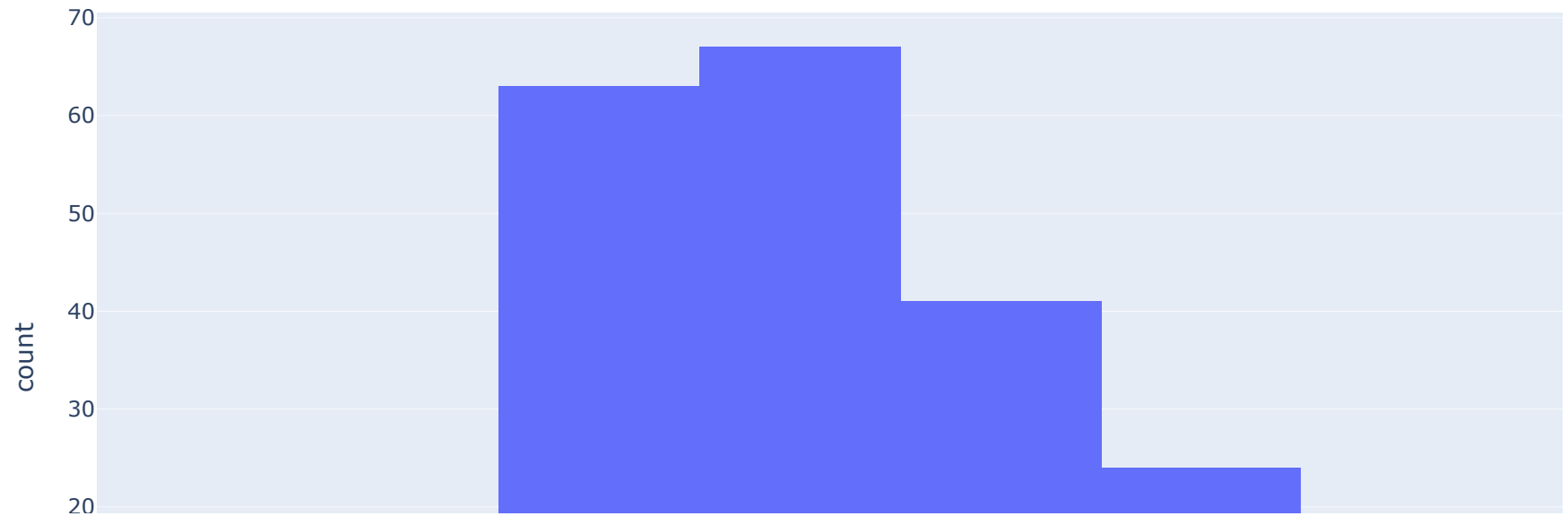
```
fig=px.histogram(df,x="total_bill")  
fig.show()
```



```
In [282... fig=px.histogram(df,x="day")  
fig.show()
```



```
In [284... fig=px.histogram(df,x="total_bill",nbins=20)  
fig.show()
```



```
In [420...] stocks=px.data.stocks()
```

```
In [430...] stocks.head()
```

Out[430...]

	date	GOOG	AAPL	AMZN	FB	NFLX	MSFT
0	2018-01-01	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
1	2018-01-08	1.018172	1.011943	1.061881	0.959968	1.053526	1.015988
2	2018-01-15	1.032008	1.019771	1.053240	0.970243	1.049860	1.020524
3	2018-01-22	1.066783	0.980057	1.140676	1.016858	1.307681	1.066561
4	2018-01-29	1.008773	0.917143	1.163374	1.018357	1.273537	1.040708

In [432...]

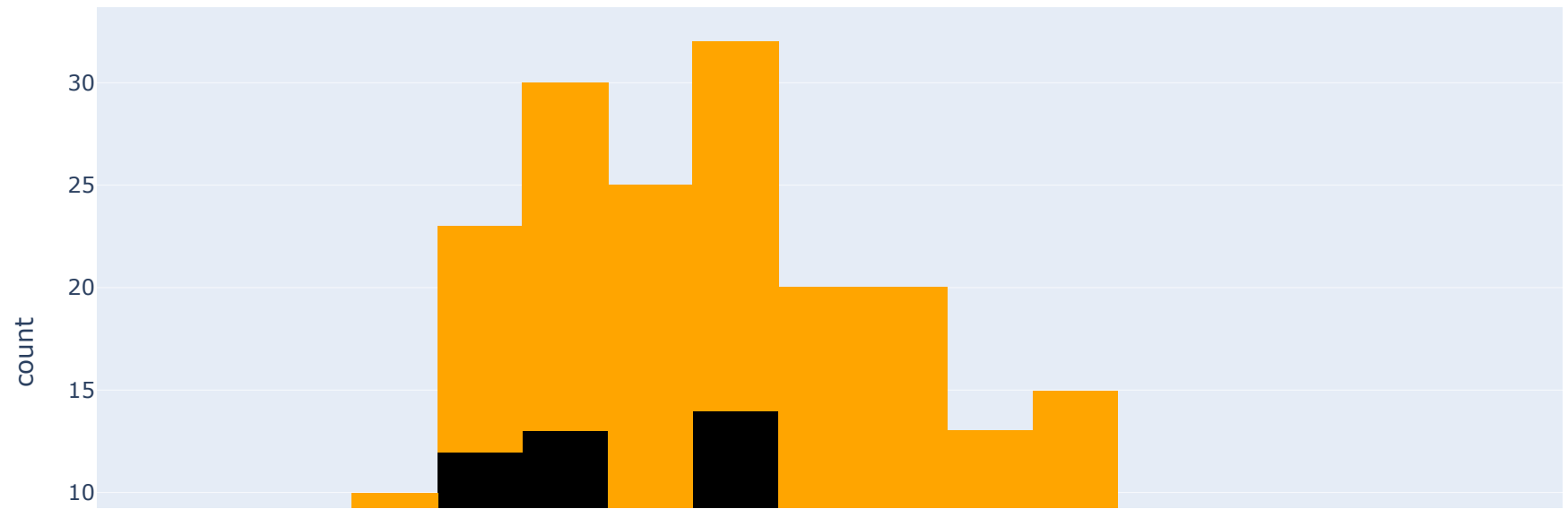
```
fig=px.histogram(stocks,y="date")  
fig.show()
```

Vishal Acharya



In [434...

```
fig=px.histogram(df,x="total_bill",color="sex",color_discrete_sequence=["black","orange"])  
fig.show()
```



```
In [306... fig=px.histogram(df,x="total_bill",y="tip",histfunc="avg")  
fig.show()
```



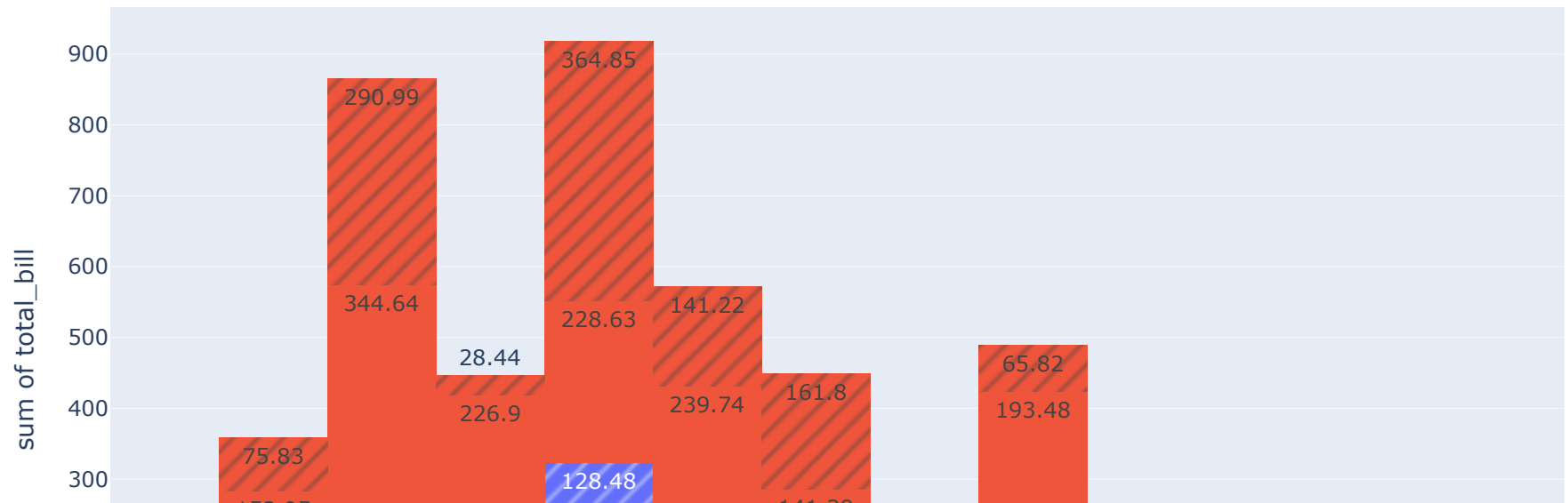
```
In [312... fig=px.histogram(df,x="sex",y="total_bill",color="sex",pattern_shape="smoker")  
fig.show()
```

Vishal Acharya



In [320...

```
fig=px.histogram(df,x="tip",y="total_bill",color="sex",pattern_shape="smoker",text_auto=True)  
fig.show()
```



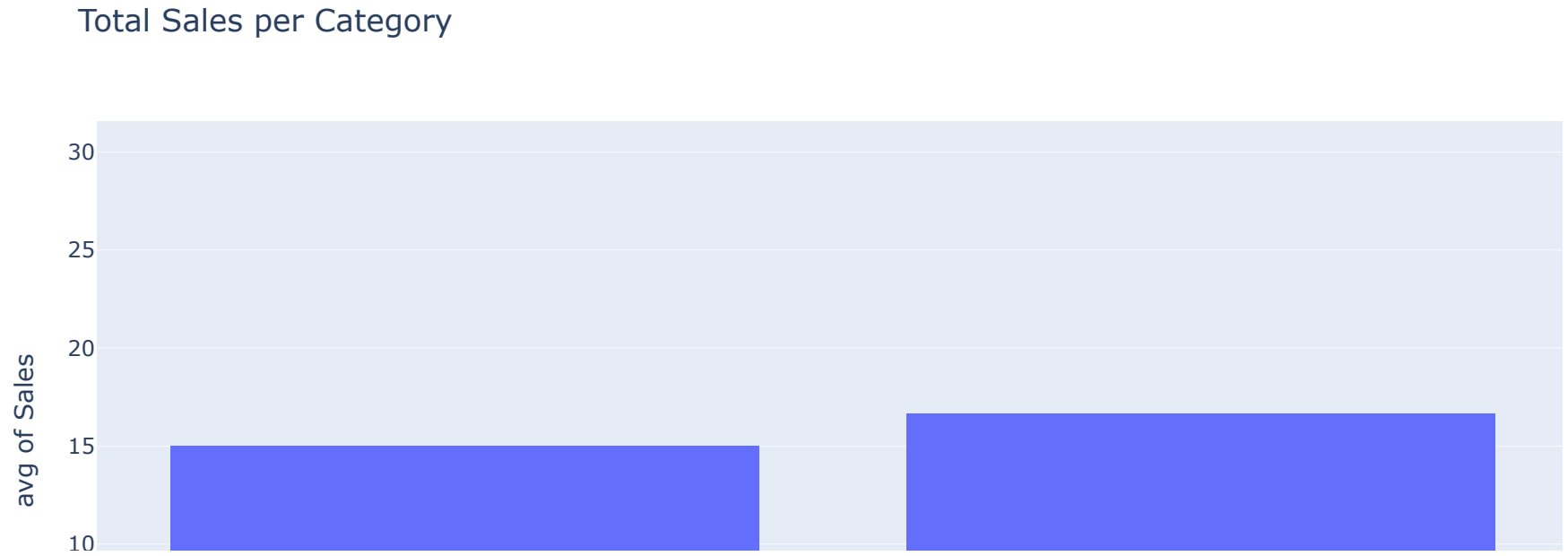
```
In [7]: import plotly.express as px
import pandas as pd

# Sample data
data = {
    "Category": ["A", "A", "B", "B", "B", "C"],
    "Sales": [10, 20, 15, 25, 10, 30]
}
```

```
df = pd.DataFrame(data)

# Histogram with sum
fig = px.histogram(
    df,
    x="Category",
    y="Sales",
    histfunc="avg",
    title="Total Sales per Category"
)

fig.show()
```



```
In [5]: import plotly.express as px
import pandas as pd

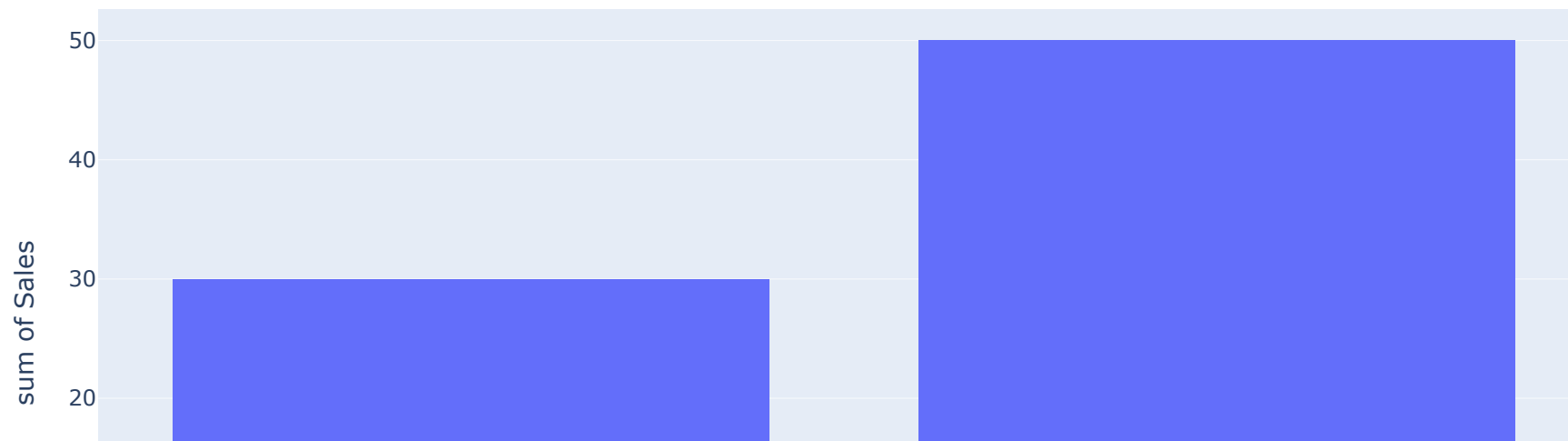
# Sample data
data = {
    "Category": ["A", "A", "B", "B", "B", "C"],
    "Sales": [10, 20, 15, 25, 10, 30]
}
```

```
df = pd.DataFrame(data)

# Histogram with sum
fig = px.histogram(
    df,
    x="Category",
    y="Sales",
    title="Total Sales per Category"
)

fig.show()
```

Total Sales per Category



Histogram

```
In [359... df=pd.read_csv("bar.csv")
```

```
In [361... df
```

Out[361...

	match_id	player	match_format	ground	run	opposite_team	ball	4s	6s
0	1	Virat Kohli	ODI	Ahmedabad	85	Australia	92	8	2
1	2	Rohit Sharma	T20	Mumbai	60	England	38	6	4
2	3	Shubman Gill	ODI	Delhi	110	Australia	105	12	1
3	4	Virat Kohli	T20	Chennai	45	England	30	4	2
4	5	Rohit Sharma	Test	Ahmedabad	132	South Africa	210	14	1
5	6	Shubman Gill	ODI	Mumbai	78	Australia	85	7	3
6	7	Virat Kohli	T20	Delhi	55	South Africa	32	5	3
7	8	Rohit Sharma	ODI	Chennai	95	England	101	9	2
8	9	Shubman Gill	T20	Ahmedabad	70	Australia	40	6	3
9	10	Virat Kohli	Test	Mumbai	120	South Africa	180	13	2
10	11	Rohit Sharma	ODI	Delhi	88	England	90	10	1
11	12	Shubman Gill	T20	Chennai	39	South Africa	28	3	1
12	13	Virat Kohli	ODI	Ahmedabad	102	Australia	98	11	2
13	14	Rohit Sharma	T20	Mumbai	48	England	27	5	2
14	15	Shubman Gill	Test	Delhi	140	South Africa	220	15	2
15	16	Virat Kohli	ODI	Chennai	67	Australia	72	6	1
16	17	Rohit Sharma	T20	Ahmedabad	59	South Africa	35	5	3
17	18	Shubman Gill	ODI	Mumbai	91	England	88	9	2
18	19	Virat Kohli	T20	Delhi	73	Australia	42	7	4
19	20	Rohit Sharma	Test	Chennai	125	South Africa	205	12	3

In [363...

```
fig=px.histogram(df,x="run")
```

```
fig.show()
```



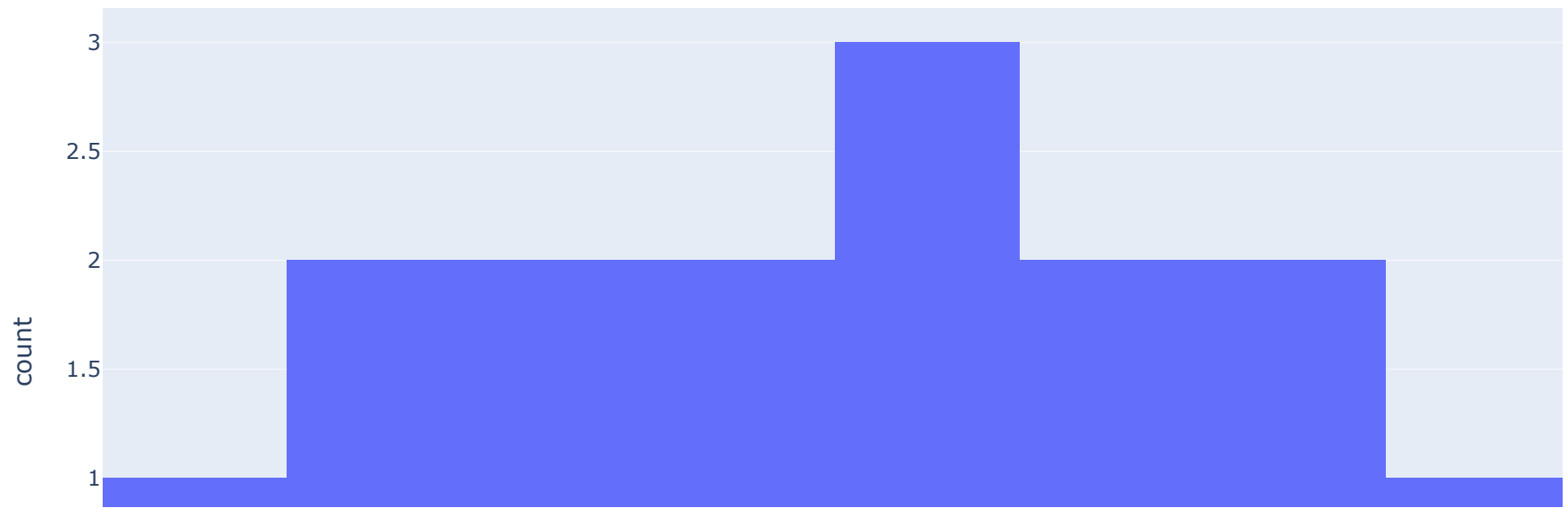
In [365...

```
fig=px.histogram(df,x="ground")  
fig.show()
```

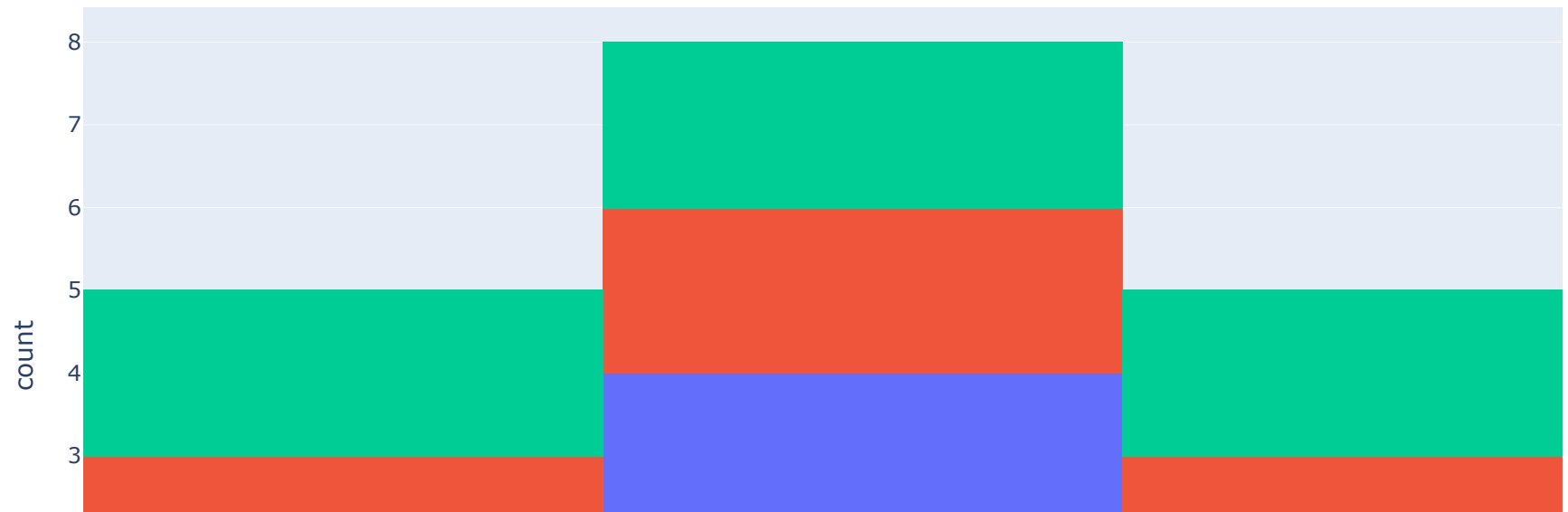


```
In [367... fig=px.histogram(df,x="run",nbins=16)  
fig.show()
```

Vishal Acharya

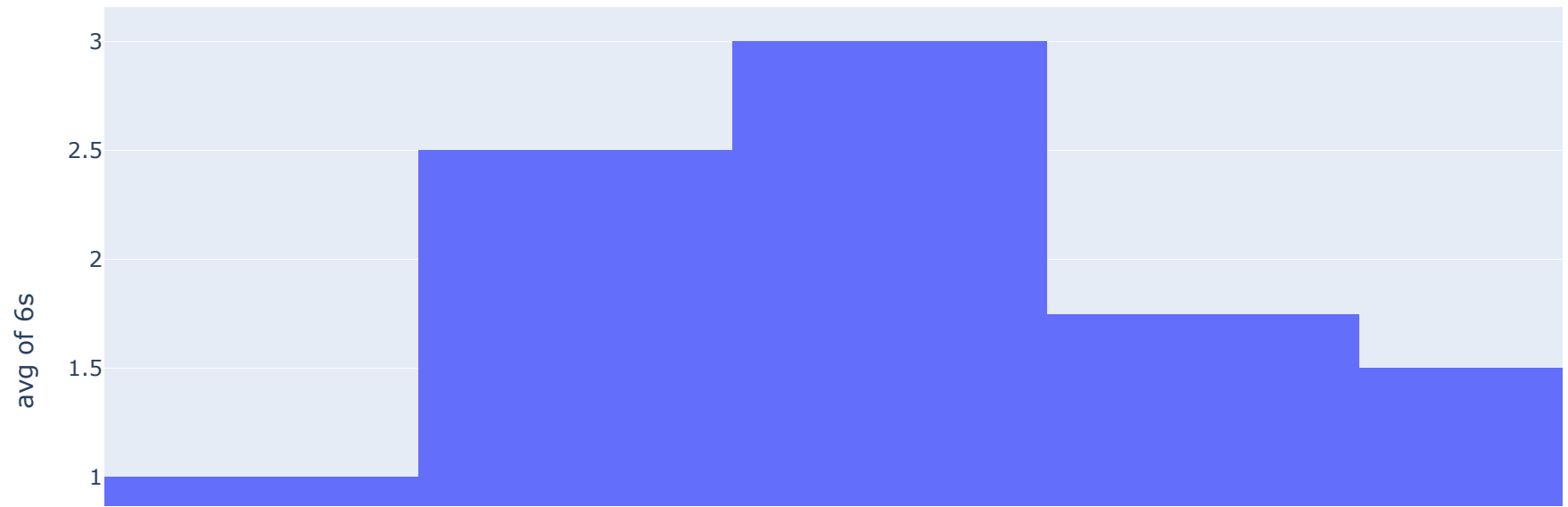


```
In [369... fig=px.histogram(df,x="6s",color="player")  
fig.show()
```



```
In [371... fig=px.histogram(df,x="run",y="6s",histfunc="avg")  
fig.show()
```

Vishal Acharya



In [373...

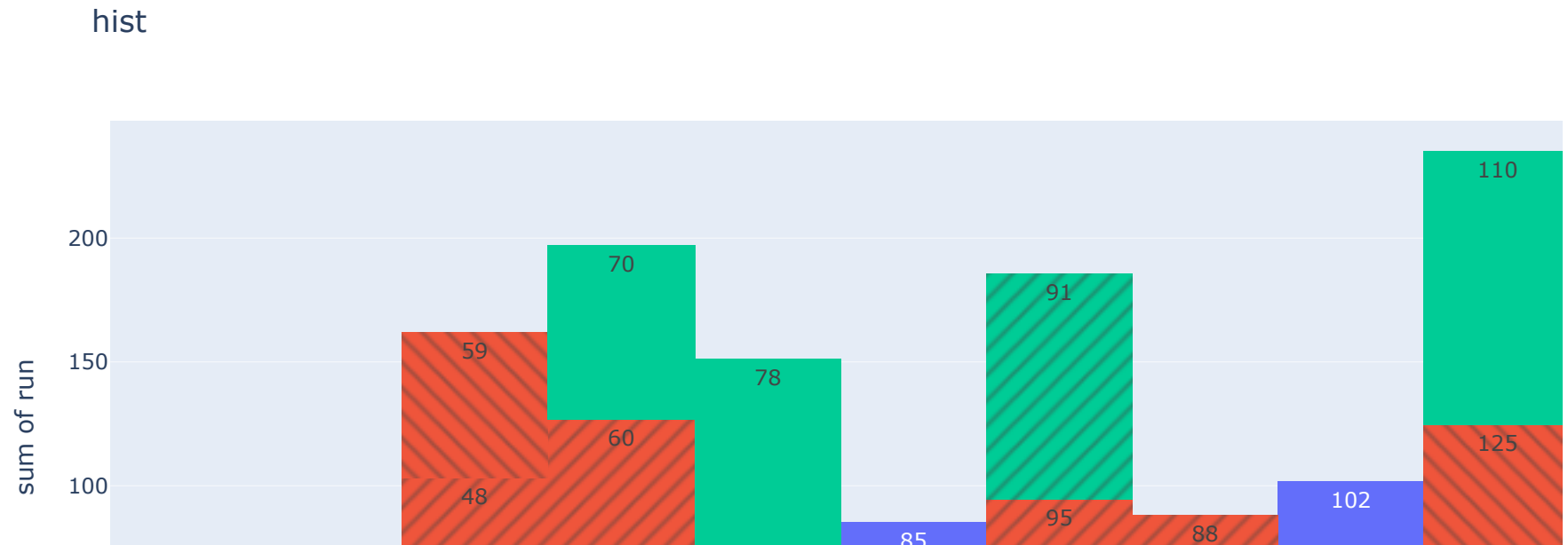
```
px.histogram(df,x="ground",y="run",color="player",pattern_shape="opposite_team")
```

Vishal Acharya



In [391...

```
px.histogram(df,x="4s",y="run",color="player",pattern_shape="opposite_team",nbins=17,text_auto=True,title="hist")
```



Pie chart

A Pie Chart shows proportions or percentages of categories in a dataset, where each slice represents a category's contribution to the total.

```
fig = px.pie(data, values, names, color_discrete_sequence, title) fig.update_traces(textinfo="percent+label", textposition, pull)
```

px.pie() Parameters

data – DataFrame containing the dataset.

values – numeric column used to determine slice size.

names – column used for category labels in the pie chart.

color_discrete_sequence – list of colors applied to slices.

title – adds title to the pie chart.

fig.update_traces() Parameters

textinfo="percent+label" – displays percentage and category label on each slice.

textposition – controls position of text (inside / outside slices).

pull – separates or explodes slices outward from the pie.

Parameter	Use
values	determines slice size
names	category labels
color_discrete_sequence	slice colors
title	chart title
textinfo	shows label/percent
textposition	text location
pull	explode pie slices

```
In [394... df=pd.read_csv("social_media_engagement.csv")
```

```
In [398... data=df.groupby("platform")["likes"].sum().reset_index()
```

```
In [400... data
```

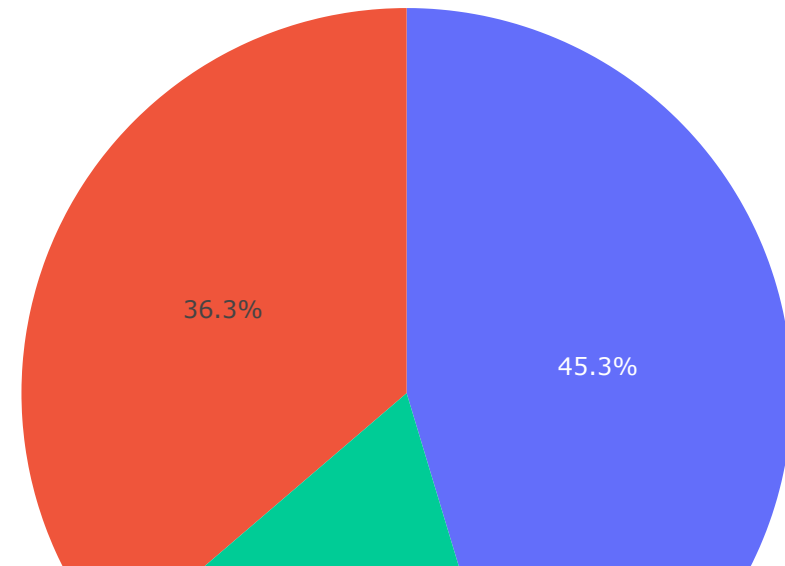
```
Out[400...

|   | platform  | likes  |
|---|-----------|--------|
| 0 | Facebook  | 86392  |
| 1 | Instagram | 107994 |
| 2 | Twitter   | 43795  |


```

```
In [402... px.pie(data,values="likes",names="platform")
```

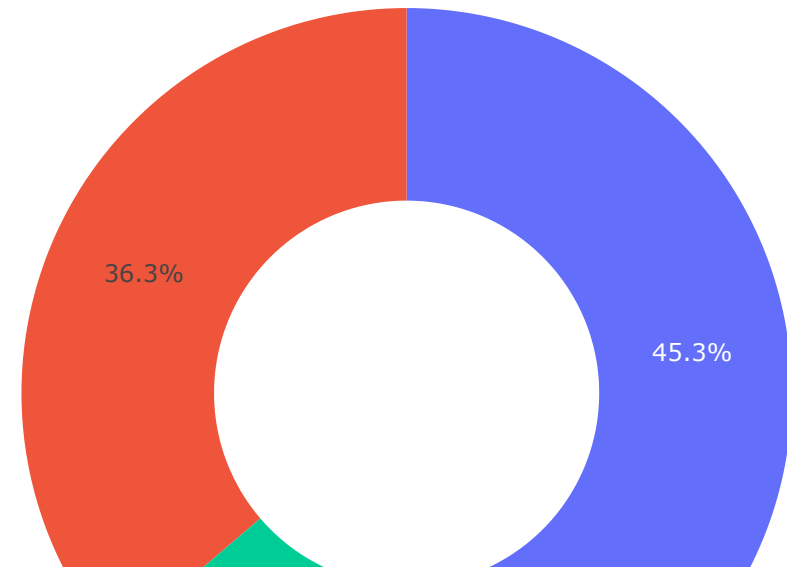
Vishal Acharya



In [404...

```
px.pie(data, values="likes", names="platform", hole=0.5)
```

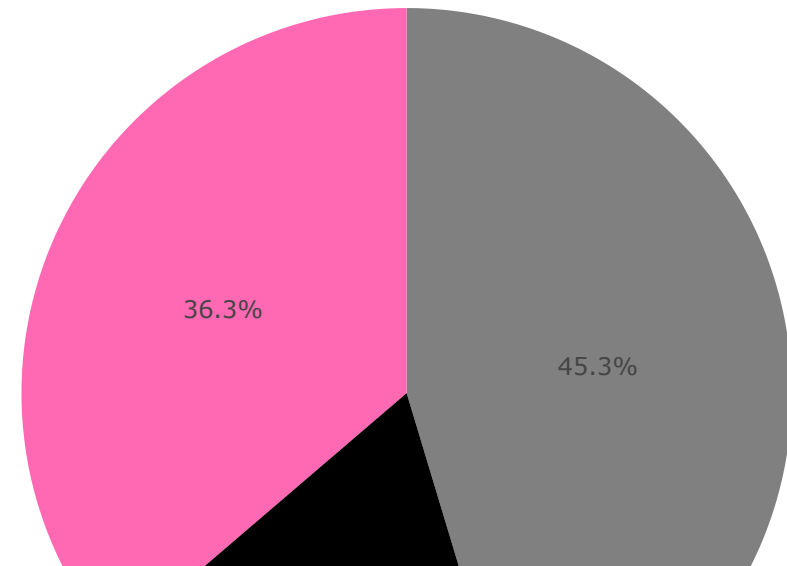
Vishal Acharya



In [406...

```
px.pie(data, values="likes", names="platform", color_discrete_sequence=["gray", "hotpink", "black"])
```

Vishal Acharya

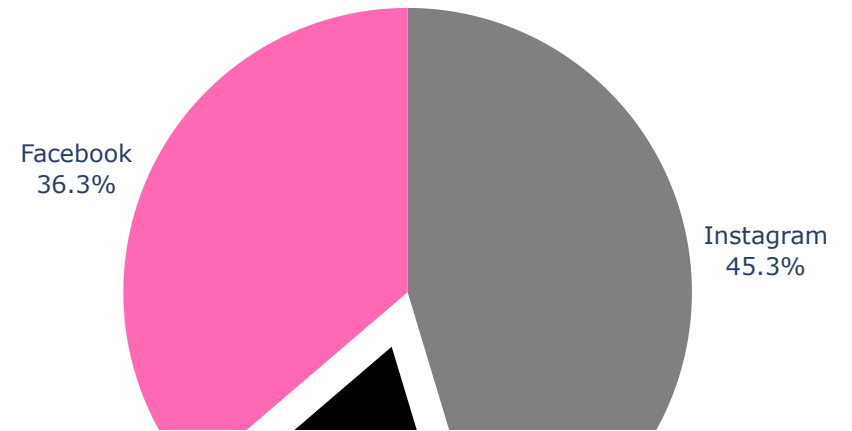


In [412...

```
fig=px.pie(data,values="likes",names="platform",color_discrete_sequence=["gray","hotpink","black"],title="pie")  
fig.update_traces(textinfo="percent+label",textposition="outside",pull=[0,0,0.2])
```

Vishal Acharya

pie



In [414...

```
df=pd.read_csv("bar.csv")
data=df.groupby("player")["run"].sum().reset_index()
data
```

Out[414...]

	player	run
0	Rohit Sharma	607
1	Shubman Gill	528
2	Virat Kohli	547

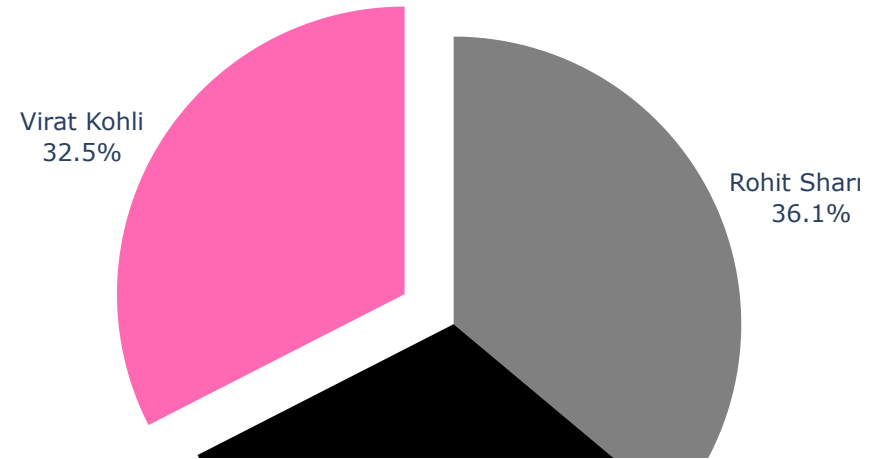
In [416...]

```
fig=px.pie(data,values="run",names="player",color_discrete_sequence=["gray","hotpink","black"],title="pie")
fig.update_traces(textinfo="percent+label",textposition="outside",pull=[0,0,0.2])

fig.show()
```

Vishal Acharya

pie

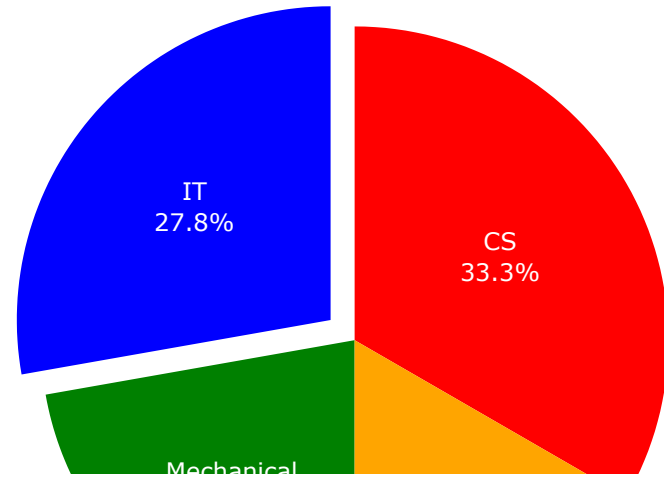


```
In [93]: import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    "Department": ["CS", "IT", "Mechanical", "Civil"],
    "Students": [120, 100, 80, 60]
})
```

```
fig = px.pie(  
    df,  
    values="Students",  
    names="Department",  
    title="Students by Department",  
    color_discrete_sequence=["red", "blue", "green", "orange"]  
)  
  
fig.update_traces(  
    textinfo="percent+label",  
    textposition="inside",  
    pull=[0,0.1,0,0]  
)  
  
fig.show()
```

Students by Department



In [444...]

```
import dash
from dash import html, dcc
import plotly.express as px
import pandas as pd

# Sample Data
data = {
    "Player": ["Kohli", "Rohit", "Hardik"],
```

```
        "Runs": [78, 65, 92]
    }

df = pd.DataFrame(data)

# Create Plotly Figure
fig = px.bar(df, x="Player", y="Runs", title="Player Runs")

# Create Dash App
app = dash.Dash(__name__)

app.layout = html.Div([
    html.H1("Simple Dash App"),
    dcc.Graph(figure=fig)
])

if __name__ == "__main__":
    app.run(debug=True)
```

Vishal Acharya



<http://127.0.0.1:8050/>

NetworkX is a Python library used to create, analyze, and visualize graphs (networks) made of nodes (vertices) and edges (connections).

Parameter	Meaning
<code>with_labels=True</code>	Show node names
<code>node_size=1000</code>	Size of nodes
<code>node_color='red'</code>	Node color
<code>width=5</code>	Edge thickness
<code>edge_color='blue'</code>	Edge color

Function	Purpose
<code>nx.Graph()</code>	Create undirected graph
<code>nx.DiGraph()</code>	Create directed graph
<code>add_node()</code>	Add single node
<code>add_nodes_from()</code>	Add multiple nodes
<code>add_edge()</code>	Add single edge
<code>add_edges_from()</code>	Add multiple edges
<code>nx.draw()</code>	Visualize graph

Simple Undirected Graph

Creates a basic graph with nodes and edges

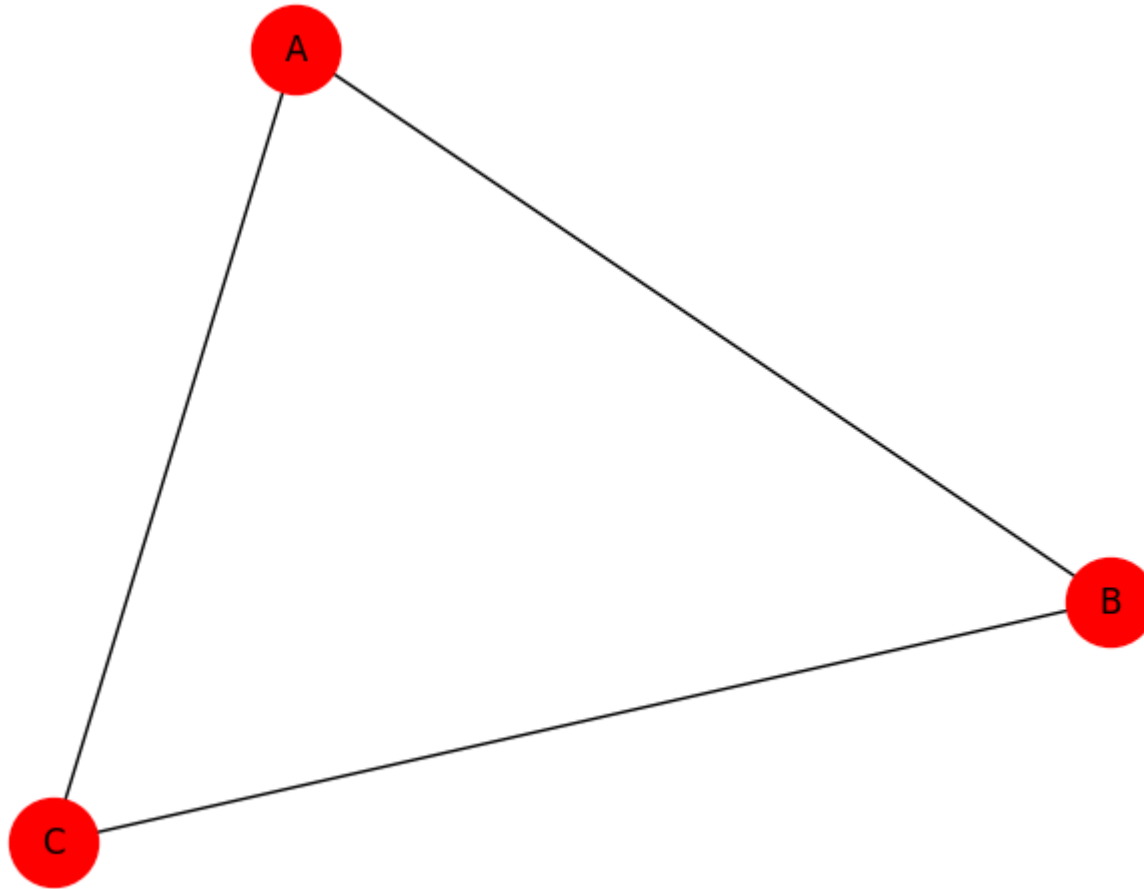
Purpose: Creates a simple network where all nodes are connected

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

G.add_nodes_from(['A', 'B', 'C'])
G.add_edges_from([('A', 'B'), ('B', 'C'), ('A', 'C')])

nx.draw(G, with_labels=True, node_size=1000, node_color='red')
plt.show()
```



Directed Graph

Shows direction between nodes.

Purpose:

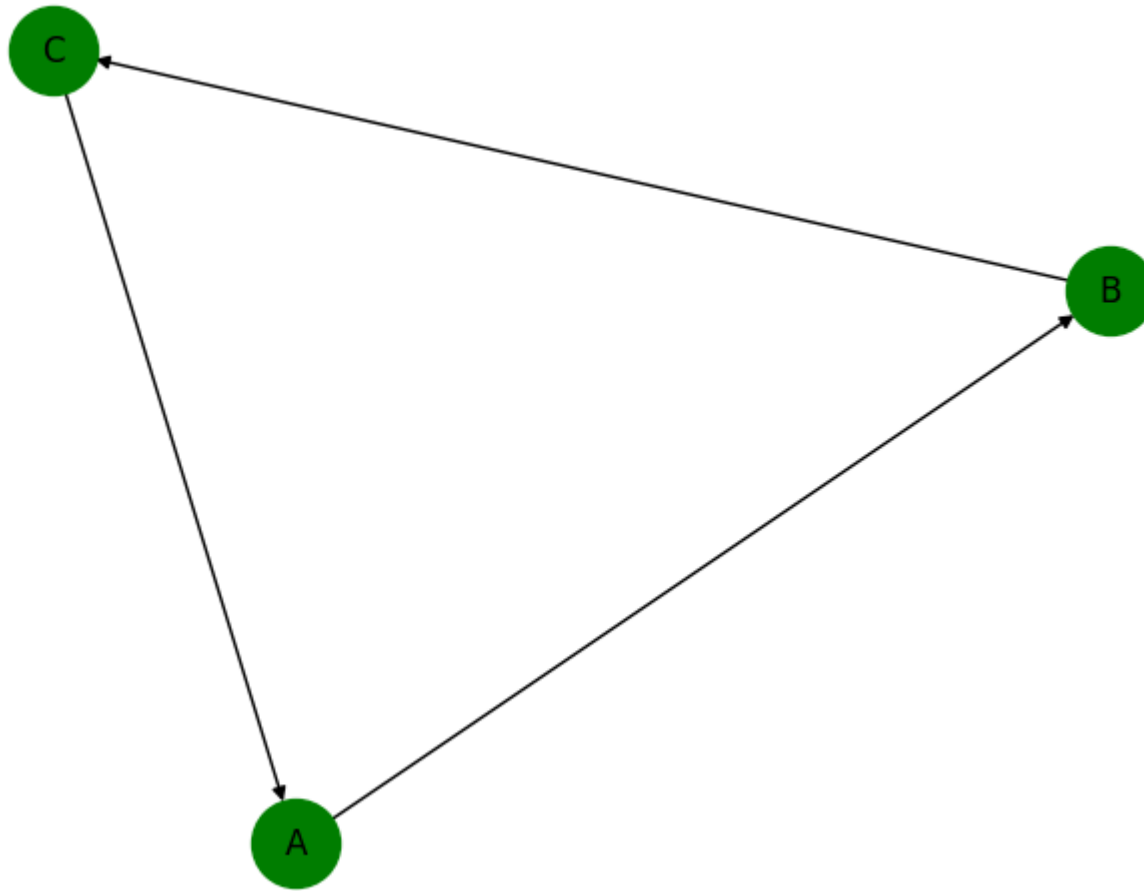
Used for one-way relationships like website links or followers.

```
In [2]: import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_edges_from([('A', 'B'), ('B', 'C'), ('C', 'A')])

nx.draw(G, with_labels=True, node_color='green', node_size=1000)
plt.show()
```



To create a Connection Matrix Plot (Adjacency Matrix Visualization) we usually visualize the matrix as a heatmap, where:

Rows = Nodes

Columns = Nodes

Value (0 or 1) = Connection between nodes

Since you didn't include the matrix yet, I'll show how to plot it and you can replace it with your matrix.

	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	0
D	0	1	0	0

```
In [4]: import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

# Connection Matrix
data = [
    [0,1,1,0],
    [1,0,1,1],
    [1,1,0,0],
    [0,1,0,0]
]

nodes = ["A", "B", "C", "D"]

# Create DataFrame
df = pd.DataFrame(data, index=nodes, columns=nodes)
```

```
# Create graph
G = nx.Graph()

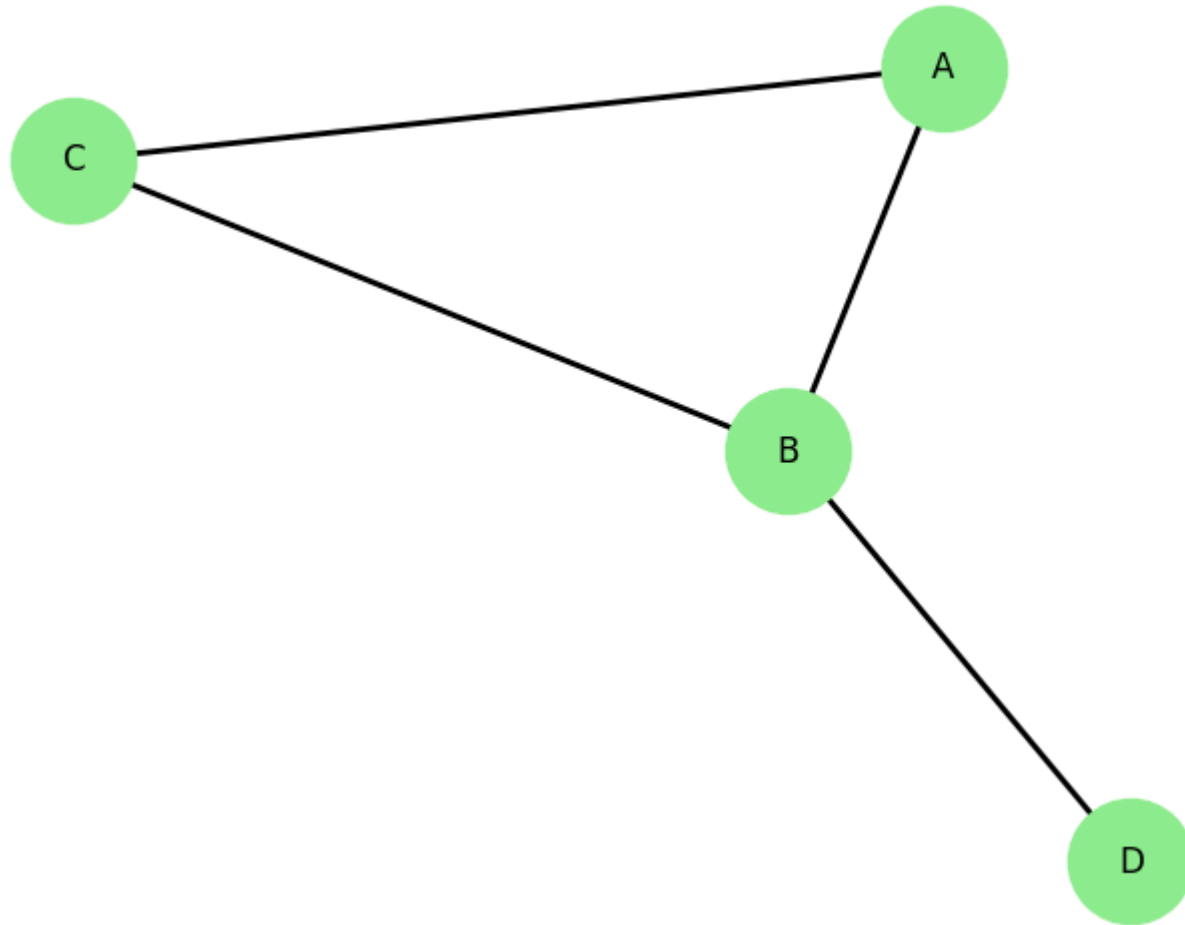
# Add nodes
G.add_nodes_from(nodes)

# Add edges based on matrix values
for i in range(len(nodes)):
    for j in range(len(nodes)):
        if df.iloc[i, j] == 1:
            G.add_edge(nodes[i], nodes[j])

# Draw graph
nx.draw(G,
        with_labels=True,
        node_size=2000,
        node_color='lightgreen',
        edge_color='black',
        width=2)

plt.title("Network Graph from Connection Matrix")
plt.show()
```

Network Graph from Connection Matrix



In []: